

Subsequence-Centric Methods for Enhancing
Time Series Analysis: Encoding, Piecewise
Similarity, and Covariate Augmentation
サブシーケンス中心の手法による時系列解析
の強化：エンコーディング、区間類似性、共
変量による拡張

Coleman Yu

余 浩旻

Abstract

Time series data are ubiquitous across many different fields. Much of the data are inherently time series data. Additionally, some data, such as strings, images, and object shapes, that are not originally time series data, can be transformed into time series. Many data mining tasks, such as classification, clustering, and motif finding, have been defined for time series data. Hence, by developing appropriate transformation methods, we can apply a plethora of well-established time series methods to our problems.

This thesis contributes three aspects in time series data mining and bioinformatics. They are a novel application of time series classification to solve complex biological problems by transforming biological data into time series, developing a more expressive distance measure framework that removes certain underlying assumptions, and using nearest neighbors as covariants to improve the performance in the task of forecasting.

In the first part, we demonstrate the utility of time series analysis in bioinformatics by studying the problem of predicting Human Dicer Cleavage Sites. Recall that bioinformatics operates at the intersection of biology, biotechnology, and informatics. In this work, we formulate a specific biology problem, which is predicting Human Dicer Cleavage sites in microRNA biogenesis, into a machine learning framework. In particular, this is a multivariate time series classification problem, which is the informatics component of bioinformatics. Due to current limitations in biotechnology, we are constrained to using 1-D RNA sequence inputs rather than 2-D or 3-D data, because these are more expensive to obtain. We propose MTSCCLeav, a method that encodes RNA sequences and the probabilities of base pairs in predicted secondary structures into time series data. To the best of our knowledge, we are the first to make use of the probabilities of base pairs in this kind of classification task on RNA data. By doing this, we frame the problem of predicting Human Dicer Cleavage sites into a Multivariate Time Series Classification (MTSC) problem. Existing approaches rely on opaque deep neural networks or complex feature engineering. They are slow, and the

feature engineering is over-designed. In contrast, our approach is simple, intuitive, and computationally efficient. The proposed transformation methods allow us to use any well-established time series tools to analyze this biological problem. Experiments demonstrate that MTSCCleave achieves comparable and even better accuracy to state-of-the-art methods while delivering a 3.7X to 28.8X speedup. Furthermore, our perturbation experiments reveal that regions near the center of pre-miRNAs are essential for cleavage-site prediction, consistent with the existing literature.

In the second part, we address the limitations of existing similarity measures. Similarity search is a core subroutine in time series data mining tasks. For example, recent studies show that a simple 1-NN classifier with an appropriate distance measure can outperform many advanced, complicated methods. While Dynamic Time Warping (DTW) and Uniform Scaling (US) are prevailing measures for handling local distortions and global scaling, respectively, and some studies have demonstrated that combining both DTW and US is necessary to obtain meaningful results. Current approaches apply a single scaling factor to the entire sequence. We argue that since distinct phases of a process often evolve at different speeds, a single scaling factor is insufficient. We introduce the first distance measure framework, namely PSD, that achieves invariance to multiple scaling factors. We also provide speed-up techniques to enable efficient computation of the PSD. Experiments show that PSD better reflects the similarity between time series with multiple phases, and that the identified phases (segmentation) provide a clearer understanding of the data.

In the third part, we study a time series primitive, namely matrix profile, in the application of time series forecasting. For a given time series T and an integer m , the matrix profile provides the information about (1) the location of its nearest neighbor and (2) the distance between it and the nearest neighbor of **each** m -subsequence in T . Obviously, the matrix profile also provides us the information for the k -nearest neighbors of each subsequence simply by running the same algorithm for k times. In our application, we are only interested in the left nearest neighbors of each subsequence because they refer to the historical occurrences. We can obtain the immediate subsequences of these historical occurrences as covariates to feed into the forecaster to improve its accuracy. Experiment shows that these covariates, which can be obtained through a computationally cheap data mining primitive called Matrix Profile, can improve the existing forecaster.

This thesis develops three model-agnostic, subsequence-focused techniques to advance time series processing—by encoding domain-specific biological sequences

into multivariate time series for classification, proposing efficient piecewise scaling for more flexible similarity computation, and generating subsequence-derived exogenous covariates to improve forecasting accuracy—demonstrating how targeted exploitation of local patterns can benefit diverse downstream tasks from bioinformatics to general time series modeling.

Acknowledgements

To begin, I want to express my deepest gratitude to my PhD supervisor at Kyoto University, Prof. Tatsuya Akutsu. I thank him for his kindness and patience. He is a brilliant researcher who can explain complex concepts simply. In a seminar, he showed that simply using clear notation makes presentations much easier to follow. More importantly, he helps students when they are in “valleys” in their lives. When a car is off track, it needs a push—so do people. I am equally indebted to my MPhil supervisor at The Hong Kong University of Science and Technology (HKUST), Prof. Raymond Chi-Wing Wong. He has continued to help and advise me even after I graduated from HKUST.

Akutsu-sensei taught me the importance of reading good materials. In the weekly seminars, he organized reading seminars, assigning some of his favorite books for us to read and discuss. His favorite book is “Probability and Computing” by Mitzenmacher and Upfal. Additionally, through the journal club he organized, students were asked to present the core ideas from their ten selected papers published in leading venues, such as Nature, Science, Cell, and PNAS. Reading beyond my fields helped me identify research gaps others missed. I think some great ideas are born when we merge the ideas from different fields. He also emphasized the importance of mathematics and proof. It is what computer scientists can stand out in the field of bioinformatics. I think it becomes especially true now, since advances in AI make it possible for almost everyone to code (i.e., Vibe coding). He advised me to use the existing tools, libraries, and studies to leverage my own research. He also taught me that a good researcher must master both oral and written presentation; otherwise, others may misinterpret your talent and the effort you put in. When we are reading a paper, he urges us to identify the novelty and analyze the pros and cons of the paper.

Raymond taught me the power of focus and “First Principles”. I was always surprised that he did not use reference management software like Zotero, preferring to annotate hard copies and even type .bib files manually. He told me that when he starts to do research, he will print out the papers and get focused on the

stack of papers (hard copies) in front of him. I am not saying it is beneficial not to use tools, but I want to emphasize the power of focus that underlies his work routine. He taught me that every good research starts with a set of good papers. He also emphasized that there is no right or wrong in research, only what you choose to do about it. His research receipt works as follows. When you are tackling a problem, you first review the relevant existing studies to understand the state of the art (SOTA) for it. If the existing work addresses your particular problem, you can apply and adapt it to your problem setting. But most of the time, since your problem must be a particular version of a general problem, the existing general solution should not work well for it. It means that you have some room to improve it. And this is the research gap!. It reminds me of when we deal with the NP-complete problem. As Kleinberg and Tardos's Algorithm Design (Chapter 10) suggests, an NP-complete problem (assuming $P \neq NP$) does not allow us to have an algorithm that possesses all three of the following desired properties simultaneously: Efficiency, Correctness, and Generalization. Hence, it is sometimes preferable to address a specific instance of the problem rather than the general one. He also emphasized the theory. He consistently noted that you need to add theory to the paper to strengthen it. He always mentioned that you need three motivations (why you are doing it) and three contributions (what you have done). Sometimes, I ask why he can write so fast, and he replies, "If you know what you are doing, then you write fast.". He also taught me that when you don't know some of the ideas, you just go back to the original idea. Doing a "deep first search" can help you reach the first principle, a concept that Elon Musk, one of the greatest entrepreneurs of our time, always emphasized.

Without these two supervisors, I would not have made it this far. I could not have finished this program.

I would like to take this opportunity to share some of my thoughts on my PhD journey. I hope the readers may learn something from what I have gone through. My PhD journey was, to quote Dickens, "the best of times and the worst of times", and to quote Churchill, "This was their finest hour". For the best parts, it allows me to explore both in my daily life and in my research. I tried many things, walked many roads, drank many colas and alcohol, and met many people. This helped me see problems from new perspectives. Regarding the worst parts, I am reminded of a quote from one of my favorite movies, "Les Choristes": "Fond de l'étang". It literally means "Bottom of the Pond". At times, I felt like a frog at the well's bottom, trying hard to get out. Research is fun but also hard. Research is about exploring something new. It is about publishing (so others can learn from

it). When I am stuck, the best solution is to aim for a reachable, well-defined goal. The goal should be clear, with obvious rewards and requirements. Also, make sure the effort of your actions can be accumulated. Like the frog, do not jump randomly, but aim to move to stable platforms towards escape. Then, each jump matters for your progress. Two of the materials helped me; they are “Eat the Frog!” and “THE PH.D. GRIND”.

I would like to thank my thesis committee members, Prof. Tatsuya Kawahara and Prof. Hisashi Kashima. I appreciate their willingness to serve as committee members, to provide me with valuable advice. I would like to acknowledge Tamura-sensei and Mori-sensei (a former member) of the Akutsu laboratory. Tamura-sensei taught me to focus on the input-output relationship when encountering new algorithms/methods, and Mori-sensei introduced me to the fascinating application of time-series analysis to gene expression data, especially in trajectory inferences (i.e., pseudotime). I would like to thank the many communities that made my life in Japan colorful. Thank you to the people of the dormitories where I lived at the Uji and Yoshida campuses. I am grateful to my Japanese language teachers. I also cherish the friends I met through Akutsu’s laboratory and the extracurricular activities, including Kendo, Table Tennis, Naginata, Karate, and Aikido. I also met many interesting people from various international student events.

I am honored to receive the Asian Future Leaders Scholarship Program (AFLSP) scholarship to support my studies in Japan. I have made a great choice by enrolling in Kyoto University Design School, which has provided me with many valuable interdisciplinary experiences and opportunities to meet people outside my field. In this program, I have conducted fieldwork in Okinawa, Hong Kong, and Bali, a rare opportunity for researchers in computer science. I would also like to thank the Institute for Chemical Research (ICR) for the Research Assistant position in the Akutsu laboratory.

Finally, I would like to express my deepest gratitude to my family and my friends who have stayed by my side with their countless acts of support; I have nothing to return to them but my love and time.

A special acknowledgment goes to my niece. As you grow up, I hope you explore the world and fulfill your eagerness for knowledge. Find materials that interest you. One day, you might find this thesis online and, I hope, find it worth reading and inspiring.

List of Publications

This thesis is based on the following papers.

- (Chapter 3) **Coleman Yu**, Raymond Chi-Wing Wong, and Tatsuya Akutsu, “MTSCClev: a Multivariate Time Series Classification (MTSC)-based Method for Predicting Human Dicer Cleavage Sites”, submitted to *IEEE Access*, under review
- (Chapter 4) **Coleman Yu**, Tatsuya Akutsu, and Raymond Chi-Wing Wong, “Scaling with Multiple Scaling Factors and Dynamic Time Warping in Time Series Searching”, submitted to *IEEE Access*, under review
- (Chapter 5) **Coleman Yu**, Raymond Chi-Wing Wong, and Tatsuya Akutsu, “Leveraging Nearest Neighbors for Time Series Forecasting with Matrix Profile”, in preparation

Other publications

- **Coleman Yu** and Raymond Chi-Wing Wong, “A Melody Composer for both Tonal and Non-Tonal Languages”, the 43rd International Computer Music Conference 2017, Shanghai, China on 16-20 Oct, 2017
 - In this study, we apply a data mining method called frequent pattern mining to capture the relationships between the pitch trend in the melody and the tone trend in lyrics and use these relationships to create a new melody for the user-given lyrics. The pitch trend and the melody trend are both time series data. It motivates me to conduct research about time series analysis from a data mining perspective. Through this study, I have a chance to apply the data mining techniques in a completely different field.
- Yi Zheng, Bogdan Enescu, Jiancang Zhuang, and **Coleman Yu**, “Data replenishment of five moderate earthquake sequences in Japan, with semi-automatic cluster selection”, *Earthquake Science*, 34:310-322, 2021

- In this study, we apply a data mining method called DBSCAN, which is a clustering method, on seismicity data, to automatically select the nearest significant earthquake cluster of a given mainshock. The clustering results are then fed into a downstream replenishment method to discover missing early aftershocks, which follow relatively large or moderate earthquakes.

Poster presentations

- **Coleman Yu** and Tatsuya Akutsu, “Aligning gene expression time series with invariance to uniform scaling with multiple scaling factors”, International Workshop on Bioinformatics and Systems Biology, Boston, USA (IBSB 2018) on 16-18 July, 2018

- In this study, we present an idea that, for time series analysis, rather than focusing on the whole sequence analysis, it is more important to focus on the subsequences of a time series. In this poster, we use gene expression data as an example to explain. Genes are expressed over time. But the expression rate is not a constant. The varying rate would be discussed in Chapter 4. The importance of the subsequence has been discussed in the framework of time series forecasting in Chapter 5. We use a data mining primitive called Matrix Profile. Given a subsequence Q with length m , we can find the nearest neighbor (location and the similarity of it with Q) in another time series A . The underlying distance metric is z-normalized ED. If Q is an m -subsequence extracted from A , it means that we annotate each m -subsequence with its nearest neighbor information. To demonstrate that this nearest-neighbor information is useful, we use it to improve a time-series forecasting model. If information is useful covariates, the performance can be improved. In addition, the usage of finding useful covariates to improve the final prediction results has also been demonstrated in Chapter 3. In that example, we find a covariate, which is the secondary structure, associated with the probability of each base pair, for the mRNA sequence data, as shown in Figure 3.1.

Contents

Abstract	i
Acknowledgements	v
List of Publications	ix
List of Figures	xv
List of Tables	xix
1 Introduction	1
1.1 Background	1
1.2 Contributions	2
1.3 Organization	2
2 Preliminaries	5
2.1 Distance Measures	5
2.1.1 Euclidean Distance (ED)	6
2.1.2 Dynamic Time Warping (DTW)	7
2.1.3 Derivative Dynamic Time Warping (DDTW)	7
2.1.4 Weighted Dynamic Time Warping (WDTW)	7
2.1.5 Weighted Derivative Dynamic Time Warping (WDDTW)	8
2.1.6 Shape Dynamic Time Warping (shapeDTW)	8
2.1.7 Amercing Dynamic Time Warping (ADTW)	9
2.2 Classification	10
2.2.1 Ridge Classifier	10
3 MTSCCleav: a Multivariate Time Series Classification (MTSC)- based Method for Predicting Human Dicer Cleavage Sites	13
3.1 Background	14

3.2	Methods	17
3.2.1	Data Preparation	18
3.2.2	Time Series Encoding	21
3.2.3	Time Series Classification	25
3.2.4	Evaluation Metrics	28
3.3	Results	29
3.3.1	Channel Ablation Study	30
3.3.2	Predictive Performance	32
3.3.3	Running Time Analysis	34
3.3.4	Subsequence Importance	35
3.3.5	Summary	35
3.4	Discussion	36
3.5	Concluding Remarks	37
4	Scaling with Multiple Scaling Factors in Time Series Searching	39
4.1	Background	40
4.2	Related Work	43
4.3	Preliminaries	44
4.3.1	Euclidean Distance (ED)	45
4.3.2	Dynamic Time Warping (DTW)	45
4.3.3	Uniform Scaling (US)	48
4.3.4	Uniform Scaling & Dynamic Time Warping (USDTW)	49
4.3.5	Lower Bounds for DTW and USDTW	49
4.4	Piecewise Scaling Distance	53
4.4.1	Piecewise Scaling & Dynamic Time Warping (PSDTW)	55
4.4.2	Speedup Techniques	57
4.4.3	Guided Distance	62
4.5	Experiments	62
4.5.1	Experimental Setup	62
4.5.2	Experimental Results	65
4.6	Concluding Remarks	69
5	Leveraging Nearest Neighbors for Time Series Forecasting with Matrix Profile	71
5.1	Background	72
5.2	Related Work	75
5.2.1	Traditional methods	75

5.2.2	Deep learning methods	75
5.2.3	Nearest neighbor-based methods	76
5.2.4	Matrix Profile-based methods	76
5.3	Preliminaries	77
5.3.1	Matrix Profile	78
5.3.2	Left Matirx Profile	80
5.3.3	Gradient boosting	81
5.4	Materials and Methods	81
5.4.1	Datasets	82
5.4.2	Problem Formulation	83
5.4.3	Evaluation Method	84
5.4.4	Forecasting via GBRT	85
5.4.5	Feature Engineering via Matrix Profile	86
5.4.6	Extends to k-Nearest Neighbors	88
5.5	Experiments	89
5.5.1	Effect of the length l of the immediate subsequence	90
5.5.2	Comparison between the two scenarios	91
5.5.3	Effect of using z-normalized instead of raw values	91
5.5.4	Effect of using pairwise difference representation instead of raw values	91
5.5.5	Summary of the experiments on the two primary models	92
5.5.6	Incorporating the target values of the nearest neighbor	93
5.5.7	Incorporating the covariates of the last point of the nearest neighbor	94
5.5.8	Extend to the case of k-nearest neighbor	94
5.5.9	Summary of the experiment	96
5.6	Concluding Remarks	96
5.6.1	Future Work	96
6	Conclusion and Future Directions	99
	Bibliography	103

List of Figures

2.1	Alignments.	6
3.1	Predicted secondary structure of the sequence S of pri-miRNA “hsa-let-7a-1” ¹ . Experimental evidence suggests that the two deviated mature miRNAs are $UGA \cdots GUU$ and $CUA \cdots UUC$. They are $S(6 : 27)$ and $S(57 : 77)$ (Both ends are inclusive.). The ends are highlighted in bold . Since $S(6 : 27)$ ($S(57 : 77)$) is near the 5' (3') end, we call it “5p (3p) mature miRNA”. The two scissors indicate the two cleavage sites. The color intensity of the nodes reflects their base-pair probability in this predicted secondary structure. The deeper the color, the higher the probability. The unpaired nodes are uncolored. The raw figure is generated by RNAfold web server ²	15
3.2	The overall pipeline of this study. Symbol notations: Cylinder - Dataset, Rectangle - Process, Parallelogram - Input / Output, Rounded Rectangle - Component.	17
3.3	Relationship of the proposed encoding methods.	21
3.4	Features generation in the transformation	26
3.5	Convolutions of HYDRA for each input time series with a set of random kernels w , organized into g groups with k kernels each. . . .	27
3.6	CD diagrams of channel ablation study.	32
3.7	CD diagrams to compare different classifiers.	34
3.8	CD diagrams to compare different encoding methods.	34
3.9	Results of the perturbation experiment.	36
4.1	Applying nearest neighbor interpolation on Q , which result in Scaled Q , that can better reflect its similarity with C	41
4.2	Q and C are in different rates. A stretching version of Q is similar to a prefix of C , but not the whole C	41
4.3	Intuition of piecewise scaling (PS).	42

4.4	Z-normalization. Resulting time series have mean = 0 and std = 1.	45
4.5	Visualization of D with local constraints. The black cells form the warping path.	47
4.6	Visualization of LB_{Keogh} and LB_{Shen}	51
4.7	Relationship of L_Q and L_C	59
4.8	Visualization of the GunPoint dataset. Left (Right): A time series of the “Gun” (“Point”) scenario. Critical periods, such as “Aiming”, are annotated.	63
4.9	The red solid time series shows an instant in the target set. The blue dashed time series shows an instant in the query set.	63
4.10	Runtime and number of distance calls comparison of GunPoint dataset. (a)-(d) $P = 1$, (e)-(h) $P = 2$, (i)-(l) $P = 3$	67
5.1	Flattening a 2D window X_i of size $(1 + M_x + M_u) \times w$ to a 1D vector X'_i of length $w + M_x + M_u$. The entries in the window are scalars. X'_i serves as the <i>predictor</i> for the regressor. Its expected <i>response</i> Y_i is a vector of length h . The regressor learns from the predictor-response pairs.	72
5.2	The left (right) matrix profile and the matrix profile of a time series. The matrix profile shows the distances between each m -subsequence and its nearest neighbor, where m is a user-given value. The left (right) matrix profile shows the same information but is limited to its left (right) nearest neighbor. For a particular m -subsequence indicated by the right gray box, its nearest neighbor is the left gray box, as indicated by the dashed line in the left matrix profile and the matrix profile . Similarly, the nearest neighbor of the left gray box is the right gray box, as indicated by the dashed line in the right matrix profile and the matrix profile . The first (last) point in each box is denoted by a red circle (triangle). l denotes the length of the immediate subsequence of the nearest neighbor. Intuitively, this subsequence should be similar to the immediate subsequence of the right gray box. To note, the left (right) matrix profile starts (ends) at a later (earlier) index.	74
5.3	Effect of different lengths l of the immediate subsequence (i.e., Number of points following the 1-NN) on RMSE.	90
5.4	Effect of different lengths l of the immediate subsequence (i.e., Number of points following the 1-NN) on MAE.	91

5.5	Using the z-normalized values instead of the raw values.	92
5.6	Using the pairwise differences instead of the raw values.	93

List of Tables

3.1	Selected representative records from miRBase. For the last two columns, the first line shows the name, the second line shows its location in the original sequence, and the third line indicates whether its existence has experimental evidence. The selected one is highlighted in bold	18
3.2	The whole sequence of “hsa-let-7a-1” and its predicted secondary structure by RNAfold. The corresponding positions of the two mature miRNAs and the probability of the unpaired “U” are highlighted in bold	19
3.3	The first row shows the “four strings” of “hsa-let-7a-1”. Their complementary strands are shown in the second row. As a whole, they are referred to as the “eight strings”.	20
3.4	Time series encoding. P is the corresponding base-pair probability sequence of S . $p_i = 1$ if we encode S without incorporating base-pair probability sequence.	23
3.5	Comparison of rocket-based classifiers [1]. $\mathcal{N}(0, 1)$: a standard normal distribution, $U(0, 1)$: a uniform distribution between 0 and 1, 1 st order difference: $\Delta T = t_2 - t_1, t_3 - t_2, \dots, t_n - t_{n-1}$	28
3.6	Channel ablation study. The best results are highlighted in bold . .	31
3.7	Performance on the 45 combinations between encoding methods and the ROCKET-based classifiers. The best results are highlighted in bold	33
3.8	Comparative analysis between MTSCCleave with the best combination of the encoding method and classifier, with the SOTA, Di-Cleave, on the three datasets. The best results of using MiniROCKET have also been shown to compare the computational efficiency. The best results are highlighted in bold	33
4.1	Details of the ten datasets for the experiments	65

4.2	The accuracy comparison for eight distance measures of the GunPoint dataset	66
4.3	The accuracy comparison for six PSED-guided distance measures of the GunPoint dataset	66
4.4	The accuracy comparison for eight distance measures of the ten datasets	68
4.5	The accuracy comparison for six PSED-guided distance measures of the ten datasets	68
4.6	The number of distance calls and runtime on PSED_vanilla and PSED_parallel_bsf of the ten datasets	69
5.1	Dataset Statistics. N : the number of time series in the dataset. n : the length of each time series T in the dataset. rate: measuring rate. start date: (date, time). h : the size of the forecasting window. T_{train} (T_{test}): the training (test) subsequence of T . To note, $ T_{\text{train}} + T_{\text{test}} = n$	82
5.2	Derived known input time-covariates with size M_u for each dataset.	83
5.3	Summary of the proposed models and their incorporated covariates. $\mathbf{NN}_{\text{imm}}$: the immediate subsequence of $\mathbf{NN}$. $\mathbf{NN}_{\text{last}}$: the last target value of $\mathbf{NN}$. $\mathbf{NN}_{\text{tar}}$: all the target values of $\mathbf{NN}$. $\mathbf{NN}_{\text{cov}}$: time-covariates of the last point of the $\mathbf{NN}$. Details of the incorporated covariates are in Algorithm 7.	88
5.4	Experimental Results without known input time-covariates.	90
5.5	Experimental Results with known input time-covariates.	90
5.6	Experimental Results with known input time-covariates for models incorporating the known input time-covariates of the last point of the nearest neighbors.	94
5.7	Experimental Results without known input time-covariates in the k -nearest neighbors case, where $k \in \{2, 3\}$	95
5.8	Experimental Results with known input time-covariates in the k -nearest neighbors case, where $k \in \{2, 3\}$	95
5.9	Experimental Results with known input time-covariates for models incorporating the known input time-covariates of the last point of the nearest neighbors in the k -nearest neighbors case, where $k \in \{2, 3\}$	95
5.10	Best models for each dataset with (RMSE, WAPE, MAE) respectively.	96

Chapter 1

Introduction

1.1 Background

Human generates a ton of data nowadays. We are producing more new data in one single day today than in the first twenty-one centuries of AD combined. We are drowning in information but thirsty for knowledge. It is natural for us to develop computational methods to accelerate the process of “harvesting” knowledge from information.

Computational tasks focus on the relationship between input and output. We would like to find the hidden function behind. To note, there are two ways to solve a computational problem. One is the algorithmic approach, and the other is the machine-learning approach.

There are two large categories of machine learning. They are supervised learning and unsupervised learning. Classification may be the most intuitive form of supervised learning. The input is data points with labels. We learn a model from the relationship between data points and the labels. The model predicts the labels for the new data points. They have many applications. For example, in medical applications, it involves classifying patients as healthy or diseased, or tumors as benign or malignant. The term “supervised” means the model has access to labeled data. In other words, it requires labeled training examples that provide ground truth. So, the model can learn the boundary between the categories. Our first study focuses on a classification problem in biology.

A representative of unsupervised learning is undoubtedly clustering. We aim to group data into distinct clusters. The key difference is that the data lack predefined labels. By grouping them, we aim to identify natural patterns hidden in the data. One example is clustering cells based on their gene-expression profiles. These

clusters might reveal distinct cell types. Note that we do not have the ground truth for the cluster set. In bioinformatics, we typically use enrichment analysis to determine whether specific gene functions are enriched in these clusters. Cell types often show enrichment for genes responsible for specific functions. This set of genes defines their biological role. In clustering, we first need to define a measure of similarity between two objects and decide what details should be ignored. This is called invariance. For example, in image classification, it should be invariant to zooming and rotation.

1.2 Contributions

In this study, our contributions mainly include three parts. First, we demonstrate a machine learning approach to analyze a biology problem in chapter 3. We propose the usage of the base pair probability sequence from the predicted secondary structure of RNA sequence as new information (i.e., covariates) for the classification task. We apply Rocket-based classifiers to identify the human dicer cleavage sites. Because of the simplicity of the transformation method and the classifiers, our proposed method achieves 3.7X to 28.8X speedup while achieving better or comparable results than the current state-of-the-art method. Second, we solve an algorithmic problem on distance measure in Chapter 4. We propose a new distance measure framework, namely PSD, that can incorporate any existing distance measures to achieve invariance for two time series with multiple rates (i.e., different scaling factors). Experiments show that our methods outperform ED, DTW, and the other five DTW-based methods. Moreover, we propose using the segmentation result returned by PSD to improve the accuracy of other distance measures. Third, we improve a forecaster with the usage of a data mining primitive, namely Matrix Profile, in Chapter 5. We propose leveraging left nearest neighbors for each forecasting window as new covariates to improve the accuracy of the underlying forecaster. We use a simple gradient boosting regression tree as the underlying forecaster. The experiment shows that this simple method can improve the accuracy of the forecaster.

1.3 Organization

In Chapter 2, we review some of the basic knowledge in biology and time series data mining, in particular, we focus on distance measures and the main core underlying Rocket-based classifiers, which is the Ridge classifier. In Chapter 3, we introduce

our study of the problem of predicting human dicer cleavage sites. We proposed a novel approach to frame this task as a multivariate time series classification problem by introducing nine encoding methods and making use of Rocket-based classifiers. In Chapter 4, we introduce a new distance measure framework, namely PSD. It releases the assumption that there is only one scaling factor existing throughout the whole time series. In Chapter 5, we introduce how to create new covariates in time series forecasting using matrix profile. This method can improve the accuracy of the existing forecaster by providing useful covariates. In Chapter 6, we give a conclusion to these two studies and provide future work on them.

Chapter 2

Preliminaries

In this chapter, we provide background on time series, with a focus on distance measures and classification, particularly the ridge classifier, which is used in the ROCKET-based classifiers for time series classification. The remaining preliminary knowledge will be provided in the corresponding chapters. Section 2.1 gives an overview of the existing distance measures used in the evaluation in Chapter 4. Section 2.2 reviews the additional knowledge about the classifiers used in Chapter 3. We start with the definition of a time series.

2.1 Distance Measures

A general form of a time series T is an ordered pair of n real-valued variables, $T = (b_1, c_1), (b_2, c_2), \dots, (b_n, c_n)$, where b_i is the behavioral attribute and c_i is the contextual attribute, where $1 \leq i \leq n$. c_i refers to the time stamp at which the measurement b_i is taken. Since the measurements are always taken in a uniform manner, t_i is simply incrementing from 1 to n uniformly. Hence, we can represent a time series more concisely as $T = t_1, t_2, \dots, t_n$, where $t_i = b_i$.

We may be interested not only in the entire time series but also in a segment of it, called a subsequence. A subsequence $T(i : j)$ of a time series T is a shorter time series, which is a contiguous subset of time points in T , that starts from position i and ends at position j . Formally, $T(i : j) = t_i, t_{i+1}, \dots, t_j$, where $1 \leq i \leq j \leq n$. We call $T(1 : m)$ as the prefix of length m of T , m -prefix in short.

To quantify the similarity between two time series, we need to define a distance measure, also known as a similarity measure, between them. Many distance measures have been proposed in the literature. Among them, the most established measures are undoubtedly Euclidean Distance (ED) and Dynamic Time Warping

(DTW). They are representatives of the two broad classes of distance measures, namely “lock-step” and “elastic”.

2.1.1 Euclidean Distance (ED)

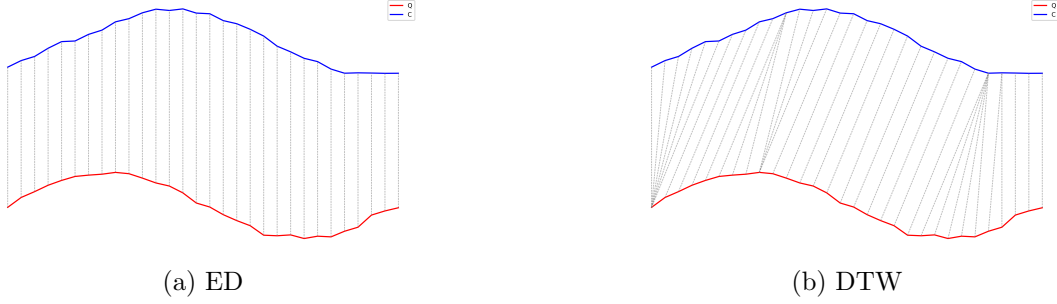


Figure 2.1: Alignments.

ED is a lock-step distance measure. Given two time series Q and C with the same length n , it compares the time point q_i of Q with the time point c_i of C at the same time (index). Note that, traditionally, lockstep distance measures require the two time series to have the same length because of the one-to-one alignment, as shown in Figure 2.1. However, in the setting of query by content, where it is always the case that $|Q| < |C|$, we can still apply a lock-step distance measure by either comparing Q with $C(1 : |Q|)$ or padding Q using its last element to lengthen it to the same length of C . Minkowski distance is a generalization of Euclidean distance. Minkowski distance is the L_p -norm of the difference between the two time series X and Y , defined as:

$$D(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}} \quad (2.1)$$

When $p = 2$, it corresponds to the Euclidean distance. When $p = 1$, it corresponds to the Manhattan distance. When $p = \infty$, it corresponds to the Chebyshev distance. In our studies, we focus on the Euclidean distance. Other lock-step distance measures include Pearson correlation distance. It accounts for the linear association between the two time series using the Pearson correlation coefficient. To note, there is another measure called Edit Distance for strings. And Edit Distance is also sometimes abbreviated as ED in string processing or bioinformatics. In this study, we focus on time series analysis and use ED to denote Euclidean distance rather than edit distance.

2.1.2 Dynamic Time Warping (DTW)

Dynamic Time Warping is an elastic measure [2]. In contrast to lock-step distance measures, elastic distance measures allow one-to-many point matching, as shown in Figure 2.1. The one-to-many point matching allows the elastic distance measures to warp in the time axis (i.e., temporally) such that it can handle the local temporal distortions. While it will be detailed in Chapter 4, it is briefly explained here. In short, it minimizes the cumulative distance between two time series, subject to constraints, by finding an optimal warping path W^* in a cost matrix, where W is the set of all possible paths. The constraints typically are (1) Boundary conditions, (2) Continuity, and (3) Monotonicity. This warping path provides feasibility for the point alignments to go beyond the 1-1 point alignment, as in the case of a lock-step distance measure,

2.1.3 Derivative Dynamic Time Warping (DDTW)

The Derivative Dynamic Time Warping (DDTW) is a variant of DTW [3]. Instead of comparing original raw values, it compares two time series using their first-order derivatives, but with an approximation. In DTW, a point on a rising trend may be mapped to a point on a falling trend. It goes against our intuition. It can be solved by comparing their first-order derivatives, which encodes the slope information. The derivative T' of a time series T is computed approximately as follows.

$$t'_i = \frac{(t_i - t_{i-1}) + \frac{t_{i+1} - t_{i-1}}{2}}{2} \quad (2.2)$$

This estimate is simply the average of “the slope of the line through t_i and t_{i-1} (i.e., its left neighbor)” and “the slope of the line through t_{i-1} (i.e., its left neighbor) and t_{i+1} (i.e., its right neighbor)”. The $1/2$ term in the second item of the numerator comes from the fact that the separation in time of the t_{i-1} and t_{i+1} is 2. Note that the estimate is not defined for the first and last elements of the time series in the above equation. In these boundary cases, we use the estimates of the second and penultimate (i.e., the second-to-last thing) as the estimates for the first and last elements, respectively.

2.1.4 Weighted Dynamic Time Warping (WDTW)

The Weighted Dynamic Time Warping (WDTW) is a variant of DTW [4]. It is a penalty-based DTW designed to prevent pathological paths. Recall that a

warping window (e.g, Sakoe-Chiba band) is enforced on the cost matrix of DTW, such that some paths are excluded. Only the paths that reside entirely in the warping window are feasible. This constraint may be too strict. WDTW uses a softer way for the same purpose. Instead of using a window to forbid the alignment of x_i and y_j that are far away in time. WDTW weights the cost of such alignment by multiplying it by a modified logistic weight function (MLWF) $\omega(k)$, defined as follows.

$$\omega(k) = \frac{\omega_{\max}}{1 + \exp(-g \cdot (k - m_c))} \quad (2.3)$$

Where:

- $k = |i - j|$. It is the phase difference (i.e., distance on the time axis from the diagonal. The diagonal refers to the line where $i = j$.)
- $\omega_{\max}$ is the desired upper bound for the weight parameter, which is suggested to be set to 1.
- m_c is the midpoint of a sequence. $m_c = m/2$.
- g is a constant that controls the level of penalization. It controls the curvature (slope) of the function.

Intuitively, if x_i and y_j are far apart temporally, it will have a larger weight to discourage their alignment and vice versa.

2.1.5 Weighted Derivative Dynamic Time Warping (WDDTW)

[4] also proposed the weighted version of DDTW. In brief, a weight is applied to the local cost function when computing DTW on the first derivative.

2.1.6 Shape Dynamic Time Warping (shapeDTW)

The Shape Dynamic Time Warping (shapeDTW) is a variant of DTW [5]. The main modification to the original DTW is the way the local distance between points is computed. Recall that DTW compares single scalar points. shapeDTW compares local descriptors. The local descriptors are constructed using a sliding window on the original series, such that for each point x , a L -subsequence with x as the center is extracted to compute the higher-level feature of x . L is the user-given length of the subsequence to consider. By default, it is set to 15 in Aeon,

which is a time series library in Python. There are several ways to construct such a descriptor. For example, a raw subsequence (i.e., a set of neighbor points surrounding the point of interest), Piecewise aggregate approximation (PAA) [6, 7], slope, derivative, HOG-1D [8].descriptor.

Then, the distance between descriptors is calculated rather than between raw values. When a raw subsequence is chosen to construct the local descriptors, a common metric used for comparing two local descriptors is the Euclidean distance. In the evaluation, a raw subsequence is chosen to construct the local descriptors, which is the default setting in Aeon.

2.1.7 Amercing Dynamic Time Warping (ADTW)

The Amercing Dynamic Time Warping (WDTW) is a variant of DTW [9]. It is also designed to constrain the amount of warping, as in cDTW and WDTW. While cDTW imposes a hard window and WDTW uses multiplicative weights (i.e, MLWF), ADTW introduces an additive penalty for non-diagonal alignment. The word “Amercing” means “fining”. The non-diagonal alignments are required to pay the fines. Unlike WDTW, which uses a multiplicative weight based on the position of the alignment, ADTW applies an additive penalty ω based on the action of warping. The non-diagonal alignments are penalized. Formally, the recursive relation for ADTW is defined as:

$$D(i, j) = d(q_i, c_j) + \min \begin{cases} D(i-1, j-1), \\ D(i-1, j) + \omega, \\ D(i, j-1) + \omega \end{cases} \quad (2.4)$$

ADTW penalizes the last two alignment actions. ω is a user-given hyperparameter. It should be a non-negative scalar constant. In practice, it is defined through cross-validation, which determines the optimal ω by training on a subset of data or heuristic search, which searches values in a user-given range.

To note, ADTW generalizes ED and DTW. If $\omega = 0$, no need to pay the fine for the non-diagonal alignment, which reduces to DTW. If $\omega \rightarrow \infty$, the non-diagonal alignment becomes prohibitive, and it reduces to ED.

2.2 Classification

In Chapter 3, we use ROCKET (RandOm Convolutional KErnel Transform) and its variants, including MiniRocket, MultiRocket, and Hydra, as the time series classifiers on the time series resulting from our encoding methods. Technically, they are not classifiers in their own right but rather feature extractors. These features are also called summary statistics. They are high-dimensional feature vectors that capture the characteristics of the original time series. The summary statistics are then fed to the classifiers to output the final classification results.

2.2.1 Ridge Classifier

The classifier that is usually chosen to work with ROCKET and its variants is a ridge classifier. The main advantage of it is speed. ROCKET and its variants generate a large number of features.

A ridge classifier is a wrapper that uses a ridge regression model as a routine to perform classification. It first maps the categorical labels of targets into continuous numbers, does the regression, and finally thresholds the numerical results from the regressor to obtain the classification result.

Given a training dataset $D = \{(x_i, y_i)\}_{i=1}^n$ with n instances, where $x_i \in \mathbb{R}^P$ is the feature vector with P dimensions and $y_i \in \{+1, -1\}$ is its label, it minimize the following optimization function.

$$\min_w \left(\sum_{i=1}^n (x_i^T w - y_i)^2 + \lambda \|w\|_2^2 \right) \quad (2.5)$$

Where $\|w\|_2^2 = \sum_{j=1}^P w_j^2$ is the L_2 norm of the weight vector and $\lambda > 0$ control the penalty. We explain the equation in brief. There are two terms inside the bracket. The first term is simply the sum of the residual, same as the one in the least squares method. The second term is called the L_2 penalty and is used to introduce bias in the fit to avoid overfitting. Hence, λ serves as a regularization hyperparameter between the trade-off between bias and variance.

Since the above optimization function in a ridge classifier has a closed-form solution, it can be solved using linear algebra rather than iterative optimization, as in logistic regression. Besides, the generated features by the random kernels in ROCKET and its variants are highly correlated. Ridge regularization, also known as the L_2 norm, can handle this case. Ridge regression shrinks regression

coefficients toward zero by adding an L2 penalty. It reduces model complexity and helps with multicollinearity.

Chapter 3

MTSCCLeav: a Multivariate Time Series Classification (MTSC)-based Method for Predicting Human Dicer Cleavage Sites

MicroRNAs (miRNAs) are small non-coding RNAs (ncRNAs) that regulate gene expression at the post-transcriptional level, thereby playing essential roles in diverse biological processes. The biogenesis of miRNAs requires dicer to cleave at specific sites on the precursor miRNAs (pre-miRNAs). Several machine learning approaches have been proposed to predict whether an input sequence contains a cleavage site. However, they rely heavily on complex feature engineering or opaque deep neural networks. It results in a lack of generalizability and a long running time. There is a need for an alternative modeling paradigm that is accurate, fast, and simple.

We proposed a novel approach to frame the task as a multivariate time series classification problem. Nine encoding methods have been proposed to convert the sequence and the predicted secondary structure into a time series. We also leveraged the probabilities of the base pairs in the predicted secondary structure. Computational experiments demonstrate that our proposed method can achieve better or comparable results in terms of using a simpler, more intuitive model and less computational time. It achieves 3.7X to 28.8X speedup. Through perturbation

experiments, we found that regions close to the center of pre-miRNAs are essential for predicting human dicer cleavage sites.

By transforming the RNA sequence and its secondary structure information into a time series and utilizing simple, state-of-the-art time series classifiers, we achieved comparable or even superior performance in a simpler and faster manner.

Code is available at: <https://github.com/colemanyu/time-series-classification-cleavage>.

3.1 Background

One of the most important theories in molecular biology is the central dogma. It depicts the flow of genetic information [10, 11]. Proteins are the functional units. The information stored in DNA is used to create them. Genes (segments) in DNA are used as templates for messenger RNAs (mRNAs) synthesis. An mRNA acts as a set of instructions to assemble a chain of amino acids, which form a linear polypeptide. To become biologically active, this chain is folded into a specific 3D structure, a proper configuration that enables it to perform its desired functions. This folded polypeptide is called a functional protein, or simply a protein. This entire process closely resembles how a computer program runs on a machine. The source code does not function by itself. First, it is translated into an assembly code (a lower-level, less human-readable form) and then into an executable file that can actually perform the intended tasks [12].

These mRNAs are called “coding RNAs” because they code for proteins. There are other genes in which the final product is the RNA molecule itself. They are called non-coding RNAs (ncRNAs). Two types of small ncRNAs are particularly important. They are microRNAs (miRNAs) and small interfering RNAs (siRNAs). Their discovery was recognized with the 2006 Nobel Prize in Physiology or Medicine¹, awarded for work completed only eight years prior [10].

In this study, we focus on miRNAs. An miRNA can regulate the expression of several proteins. Hence, understanding the biogenesis of miRNAs is of great value. It involves the processing of primary miRNAs (pri-miRNAs). RNAs are 3D molecules. However, it is hard to measure the 3D structure (tertiary structure) from the experiment and predict it from 1D sequence. We can understand their properties by analyzing their 1D sequence or 2D structure, known as secondary structure. RNA sequence is easily obtained through sequencing. The sequence

¹The Nobel Prize in Physiology or Medicine 2006 - NobelPrize.org: <https://www.nobelprize.org/prizes/medicine/2006/summary/> (Accessed on: 2025-06-13).

and its predicted secondary structure of a pri-miRNA “hsa-let-7a-1” is shown in Figure 3.1.

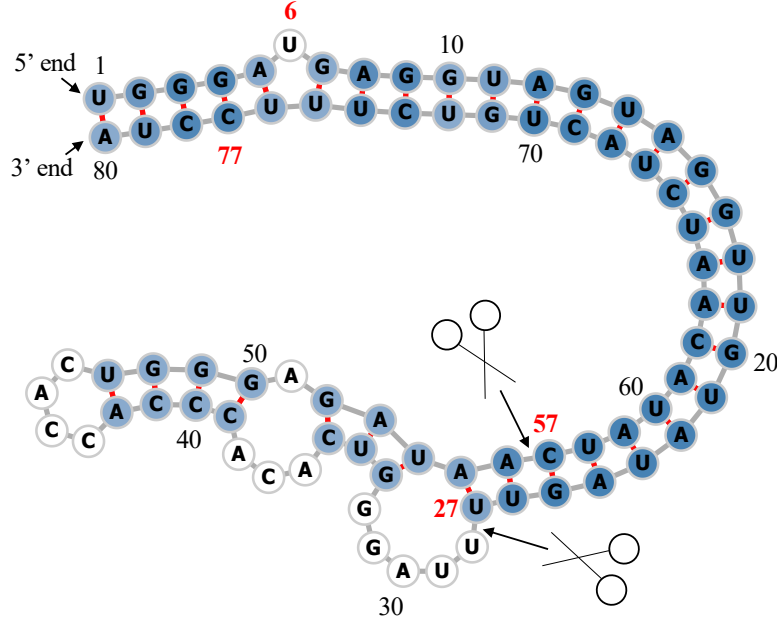


Figure 3.1: Predicted secondary structure of the sequence S of pri-miRNA “hsa-let-7a-1”². Experimental evidence suggests that the two deviated mature miRNAs are $UGA \cdots GUU$ and $CUA \cdots UUC$. They are $S(6 : 27)$ and $S(57 : 77)$ (Both ends are inclusive.). The ends are highlighted in **bold**. Since $S(6 : 27)$ ($S(57 : 77)$) is near the 5' (3') end, we call it “5p (3p) mature miRNA”. The two scissors indicate the two cleavage sites. The color intensity of the nodes reflects their base-pair probability in this predicted secondary structure. The deeper the color, the higher the probability. The unpaired nodes are uncolored. The raw figure is generated by RNAfold web server³.

Recall that a pri-miRNA contains a hairpin loop, also called a stem loop. A microprocessor complex comprising Drosha and DGCR8 cleaves the pri-miRNA to form a precursor miRNA (pre-miRNA) inside the nucleus. The stem-loop is still preserved, but the two arms become shorter. After that, the pri-miRNA is transported by Exportin 5 from the nucleus to the cytoplasm. It is further cleaved by an enzyme called dicer [13]. Dicer cleaves the stem-loop from the two arms at the two cleavage sites, shown as the two scissors in Figure 3.1. The stem-loop is removed. It results in a short double-stranded miRNA molecule, known

²Its miRBase entry: <https://mirbase.org/hairpin/MI0000060>. (Accessed on: 2025-06-12).

³RNAfold web server: <http://rna.tbi.univie.ac.at/cgi-bin/RNAWebSuite/RNAfold.cgi>. (Accessed on: 2025-06-12). The figure is viewed in “forna”. This view option can be chosen on the website.

as an miRNA duplex, which consists of the 5p strand and the 3p strand⁴. These molecules may be subjected to additional trimming. The miRNA duplex is loaded into an RNA-induced silencing complex (RISC). RISC unwinds the duplex and tends to retain the strand with the less stable 5' end as the guide strand. The other strand is called the passenger strand. The retained strand guides the RISC to silence the target mRNA. Note that both strands can become the guide strand.

Dicer plays an important role in the biogenesis of miRNAs. It is reasonable to argue that the structure of the pre-miRNAs informs dicer about the cleavage process. It would be of great benefit to understand how dicer selects cleavage sites from the neighborhood information near the cleavage sites. Studies [14, 15, 16] revealed that the secondary structures are essential for cleavage site determination. Hence, to predict or classify whether a subsequence, extracted from the sequence of a pri-miRNA, contains a cleavage site, we can make use of both the sequence and secondary structure information. PHDcleav employed support vector machines (SVM), leveraging sequence and structure-based features for the classification [17]. LBSIZEcleav improved upon it by considering the loop and bulge lengths [18]. [19] proposed an ensemble learning approach, using a gradient boosting machine for better accuracy. [20] developed a deep learning model, namely DiCleave. This model used an autoencoder to learn the secondary structure embeddings of pre-miRNAs from all the species in the miRBase database and leveraged this information. All these methods begin with curated pre-miRNA sequences from the miRBase database. Their secondary structures are predicted. Patterns are extracted from the sequence and the secondary structure. They create the positive cleavage patterns by setting the cleavage sites at the middle of the patterns. The follow-up work of [20], which created the cleavage pattern by allowing cleavage sites to appear at any position within the pattern, instead of the middle only [21]. It created a much larger dataset. This increased dataset facilitates the learning of the deep learning method at the cost of increased running time. We utilized the original dataset setting [17, 18, 19, 20]. DiCleave is the current state-of-the-art (SOTA) for this problem with the original dataset setting.

These models suffer several limitations. They rely heavily on complicated feature engineering or opaque deep learning models [19, 20, 21]. It results in a lack of generalizability and a long running time. There is a need to design a simpler model so that it can be easily extended to other prediction tasks on RNA data. One way to analyze sequence data is to transform it into time series data.

⁴The 5p strand comes from the 5' arm while the 3p strand comes from the 3' arm. For the directionality, the 5p (3p) strand retains the original 5' (3') end of the pre-miRNA.

In response to this, we proposed a multivariate time series classification-based method. Our contributions are shown as follows.

1. To the best of our knowledge, we are the first to frame the prediction of the cleavage sites as a multivariate time series classification problem.
2. We introduced several encoding methods to convert RNA data to time series.
3. We proposed utilizing the base-pair probabilities in the predicted secondary structure for the prediction. To our surprise, this information has been ignored in the existing studies.
4. For computational efficiency, our method achieves a 3.7X to 28.8X speedup compared to the state-of-the-art (SOTA).
5. We conducted perturbation-based experiments. It shows that regions close to the cleavage sites are important for this problem. It is consistent with the existing study [19].

3.2 Methods

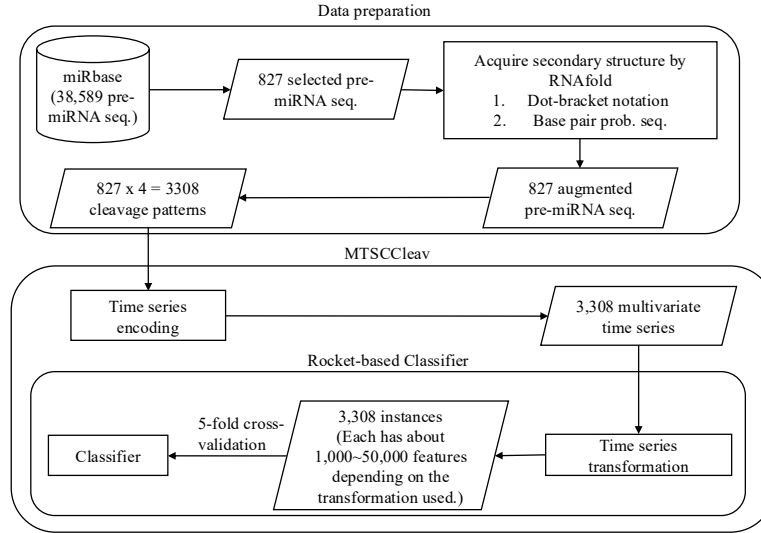


Figure 3.2: The overall pipeline of this study. Symbol notations: Cylinder - Dataset, Rectangle - Process, Parallelogram - Input / Output, Rounded Rectangle - Component.

The overall pipeline of this study is summarized in Figure 3.2.

3.2.1 Data Preparation

Accession	Name	Organism	Sequence	Mature miRNA 1	Mature miRNA 2
MI0000001	cel-let-7	Caenorhabditis elegans	<i>UACAC...UUCGA</i>	cel-let-7-5p 17:38 experimental	cel-let-7-3p 60:81 experimental
MI0000060	hsa-let-7a-1	Homo sapiens	<i>UGGGA...UCCUA</i>	hsa-let-7a-5p 6:27 experimental	hsa-let-7a-3p 57:77 experimental
MI0000114	hsa-mir-107	Homo sapiens	<i>CUCUC...ACAGA</i>	hsa-miR-107 50:72 experimental	NA
MI0000238	hsa-mir-196a-1	Homo sapiens	<i>GUGAA...UUCAC</i>	hsa-miR-196a-5p 7:28 experimental	hsa-miR-196a-1-3p 45:65 not experimental

Table 3.1: Selected representative records from miRBase. For the last two columns, the first line shows the name, the second line shows its location in the original sequence, and the third line indicates whether its existence has experimental evidence. The selected one is highlighted in **bold**.

We used miRBase database [22]⁵. The database comprises miRNA data from various organisms [23]. The database contains 38,589 miRNA records. Each record refers to an miRNA sequence, along with other properties such as name, accession, organism, and information on its derivative miRNA products. We are interested in pri-miRNA in humans. The derivative miRNA products are the mature miRNAs. The database also annotates the location of the mature miRNA within the original sequence and indicates whether its existence has experimental evidence.

Table 3.1 shows its four representative records. We first selected the records from humans (Homo sapiens). It resulted in 1,917 records. To identify the actual locations of the two cleavage sites in the pri-miRNA sequence supported by experimental evidence, we selected records that have two mature miRNAs resulting from cleavage at the 5p arm and the 3p arm, both of which have experimental support. Hence, only “MI0000060” (“hsa-let-7a-1”) would be selected in the table. It would serve as our running example. Its whole sequence is listed in Table 3.2. After the selection process, we selected 827 experimental validated pre-miRNA sequences, each with its two mature miRNA products. This formed our dataset.

Augment the Dataset with Secondary Structure Information

We leveraged the predicted secondary structure of these sequences to enhance the accuracy of the classification. Recall that a specific three-dimensional (3D) structure is required for DNA, RNA, and protein to perform functions [24]. However,

⁵The website is www.mirbase.org, and the newest version of the database is Release 22.1 (Accessed on 2025-06-22).

Sequence	Secondary Structure (In Dot-bracket notation)
1 UGGGA UGAGGUAGUAGGUUGUAUAGUU 27 28 UUAGGGUCACACCCACCACUGGGAGAU 54 55 AA CUAUACA AUCU ACUGUCUUUCCUA 80	1 ((((((.(((((((((((((((((((((((27 28 UUAGGGUCACACCCACCACUGGGAGAU 54 55)))))))(((((((((((((((((((())) 80
Base-pair probabilities sequence (the first 10 bases)	
1 (0.549, 0.946, 0.987, 0.987, 0.904) 5 6 (0.000 , 0.841, 0.974, 0.981, 0.890) 10	

Table 3.2: The whole sequence of “hsa-let-7a-1” and its predicted secondary structure by RNAfold. The corresponding positions of the two mature miRNAs and the probability of the unpaired “U” are highlighted in **bold**.

finding these 3D structures using experimental methods such as X-ray crystallography or nuclear magnetic resonance (NMR) is costly and time-consuming. Hence, prediction methods for such 3D structures are necessary and helpful for downstream analysis. However, predicting the 3D structures is challenging. One of the reasons is that there are some “nonconventional” base-pair interactions (e.g., noncanonical and rare A-G) that allow an RNA sequence to fold into a 3D structure, in addition to the (G, U) wobble pair, which is common and functionally important in RNA secondary structures. It makes the search space for prediction much larger than, in the 2D case, the secondary structure. The local structures of the 3D structures, the secondary structures, only focus on the conventional base-pair interactions [11]. Hence, predicting secondary structures is easier and faster. We employed RNAfold from the ViennaRNA Package⁶ to predict the secondary structure for a given pri-miRNA S [25]. RNAfold returns the secondary structure in the dot-bracket notation and a matrix of base-pair probabilities. The matrix is a square matrix with the side length $|S|$, where each entry m_{ij} is the probability of base s_i paired up with base s_j . Dot-bracket notation is a way of representing the secondary structure of S . Open parentheses “(” (Close parentheses “)”) indicates that the base is paired with a complementary base further (earlier) along in S . Dot “.” indicates that the base is unpaired. Equipped with the matrix, we can construct the base-pair probability sequence of S . The predicted secondary structure and the base-pair probability sequence of our running example are shown in Table 3.2.

Extract Cleavage Patterns

The locations of the two mature miRNAs on the whole sequence indicate the probable locations of the two cleavage sites. The 5p cleavage site must be beyond

⁶The latest stable release is Version 2.7.0 (Accessed on 2025-06-22).

and near the ending location of the 5p mature miRNA. We deemed the immediate bond next to the 5p mature miRNA’s ending position the 5p cleavage site, with the knowledge that the actual cleavage site may not be this immediate bond but rather the nearby bonds after it. The same applies to the 3p cleavage site. It is located at the immediate bond before the starting position of the 3p mature miRNA.

For each arm of each whole sequence, we extracted a 14-string⁷ with the cleavage site located at the center of the string. The first 7 nt (nucleotide) before the center are highlighted in **bold**. In our running example, it would be “**UAUAGUU**UUAGGU” for the 5p cleavage site and “**GAGAUAA**CUAUACA” for the 3p cleavage site. We refer to these 14-strings as cleavage patterns. We also generate non-cleavage patterns by selecting a 14-string with the center 6 nt away from the corresponding cleavage sites towards the corresponding mature miRNA [18, 19] for each arm of each whole sequence. So, in our running example, the 5p non-cleavage pattern would be “**AGGUUGU**AUAGUUU”. The 3p non-cleavage pattern would be “**ACUAUAC**AAUCUAC”.

In conclusion, for a given pri-miRNA sequence, we can generate two cleavage patterns and two non-cleavage patterns. We call these four patterns simply the “four strings” of a given pri-miRNA. We also call each string a strand. The “four strings” of our running example are listed in Table 3.3.

	5p cleav	5p non-cleav	3p cleav	3p non-cleav
Input strand	UAUAGUU UUAGGGU	AGGUUGU AUAGUUU	GAGAUAA CUAUACA	ACUAUAC AAUCUAC
Complementary strand	AUAUCAA_____UA	UCUAACAUAUCAA_	C_CUGUUGAU AUGU	UGAU AUGUUGGAUG

Table 3.3: The first row shows the “four strings” of “hsa-let-7a-1”. Their complementary strands are shown in the second row. As a whole, they are referred to as the “eight strings”.

We can construct the complementary strand of each of the strands in the “four strings” by finding the corresponding paired base for each of the bases in the input strand by considering the secondary structure information. We use “_” to denote the unpaired base in the complementary strand. For example, in Figure 3.1, “UUAGG” in the 5p cleavage pattern is unpaired, while other bases pair with some bases, the resulting complementary strand is “AUAUCAA_____UA”. There is a loop/budge there. We refer to the “four strings” and the four complementary strands together as the “eight strings” of the input pre-miRNA. It is also shown in Table 3.3.

⁷String with length = 14.

3.2.2 Time Series Encoding

A *time series* $T = t_1, t_2, \dots, t_n$ is a sequence of real-valued numbers⁸. A short contiguous region of T is called a subsequence. A *subsequence* $T(i : j) = t_i, t_{i+1}, \dots, t_j$ of a time series T is a shorter time series that starts from position i and ends at position j , where $i < j$.

Strings and time series are temporal sequences. The difference between strings and time series lies in their behavioral attributes [26]. For strings, an entry is a letter from a predefined set called the *alphabet*. For example, the alphabet is $\{A, C, G, T\}$ in the DNA string, while $\{A, C, G, U\}$ in the RNA string. For time series, an entry is a real number. Unlike real numbers, there is no ordering in the alphabet unless some external domain knowledge is introduced.

The study of applying signal processing techniques to genomic data is called “Genomic Signal Processing” (GSP) [27, 28]. In the field of GSP, the time series representations of DNA strings are referred to as DNA numeric representations (DNR). Many DNRs have been proposed. We noted that DNA strings and RNA strings are equivalent from a computational standpoint. Many transformation methods designed for DNA can be applied to RNA by simply substituting T with U . We present nine encoding methods. The relationship among them is shown in Figure 3.3.

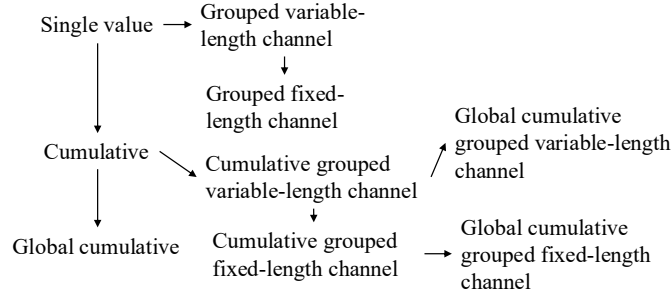


Figure 3.3: Relationship of the proposed encoding methods.

Single Value versus Cumulative

One of the simple, if not the simplest, encoding is to map the letters into numbers. Domain knowledge can be utilized. This approach is called the “Single value mapping” [29, 30, 31, 32, 27]. One single value is assigned to each of the letters. [33] employed the atomic number of each nucleotide as the transformed values,

⁸Unless otherwise specified, we denote entries of a time series (e.g., T) using the corresponding lowercase letter (e.g., t).

where $\{G = 78, A = 70, C = 58, T = 66\}$. [34] used electron-ion interaction potential representation (EIIP) as such value, where $\{G = 0.0806, A = 0.1260, C = 0.1340, T = 0.1335\}$. Our goal is to transform the input strand and its complementary strand into time series, aiming to capture the information contained in these sequences and the secondary structure implied by them. We employed the following reasoning to assign the value:

1. We employ the complementary property [35, 31] during encoding. Recall that in the base-pairing rules, G pairs with C to form three hydrogen bonds while A pairs with U ⁹ to form two hydrogen bonds. G - C pairs are more stable than A - U pairs. G (U) can be regarded as the “inverse” of C (A). We can preserve these base-pairing rules in the encoding by assigning G (A) and C (U) opposite values.
2. G and A have a two-ring structure. They are purines. C and U have a single-ring structure. They are pyrimidines. Hence, we put G and A (C and U) on the same side of the number line with zero in the middle.
3. The lower stability of A - U pairs promotes strand separation, thereby facilitating the unwinding of the miRNA duplex during RISC loading. Regions rich in A and U are thus more likely to undergo strand selection and cleavage events. We assigned A (U) with a larger absolute value than G (C) to reflect this functional relevance. It aims to highlight sequence regions with higher cleavage potential.

It results in our baseline transformation method, namely “Single value mapping” as shown in row 1 of Table 3.4. S is the input strand. When we encode S without incorporating the corresponding base-pair probability sequence P , we set $p_i = 1$ for all the entries of P . We use the first ten nucleotides of the complementary strand of the 3p cleav of “hsa-let-7a-1”, as shown in Table 3.3 as S in the examples in Table 3.4.

With the assigned value to each nucleotide defined in single-value mapping, we can compute a cumulative sum of those values over time. It captures the aggregated signal by accumulating past events, allowing us to focus on the trend [36, 37]. We named this method as “Cumulative mapping”, shown in row 4 of Table 3.4.

⁹In DNA, A pairs with T .

	Encoding	Algorithm	Example $S = C, -, C, U, G, U, U, G, A, U$ $P = 0.843, 0.000, 0.807, 0.807, 0.793,$ $0.914, 0.982, 1.000, 0.999, 0.999$
1	Single value mapping [29, 30, 31, 32, 27]	for $i = 1$ to $ S $: $t_i = \begin{cases} 2 \cdot p_i & \text{if } s_i = A \\ 1 \cdot p_i & \text{if } s_i = G \\ -1 \cdot p_i & \text{if } s_i = C \\ -2 \cdot p_i & \text{if } s_i = U \\ 0 & \text{otherwise} \end{cases}$ return T	Without base-pair probability sequence: $T = -1, 0, -1, -2, 1, -2, -2, 1, 2, -2$ With base-pair probability sequence: $T = -0.843, 0.000, -0.807, -1.614,$ $0.793, -1.829, -1.963,$ $1.000, 1.999, -1.998$
2	Grouped variable-length channel mapping	$j = 1, k = 1$ for $i = 1$ to $ S $: $t_j^1 = \begin{cases} 1 \cdot p_i & \text{if } s_i = A \\ -1 \cdot p_i & \text{if } s_i = U \\ 0 & \text{otherwise} \end{cases}$ $t_k^2 = \begin{cases} 1 \cdot p_i & \text{if } s_i = G \\ -1 \cdot p_i & \text{if } s_i = C \end{cases}$ if $(s_i = G)$ or $(s_i = C)$: increment k by 1 else: increment j by 1 return T^1, T^2	Without base-pair probability sequence: $T^1 = 0, -1, -1, -1, 1, -1$ $T^2 = -1, -1, 1, 1$ With base-pair probability sequence: $T^1 = 0.000, -0.807, -0.914, -0.982, 0.999, -0.999$ $T^2 = -0.843, -0.807, 0.793, 1.000$
3	Grouped fixed-length channel mapping	for $i = 1$ to $ S $: $t_i^1 = \begin{cases} 1 \cdot p_i & \text{if } s_i = A \\ -1 \cdot p_i & \text{if } s_i = U \\ 0 & \text{otherwise} \end{cases}$ $t_i^2 = \begin{cases} 1 \cdot p_i & \text{if } s_i = G \\ -1 \cdot p_i & \text{if } s_i = C \\ 0 & \text{otherwise} \end{cases}$ return T^1, T^2	Without base-pair probability sequence: $T^1 = 0, 0, 0, -1, 0, -1, -1, 0, 1, -1$ $T^2 = -1, 0, -1, 0, 1, 0, 0, 1, 0, 0$ With base-pair probability sequence: $T^1 = 0.000, 0.000, 0.000, -0.807,$ $0.000, -0.914, -0.982,$ $0.000, 0.999, -0.9999$ $T^2 = -0.843, 0.000, -0.807, 0.000,$ $0.793, 0.000, 0.000,$ $1.000, 0.000, 0.000$
4	Cumulative mapping [36, 37]	$t_1 = 0$ for $i = 1$ to $ S $: $t_{i+1} = \begin{cases} t_i + 2 \cdot p_i & \text{if } s_i = A \\ t_i + 1 \cdot p_i & \text{if } s_i = G \\ t_i - 1 \cdot p_i & \text{if } s_i = C \\ t_i - 2 \cdot p_i & \text{if } s_i = U \\ t_i & \text{otherwise} \end{cases}$ return T // $ T = S + 1$	Without base-pair probability sequence: $T = 0, -1, -1, -2, -4, -3, -5, -7, -6, -4, -6$ With base-pair probability sequence: $T = 0.000, -0.843, -0.843, -1.650,$ $-3.265, -2.471, -4.300, -6.263,$ $-5.264, -3.265, -5.263$
5	Cumulative grouped variable-length channel mapping	$t_1^1 = 0, t_1^2 = 0$ $j = 1, k = 1$ for $i = 1$ to $ S $: $t_{j+1}^1 = \begin{cases} t_j^1 + 1 \cdot p_i & \text{if } s_i = A \\ t_j^1 - 1 \cdot p_i & \text{if } s_i = U \\ t_j^1 & \text{if } s_i = - \end{cases}$ $t_{k+1}^2 = \begin{cases} t_k^2 + 1 \cdot p_i & \text{if } s_i = G \\ t_k^2 - 1 \cdot p_i & \text{if } s_i = C \end{cases}$ if $(s_i = G)$ or $(s_i = C)$: increment k by 1 else: increment j by 1 return T^1, T^2	Without base-pair probability sequence: $T^1 = 0, -1, -2, -3, -2, -3$ $T^2 = 0, -1, -2, -1, 0$ With base-pair probability sequence: $T^1 = 0.000, -0.807, -1.722,$ $-2.703, -1.704, -2.703$ $T^2 = 0.000, -0.843, -1.650,$ $-0.857, 0.143$
6	Cumulative grouped fixed-length channel mapping	$t_1^1 = 0, t_1^2 = 0$ for $i = 1$ to $ S $: $t_{i+1}^1 = \begin{cases} t_i^1 + 1 \cdot p_i & \text{if } s_i = A \\ t_i^1 - 1 \cdot p_i & \text{if } s_i = U \\ t_i^1 & \text{otherwise} \end{cases}$ $t_{i+1}^2 = \begin{cases} t_i^2 + 1 \cdot p_i & \text{if } s_i = G \\ t_i^2 - 1 \cdot p_i & \text{if } s_i = C \\ t_i^2 & \text{otherwise} \end{cases}$ return T^1, T^2 // $ T^1 = T^2 = S + 1$	Without base-pair probability sequence: $T^1 = 0, 0, 0, 0, -1, -1, -2, -3, -3, -2, -3$ $T^2 = 0, -1, -1, -2, -2, -1, -1, -1, 0, 0, 0$ With base-pair probability sequence: $T^1 = 0.000, 0.000, 0.000, 0.000,$ $-0.807, -0.807, -1.722, -2.703,$ $-2.703, -1.704, -2.703$ $T^2 = 0.000, -0.843, -0.843, -1.650,$ $-1.650, -0.857, -0.857, -0.857,$ $0.143, 0.143, 0.143$

Table 3.4: Time series encoding. P is the corresponding base-pair probability sequence of S . $p_i = 1$ if we encode S without incorporating base-pair probability sequence.

Grouped Variable-Length Channel versus Grouped Local-Length Channel

We can transform the input strand into a multivariate time series with two channels using grouped binary encoding, where nucleotides are grouped into (A, U) and (G, C) . It releases our third assumption that $A (U)$ has a larger absolute value than $G (C)$. We proposed two variations. The first one allows the output to be variable-length sequences per channel, depending on group-specific occurrences. The second one always returns two resulting sequences of a fixed length. Two variations extended from single value mapping are shown in rows 2 and 3, while those extended from cumulative mapping are shown in rows 5 and 6 in Table 3.4.

Global Cumulative versus Local Cumulative

In cumulative mapping and its variations, we can choose where to start the accumulation. For a given subsequence S' of the whole sequence S , accumulation can start from the beginning of S even if only S' is used downstream. It can also begin just at the start of the S' . The first one preserves the global context. It can be useful when previous nucleotides (those before S') influence later interpretation. The second one focuses solely on local history in S' , ignoring global history. It is helpful if the previous nucleotides do not affect the chemical property of S' .

Consider $T = 0, -1, \dots, -6$ of the input string S in “Cumulative mapping” in Table 3.4, which accumulates from 0. S is the suffix with length = 10 of the constructed complementary strand of $S(1 : 63)$ in Figure 3.1. If we start the accumulation from the first entry of the constructed complementary strand instead, it will yield a different result. Suppose that the last entry of the time series encoded in the cumulative mapping of the constructed complementary strand is -8, the time series encoded in the “Global cumulative mapping” for S would accumulate from -8 instead of 0. The result is $T = -8, -9, \dots, -14$. Note that it has the same trend as the original T . This “Global cumulative” concept can be applied to every cumulative-based method, as shown in Figure 3.3.

Incorporating Base-Pair Probabilities

We can incorporate the base-pair probabilities P in the encoding by thinking of it as the weight or confidence p_i in the value assignment of each nucleotide s_i . It is implemented by multiplying the base-pair probability p_i of the nucleotide s_i with the assigned value of the kind of nucleotide of s_i during encoding, as shown in Table 3.4.

Transforming the Secondary Structure into a Time Series

We can transform the secondary structure in the dot-bracket notation into a time series by “Single value mapping”, where “(” maps to 1, “.” maps to 0, and “)” maps to -1.

3.2.3 Time Series Classification

In univariate time series classification, an instance in the dataset consists of a time series $x = x_1, x_2, \dots, x_m$ with m observations and a discrete class label y , which takes c possible values [38, 39]. If $c = 2$, we refer to binary classification. If $c > 2$, we refer to multi-class classification. In multivariate time series classification, the time series is not a single sequence but a list of sequences. Each sequence is called a channel. There are many classifiers defined for time series data, including distance-based, feature-based, interval-based, shapelet-based, dictionary-based, convolution-based, and deep learning-based classifiers. Additionally, two or more of the above approaches can be combined, resulting in hybrid approaches [1, 39, 38]. We employed convolution-based classifiers due to their simplicity and accuracy.

Convolution-Based Classifiers

Convolution-based classifiers first use randomly parameterized kernels to perform convolutions on the original time series T . A kernel is referred to as parameterized because its behavior is governed by a set of parameters, which will be discussed in detail later. Convolution is an operation to transform T to another time series M , where M is called the activation map. Its entry M_i is calculated by applying a kernel ω with length l to T at position i , defined as follows:

$$M_i = T(i : i + l - 1) * \omega = \sum_{j=0}^{l-1} t_{i+j} \cdot \omega_{1+j}$$

To note, $|T(i : i + l - 1)| = |\omega| = l$. Entries M_i ’s are calculated by sliding ω across T and computing a dot product. Additionally, although the original paper [40] used the term “convolution” to refer to the above operation, “cross-correlation” may be a more suitable term for this operation. Recall T with length m has $(m - l + 1)$ sliding windows of length l , given that the increment is 1¹⁰, which defines the length of M .

¹⁰One step to the right per time.

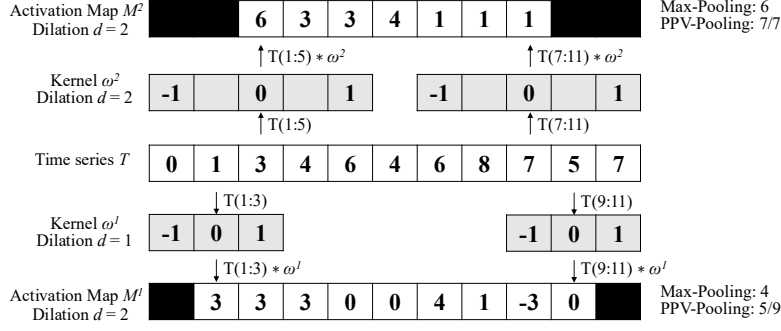


Figure 3.4: Features generation in the transformation

Figure 3.4 shows two kernels ω^1 and ω^2 with lengths 3 and 5, respectively. Each of which performs a convolution with T and returns two activation maps, M^1 and M^2 , respectively. For example, $M_1^1 = T(1 : 3) * \omega^1 = 3$. By sliding ω^1 one time stamp at a time, an activation map M^1 with length $= (m - l + 1) = 11 - 3 + 1 = 9$ is obtained. Then, pooling operations, such as the maximum (MAX) and proportion of positive values (PPV), are applied on M^1 to derive the summary features. In Figure 3.4, MAX and PPV are applied on M^1 and M^2 . The summary features of M^1 are 4 and 5/9, which correspond to MAX and PPV, respectively. Dilation refers to a method that enables a kernel to cover a larger portion by creating empty spaces between entries in the kernel. The dilation d of ω^2 is 2. It introduces a gap of 1 in every two values of ω^2 .

The most popular convolution-based approach is the Random Convolutional Kernel Transform (ROCKET) [40]. It generates a large number of randomly parameterized kernels, ranging from thousands to tens of thousands. The kernel's parameters include length, weights (the entries inside the kernel), bias (the value added to the result of the convolution operation), and dilation. Additionally, padding can be applied to T at the start and end, ensuring M has the same length as the input. To note, T , M_1 , and M_2 in Figure 3.4 have different lengths. The summary statistics of the activation map are obtained through two pooling operations: MAX and PPV. Hence, for k kernels, the transformed data has $2k$ features. The default value of k is 10,000.

There are two extensions of ROCKET. They are MiniROCKET [41] and MultiROCKET [42]. MiniROCKET removes unnecessary operations and many of the random components in the definition of kernels used by ROCKET. It speeds up Rocket by over an order of magnitude with no significant difference in accuracy, making the classifier almost deterministic. For example, the kernel length is fixed, and only two weight values are used. Only PPV is used for the summary statis-

tics. MultiROCKET is extended from MiniROCKET. The main improvement of it is to extract features from first-order differences as defined in Table 3.5 and add three new pooling operations [42]. The three added operations are mean of positive values (MPV), mean of indices of positive values (MIPV) and longest stretch of positive values (LSPV).

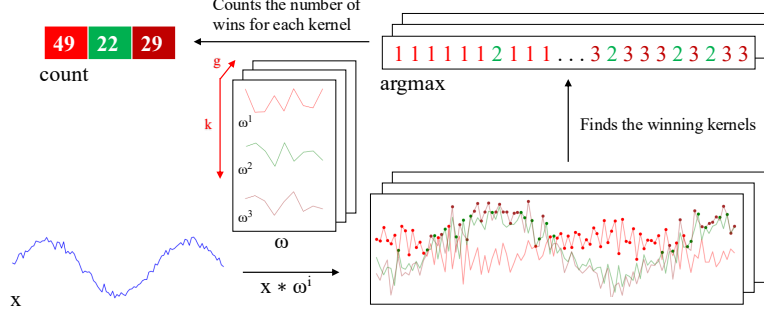


Figure 3.5: Convolutions of HYDRA for each input time series with a set of random kernels w , organized into g groups with k kernels each.

The HYbrid Dictionary-ROCKET Architecture (Hydra) combines dictionary-based and convolution-based models [43]. Similar to ROCKET-based classifiers, it uses random kernels to extract features from the input time series. But it groups the kernels into g groups of k kernels each, as shown in Figure 3.5. Each time series is passed through all the groups. For each group of kernels, we slide them across T and compute the dot product at each timestamp. Recall that the dot product of two input vectors (x and w_i) has the maximum value when the two vectors align in the same direction and the minimum value when they are oriented in opposite directions. We record the kernel that best matches the subsequence of T at each timestamp in each group (i.e., argmax). We refer to these kernels as the winning kernels. This results in a k -dimensional count vector for each of the g groups, where $k = 3$ in Figure 3.5. This results in a total of $g \times k$ features, with default values of $g = 64$ and $k = 8$. It uses a total of $k \times g = 512$ kernels per dilation. In addition to recording the kernel with the maximum response, we can also record the kernel with the minimum response, knowing that this kernel will be the best match with the “inverted” subsequence of T . Hydra is applied to both the original time series and its first-order differences. Hydra generated approximately 1000 features for each instance in our dataset. [43] found that it can improve the accuracy by concatenating features generated from Hydra with those from MultiRocket. This classifier is called MultiROCKET-Hydra.

These five classifiers share the same simple design pattern. It involves the over-production of features followed by a selection strategy. A large number of features

(1,000 $\sim$ 50,000) are generated for each instance. The features are then fed into a simple linear classifier. It determines which features are most useful and returns the final classification result. A ridge classifier is used in this study. It is a linear classifier that extends ridge regression to classification tasks by applying a threshold to the predicted values. It uses L2 regularization to prevent overfitting. The regularization strength is selected by internal cross-validation. A Ridge classifier is suggested for small datasets, as in our case, while a logistic regression classifier is suggested for large datasets [1].

While these five classifiers are often referred to as classifiers [1], they are technically time series transformation methods for generating features that are then fed to a downstream classifier. The comparison of them is shown in Table 3.5. For MiniROCKET and MultiROCKET, the bias is determined from the convolution output, and the dilation depends on the length of the input time series [41, 42]. The main differences among ROCKET-based classifiers lie in how the summary features are generated. The generation of the summary features depends on:

1. Kernels, which are defined based on the parameters, which consist of kernel length, kernel weights, bias, and dilation.
2. The way that padding applies to T , which leads to activation maps with different lengths.
3. The pooling operations, which are used in extracting features on the activation map.

	ROCKET	MiniROCKET	MultiROCKET	Hydra
kernel length	{7, 9, 11}	9	9	9
kernel weights	$\mathcal{N}(0, 1)$	{-1, 2}	{-1, 2}	$\mathcal{N}(0, 1)$
bias	$\mathcal{U}(0, 1)$	from output	from output	none
dilation	random	fixed (input-relative)	fixed (input-relative)	random
padding	random	fixed	fixed	always
pooling operations	MAX, PPV	PPV	PPV, MPV, MIPV, LSPV	Response per Kernel/Group
1 st order difference	no	no	yes	yes
feature vector size	20k	10k	50k	relative to input

Table 3.5: Comparison of rocket-based classifiers [1]. $\mathcal{N}(0, 1)$: a standard normal distribution, $\mathcal{U}(0, 1)$: a uniform distribution between 0 and 1, 1st order difference: $\Delta T = t_2 - t_1, t_3 - t_2, \dots, t_n - t_{n-1}$.

3.2.4 Evaluation Metrics

To evaluate the performance of our time series-based classification (MTSC) model, we adopted five standard classification metrics. They are Accuracy (Acc), Speci-

ficity (Sp), Sensitivity (Sn), F1 score (F1), and Matthews Correlation Coefficient (MCC) [44].

$$\begin{aligned}
Acc &= \frac{TP + TN}{TP + TN + FP + FN} \\
Sp &= \frac{TN}{TN + FP} \\
Sn &= \frac{TP}{TP + FN} \\
F1 &= \frac{2 \times TP}{2 \times TP + FP + FN} \\
MCC &= \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}
\end{aligned}$$

where TP, TN, FP, and FN are the number of true positives, true negatives, false positives, and false negatives, respectively.

To extend a binary metric to multi-class problems, we can treat the data as a collection of binary problems, one for each class. One class is treated as positive while the other classes are treated as negative. Then, the multi-class metrics can be obtained by averaging binary metric calculations across the set of classes. There are different ways of doing the averaging. Here, we adopted a macro-averaging approach. It treats each class equally and calculates the mean of the binary metrics. To use *MCC* in the multiclass case, it can be defined in terms of a confusion matrix C for K classes, where $C_{i,j}$ is the number of observations that are actually in class i and predicted to be in class j [45].

$$MCC_{multi} = \frac{c \times s - \sum_k^K p_k \times t_k}{\sqrt{(s^2 - \sum_k^K p_k^2) \times (s^2 - \sum_k^K t_k^2)}}$$

where $t_k = \sum_i^K C_{i,k}$ (denoting the number of times class k actually occurred), $p_k = \sum_i^K C_{k,i}$ (denoting the number of times class k was predicted), $c = \sum_k^K C_{k,k}$ (denoting the total number of samples correctly predicted) and $s = \sum_i^K \sum_j^K C_{i,j}$ (denoting the total number of samples).

3.3 Results

The code implementing our method is available at <https://github.com/colemanyu/time-series-classification-cleavage>. The dataset of this study is available at <https://www.mirbase.org>.

In all experiments, the models were trained and tested using 5-fold cross-validation. We retrieved 827 empirically validated sequences of pre-miRNAs. There are 5p arm and 3p arm in each sequence. For each arm, we defined a cleavage pattern and a non-cleavage pattern. Three datasets, namely “5p arm”, “3p arm”, and “multi-class” were constructed by these patterns. We refer to the cleavage patterns as positive instances and the non-cleavage patterns as negative instances. The 5p arm dataset comprises 827 positive instances and an equal number of negative instances. The 5p arm and 3p arm datasets are binary-class datasets. The multi-class dataset comprises all patterns from both the 5p arm and the 3p arm. There are 827 “5p” instances¹¹, 827 “3p” instances, and 1,654 negative instances.

For every fold in 5-fold cross-validation, the dataset was divided into a training set and a test set with sizes of 80% and 20% of the whole dataset, respectively. We kept the class distribution approximately the same in each fold, since it is in the original dataset. In each fold derived from the 5p arm and 3p arm datasets, the training set has a size of 1,323, and the test set has a size of 331. In each fold derived from the multi-class dataset, the training set has a size of 2,262, and the test set has a size of 662. We reported the average of the five classification metrics.

The ROCKET-based classifiers require all channels in the multivariate time series to have equal length. We applied padding to the shorter channels using the constant value 100, which does not appear in the original time series. It ensures the padding does not introduce ambiguity or interfere with the semantic meaning of the encoded nucleotide signals.

3.3.1 Channel Ablation Study

We utilized three types of data as the input features for each instance. They are (1) the RNA sequence, which consists of the primary strand and its complementary strand, (2) the secondary structure information, and (3) the base-pair probability sequence. To input the data into our time series-based classifiers, we converted them into multivariate time series. The primary strand and its complementary strand are each encoded into one or two channels, using the encoding methods in Table 3.4. For example, single value mapping encodes a strand in one channel, while grouped variable-length channel mapping encodes in two channels. The secondary structure information is converted into a univariate time

¹¹Cleavage patterns from the 5p arm.

series. The base-pair probability sequence is already in numerical form and does not require further transformation. It can be used either as a standalone channel or incorporated into the encoding of the complementary strand. We performed a channel ablation study to determine the most informative combination of the above channels.

We referred to the multivariate time series that consists of the channels from the RNA sequence only as the baseline setting. We added the other channels to this baseline. It leads to the following configurations (cfgs):

1. (cfg 1) Baseline: Time series derived only from the RNA sequence.
2. (cfg 2) Baseline + Secondary structure: Baseline + time series representation of the secondary structure.
3. (cfg 3) Baseline + Base-pair probability (Standalone): Baseline + the base-pair probability sequence as a standalone channel.
4. (cfg 4) Baseline + Base-Pair probability (Incorporated): Baseline with the base-pair probability sequence incorporated into the encoding of the complementary strand.

	Classifier	5p arm					3p arm					multi-class				
		Acc	Sp	Sn	F1	MCC	Acc	Sp	Sn	F1	MCC	Acc	Sp	Sn	F1	MCC
Baseline (cfg 1)	ROCKET	0.781	0.743	0.819	0.789	0.563	0.790	0.773	0.807	0.793	0.580	0.717	0.838	0.685	0.700	0.538
	MiniROCKET	0.755	0.728	0.782	0.762	0.512	0.788	0.781	0.794	0.789	0.576	0.685	0.823	0.653	0.662	0.486
	MultiROCKET	0.784	0.767	0.801	0.787	0.569	0.803	0.792	0.814	0.805	0.606	0.691	0.830	0.667	0.672	0.501
	Hydra	0.830	0.800	0.860	0.835	0.663	0.808	0.797	0.820	0.810	0.617	0.731	0.844	0.696	0.712	0.560
	MultiROCKET-Hydra	0.796	0.778	0.815	0.800	0.594	0.807	0.767	0.816	0.808	0.614	0.701	0.836	0.681	0.686	0.520
Baseline + Secondary Structre (cfg 2)	ROCKET	0.847	0.832	0.862	0.849	0.695	0.855	0.842	0.868	0.857	0.711	0.836	0.907	0.828	0.833	0.736
	MiniROCKET	0.825	0.807	0.843	0.827	0.652	0.822	0.802	0.843	0.826	0.646	0.823	0.900	0.812	0.818	0.715
	MultiROCKET	0.812	0.803	0.822	0.814	0.626	0.824	0.809	0.839	0.826	0.649	0.796	0.888	0.791	0.792	0.673
	Hydra	0.845	0.816	0.873	0.849	0.691	0.846	0.817	0.874	0.850	0.693	0.830	0.901	0.814	0.826	0.724
	MultiROCKET-Hydra	0.817	0.809	0.826	0.819	0.635	0.825	0.816	0.834	0.826	0.652	0.803	0.891	0.798	0.800	0.684
Baseline + Base-pair probability (Standalone) (cfg 3)	ROCKET	0.842	0.828	0.855	0.844	0.684	0.855	0.856	0.854	0.855	0.710	0.795	0.885	0.783	0.789	0.670
	MiniROCKET	0.817	0.820	0.814	0.816	0.634	0.836	0.834	0.838	0.836	0.673	0.772	0.872	0.757	0.764	0.632
	MultiROCKET	0.822	0.813	0.832	0.824	0.645	0.825	0.831	0.820	0.824	0.651	0.758	0.866	0.747	0.750	0.612
	Hydra	0.846	0.827	0.865	0.849	0.693	0.851	0.840	0.861	0.852	0.702	0.789	0.879	0.769	0.780	0.658
	MultiROCKET-Hydra	0.822	0.809	0.834	0.824	0.644	0.835	0.840	0.830	0.834	0.670	0.759	0.866	0.746	0.750	0.611
Baseline + Base-pair probability (Incorporated) (cfg 4)	ROCKET	0.799	0.771	0.827	0.805	0.600	0.809	0.786	0.832	0.813	0.619	0.737	0.850	0.712	0.724	0.573
	MiniROCKET	0.776	0.756	0.797	0.781	0.554	0.801	0.808	0.794	0.799	0.603	0.705	0.835	0.675	0.684	0.521
	MultiROCKET	0.814	0.801	0.828	0.817	0.630	0.816	0.812	0.820	0.816	0.634	0.726	0.848	0.706	0.712	0.556
	Hydra	0.822	0.787	0.857	0.828	0.647	0.834	0.828	0.840	0.835	0.669	0.759	0.862	0.734	0.746	0.608
	MultiROCKET-Hydra	0.814	0.802	0.820	0.817	0.629	0.820	0.825	0.816	0.819	0.642	0.736	0.853	0.717	0.723	0.874

Table 3.6: Channel ablation study. The best results are highlighted in **bold**.

We used single value mapping as the encoding method. Table 3.6 shows the result. From the table, we can see that the addition of secondary structure, base-pair probability as a standalone channel, and base-pair probability incorporated in the encoding of the complementary strand can improve the performance. We plotted the critical difference (CD) diagram as shown in Figure 3.6 to visualize Table 3.6 to make the performances of different combinations more obvious. In

CD diagrams, lower-ranked methods (toward the right) are better. A horizontal bar connecting combinations indicates no statistically significant difference.

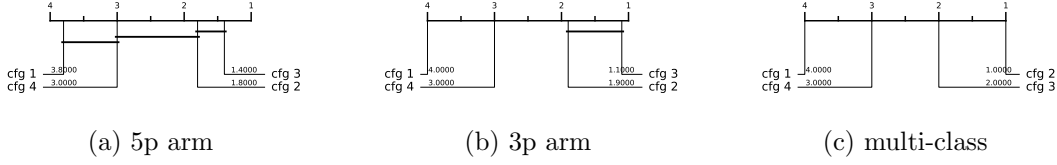


Figure 3.6: CD diagrams of channel ablation study.

From Figure 3.6, we can see that including time series derived from secondary structure information and base-pair probability as a separate channel can significantly improve the performance of the classifiers. Incorporating the base-pair probability sequence in the time series encoding of the complementary strand can also improve the classifier, but to a minor degree compared to serving as a standalone channel. In our downstream analysis, we adopted the combination of RNA sequence time series, secondary structure time series, and base-pair probability time series as our multivariate time series input, with 4 to 6 channels, depending on the encoding used.

3.3.2 Predictive Performance

The experiment was conducted on three datasets: the 5p arm, the 3p arm, and the multi-class datasets. Recall that we have nine encoding methods and five ROCKET-based classifiers. It results in 45 combinations of encoding methods and classifiers.

The result is shown in Table 3.7. The best combination of encoding method and classifier is shown in Table 3.8. For the 5p arm dataset, the best combination is “Global Cumulative grouped fixed-length channel mapping + ROCKET”. For all five classification metrics, it outperforms the state-of-the-art (SOTA) method, DiCleave. For the 3p arm dataset, the best combination is “Global Cumulative grouped fixed-length channel mapping + ROCKET”. Out of the five classification metrics, it outperforms DiCleave, except in specificity. For the multi-class dataset, the best combination is “Global Cumulative grouped fixed-length channel mapping + ROCKET”. For all five classification metrics, it outperforms DiCleave. Note that for the 3p arm and the multi-class datasets, the combination of “Cumulative grouped fixed-length channel mapping + ROKCET” also attains the best result.

To summarize Table 3.7, we plot the CD diagrams for finding the best classifier, as shown in Figure 3.7, and the best encoding method, as shown in Figure 3.8.

	Classifier	5p arm					3p arm					multi-class				
		Acc	Sp	Sn	F1	MCC	Acc	Sp	Sn	F1	MCC	Acc	Sp	Sn	F1	MCC
Single value mapping (enc 1)	ROCKET	0.849	0.842	0.857	0.851	0.699	0.863	0.854	0.873	0.865	0.727	0.853	0.917	0.847	0.851	0.764
	MiniROCKET	0.823	0.809	0.837	0.825	0.647	0.823	0.828	0.817	0.822	0.647	0.835	0.906	0.828	0.833	0.735
	MultiROCKET	0.821	0.802	0.840	0.824	0.643	0.839	0.826	0.852	0.841	0.679	0.811	0.894	0.806	0.809	0.697
	Hydra	0.843	0.820	0.867	0.847	0.688	0.838	0.819	0.857	0.841	0.677	0.831	0.901	0.815	0.827	0.727
	MultiROCKET-Hydra	0.820	0.803	0.837	0.823	0.640	0.840	0.830	0.850	0.841	0.680	0.816	0.896	0.810	0.814	0.704
Grouped variable-length channel mapping (enc 2)	ROCKET	0.835	0.826	0.844	0.836	0.670	0.855	0.849	0.861	0.856	0.710	0.846	0.913	0.839	0.844	0.752
	MiniROCKET	0.843	0.833	0.853	0.844	0.686	0.831	0.821	0.842	0.833	0.663	0.837	0.907	0.828	0.834	0.737
	MultiROCKET	0.819	0.809	0.828	0.820	0.638	0.817	0.814	0.820	0.818	0.634	0.890	0.894	0.806	0.808	0.695
	Hydra	0.825	0.780	0.869	0.832	0.653	0.811	0.769	0.854	0.819	0.626	0.818	0.892	0.765	0.812	0.705
	MultiROCKET-Hydra	0.818	0.814	0.822	0.819	0.636	0.831	0.825	0.837	0.832	0.662	0.820	0.900	0.815	0.818	0.710
Grouped fixed-length channel mapping (enc 3)	ROCKET	0.851	0.843	0.859	0.852	0.702	0.863	0.850	0.875	0.864	0.726	0.849	0.915	0.843	0.847	0.757
	MiniROCKET	0.844	0.836	0.853	0.845	0.689	0.840	0.826	0.855	0.843	0.682	0.851	0.915	0.844	0.849	0.760
	MultiROCKET	0.831	0.815	0.848	0.834	0.663	0.824	0.813	0.836	0.826	0.649	0.811	0.898	0.808	0.808	0.698
	Hydra	0.848	0.816	0.880	0.853	0.699	0.862	0.839	0.884	0.864	0.724	0.843	0.908	0.837	0.839	0.746
	MultiROCKET-Hydra	0.836	0.813	0.859	0.839	0.672	0.833	0.820	0.845	0.835	0.665	0.828	0.905	0.824	0.826	0.725
Cumulative mapping (enc 4)	ROCKET	0.850	0.834	0.866	0.852	0.701	0.863	0.855	0.871	0.864	0.726	0.852	0.915	0.842	0.850	0.762
	MiniROCKET	0.840	0.821	0.860	0.843	0.682	0.840	0.837	0.844	0.841	0.682	0.843	0.911	0.835	0.840	0.747
	MultiROCKET	0.822	0.809	0.834	0.824	0.644	0.832	0.830	0.834	0.832	0.665	0.820	0.898	0.810	0.816	0.709
	Hydra	0.848	0.819	0.878	0.853	0.698	0.853	0.856	0.869	0.855	0.705	0.845	0.910	0.830	0.841	0.749
	MultiROCKET-Hydra	0.824	0.811	0.856	0.825	0.647	0.838	0.833	0.843	0.839	0.677	0.821	0.898	0.810	0.817	0.711
Cumulative grouped variable-length channel mapping (enc 5)	ROCKET	0.843	0.821	0.866	0.847	0.688	0.856	0.840	0.871	0.857	0.712	0.855	0.916	0.843	0.851	0.766
	MiniROCKET	0.845	0.826	0.865	0.848	0.691	0.836	0.833	0.838	0.836	0.672	0.840	0.909	0.833	0.838	0.742
	MultiROCKET	0.826	0.814	0.838	0.824	0.653	0.815	0.820	0.810	0.814	0.631	0.826	0.902	0.800	0.824	0.721
	Hydra	0.850	0.819	0.880	0.854	0.701	0.834	0.807	0.861	0.838	0.669	0.833	0.903	0.818	0.829	0.731
	MultiROCKET-Hydra	0.824	0.810	0.838	0.826	0.649	0.833	0.833	0.833	0.833	0.666	0.830	0.903	0.821	0.827	0.726
Cumulative grouped fixed-length channel mapping (enc 6)	ROCKET	0.856	0.836	0.876	0.858	0.712	0.870	0.861	0.879	0.871	0.741	0.863	0.921	0.852	0.860	0.780
	MiniROCKET	0.856	0.837	0.874	0.858	0.712	0.842	0.839	0.845	0.843	0.685	0.845	0.912	0.837	0.843	0.751
	MultiROCKET	0.820	0.802	0.839	0.824	0.642	0.798	0.798	0.798	0.798	0.597	0.809	0.894	0.806	0.807	0.694
	Hydra	0.850	0.814	0.885	0.855	0.701	0.855	0.840	0.869	0.857	0.711	0.847	0.910	0.831	0.843	0.752
	MultiROCKET-Hydra	0.820	0.801	0.839	0.823	0.641	0.807	0.813	0.802	0.806	0.615	0.821	0.900	0.817	0.819	0.713
Cumulative mapping (enc 7)	ROCKET	0.850	0.834	0.867	0.844	0.701	0.853	0.838	0.867	0.854	0.706	0.856	0.917	0.845	0.853	0.768
	MiniROCKET	0.847	0.832	0.862	0.849	0.695	0.848	0.839	0.857	0.850	0.697	0.845	0.911	0.836	0.843	0.750
	MultiROCKET	0.827	0.819	0.834	0.828	0.653	0.847	0.842	0.853	0.848	0.695	0.825	0.901	0.817	0.822	0.718
	Hydra	0.851	0.821	0.880	0.855	0.703	0.861	0.848	0.874	0.863	0.722	0.847	0.911	0.834	0.844	0.753
	MultiROCKET-Hydra	0.829	0.823	0.834	0.830	0.658	0.843	0.838	0.849	0.844	0.688	0.832	0.905	0.823	0.829	0.730
Cumulative grouped variable-length channel mapping (enc 8)	ROCKET	0.840	0.814	0.867	0.844	0.682	0.853	0.838	0.867	0.854	0.706	0.856	0.917	0.845	0.853	0.768
	MiniROCKET	0.848	0.834	0.862	0.850	0.697	0.841	0.824	0.859	0.844	0.683	0.844	0.911	0.856	0.842	0.748
	MultiROCKET	0.834	0.828	0.839	0.834	0.668	0.831	0.821	0.842	0.833	0.663	0.828	0.904	0.823	0.826	0.724
	Hydra	0.857	0.821	0.894	0.862	0.717	0.822	0.786	0.857	0.828	0.645	0.826	0.898	0.806	0.820	0.717
	MultiROCKET-Hydra	0.837	0.834	0.839	0.837	0.674	0.834	0.827	0.840	0.835	0.668	0.835	0.907	0.828	0.832	0.734
Cumulative grouped fixed-length channel mapping (enc 9)	ROCKET	0.856	0.836	0.876	0.858	0.712	0.870	0.861	0.879	0.871	0.741	0.863	0.921	0.852	0.860	0.780
	MiniROCKET	0.857	0.845	0.870	0.859	0.715	0.840	0.821	0.859	0.843	0.681	0.844	0.911	0.837	0.842	0.749
	MultiROCKET	0.829	0.825	0.833	0.830	0.658	0.820	0.816	0.823	0.820	0.640	0.819	0.900	0.816	0.817	0.710
	Hydra	0.856	0.817	0.894	0.861	0.713	0.859	0.838	0.880	0.862	0.719	0.846	0.911	0.832	0.843	0.752
	MultiROCKET-Hydra	0.829	0.824	0.834	0.830	0.658	0.822	0.825	0.819	0.821	0.644	0.827	0.904	0.823	0.824	0.722

Table 3.7: Performance on the 45 combinations between encoding methods and the ROCKET-based classifiers. The best results are highlighted in **bold**.

Dataset	Methods	Acc	Sp	Sn	F1	MCC	Time (s)
5p arm	enc 9 + MiniROCKET	0.857	0.845	0.870	0.859	0.715	0.787
	DiCleave	0.818	0.790	0.846	0.822	0.653	21.249
3p arm	enc 9 + ROCKET	0.870	0.861	0.879	0.871	0.741	4.311
	enc 7 + MiniROCKET	0.848	0.839	0.857	0.850	0.697	0.989
	DiCleave	0.854	0.891	0.817	0.847	0.715	15.919
multi-class	enc 9 + ROCKET	0.863	0.921	0.852	0.860	0.780	12.208
	enc 3 + MiniROCKET	0.851	0.915	0.844	0.849	0.760	4.550
	DiCleave	0.820	0.895	0.804	0.815	0.710	131.151

Table 3.8: Comparative analysis between MTSCCleave with the best combination of the encoding method and classifier, with the SOTA, DiCleave, on the three datasets. The best results of using MiniROCKET have also been shown to compare the computational efficiency. The best results are highlighted in **bold**.

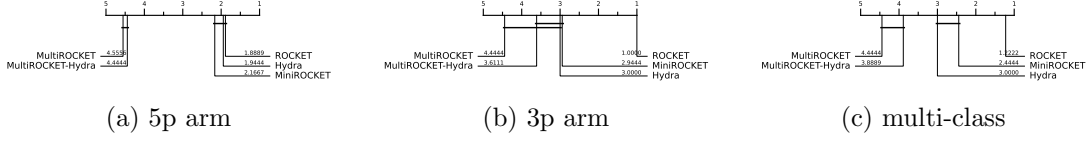


Figure 3.7: CD diagrams to compare different classifiers.

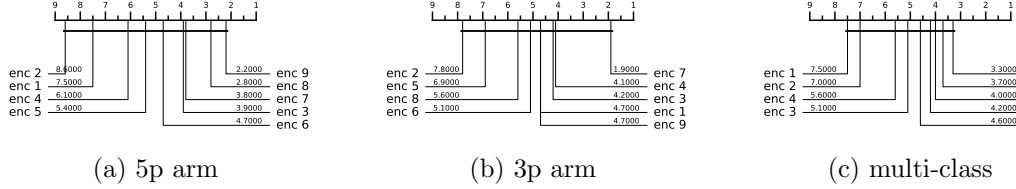


Figure 3.8: CD diagrams to compare different encoding methods.

3.3.3 Running Time Analysis

To compare the computational efficiency of MTSCCleave and DiCleave, we conducted a comparative analysis of their running times. For DiCleave, we employed the code from its supporting website¹², without any modifications. All experiments were conducted on the same machine (a personal laptop equipped with an Apple M1 Pro chip and 16 GB of memory) and using the same splits of the training and test datasets under 5-fold cross-validation to ensure fairness. The reported running times are the averages of the five runs. The timing results were measured from the training phase to the return of the five classification metrics. The result is shown in Table 3.8. MiniROCKET is the most computationally efficient of the five rocket-based classifiers. We also included its best result, along with the corresponding encoding method, even though this combination may not be the best overall.

MTSCCleave demonstrated a significant advantage in computational efficiency, achieving an average 27.0X, 3.7X, and 10.7X speedup over DiCleave, for the 5p arm, 3p arm, and multi-class datasets, respectively. If we consider using the MiniROCKET in the case of 3p arm and multi-class datasets, it achieves 16.1X and 28.8X speedup. To note, in the case of the 3p arm dataset, the performance of MiniROCKET is only slightly worse than DiCleave. In the case of the multi-class dataset, even the performance of MiniROCKET is better than DiCleave. DiCleave is a deep learning-based method that requires substantial time for model inference, while MTSCleave leverages efficient ROCKET-based classifiers. This significant

¹²

<https://github.com/MGuard0303/DiCleave> (Accessed on: 2025-07-13).

reduction in runtime makes MTSCCLeav more suitable for large-scale data and real-time applications.

3.3.4 Subsequence Importance

To evaluate the sensitivity of MTSCCLeav to subsequences of the input, we conducted a perturbation experiment to evaluate the importance of subsequences based on masking windows. The goal of this experiment is to identify which subsequences of the entire time series are critical for classification. We examine how various modifications to the original input impact model performance. It suggests which features are essential for classification.

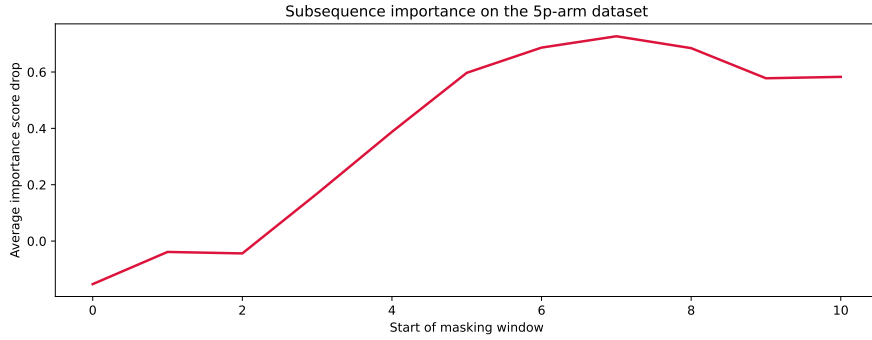
The model was trained on the original training dataset. For each instance in the test dataset, we measure its original score and the masked score. We slid a masking window w with a fixed length over the input time series T . $|w|$ was set to 4. For each window position $i \in \{1, 2, \dots, |T| - |w| + 1\}$, we masked all entries across all the channels of T within the window. Hence, we removed or hid that portion of information from the model during inference. The changes in classification performance in terms of accuracy relative to the unmasked original score of each i are recorded. Intuitively, if the information of a subsequence is critical for the classification, the masking of this subsequence would lead to a great drop in classification performance. We aggregated the importance score across the test dataset.

The result is shown in Figure 3.9. For the encoding methods, we cannot use the methods derived from the cumulative mapping because the accumulation would leak information from the masked region. We adopted “Grouped fixed-length channel mapping” as the encoding method and ROCKET as the classifier. “Grouped fixed-length channel mapping” is the best encoding, other than the methods derived from the cumulative mapping, in all datasets, as shown in Figure 3.8. ROCKET is the best classifier, as shown in Figure 3.7..

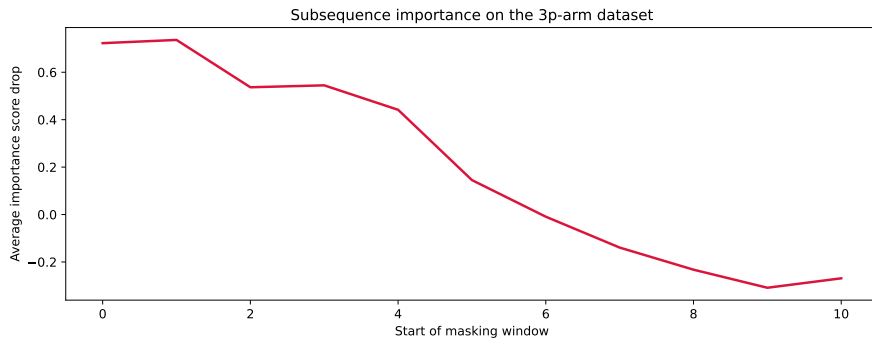
In the 5p arm dataset, we found that masking subsequences at the tailing part caused a significant drop in the importance score, as shown in Figure 3.9 (a). In the 3p arm dataset, we found that masking subsequences at the leading part caused a significant drop in the importance score, as shown in Figure 3.9 (b).

3.3.5 Summary

Our method achieves better or comparable predictive results and a 3.7X to 28.8X speedup compared to the state-of-the-art (SOTA).



(a) 5p arm



(b) 3p arm

Figure 3.9: Results of the perturbation experiment.

3.4 Discussion

The channel ablation study reveals that the involvement of the time series derived from the secondary structure can improve accuracy. It suggests the importance of RNA folding in dicer processing. Furthermore, we found that the base-pair probability sequence of the secondary structure can also enhance accuracy. To the best of our knowledge, it is a novel application of the base-pair probability sequence. Experiments show that using the probability sequence as an additional channel can enhance accuracy more than incorporating it in the encoding. It is likely because keeping it as an additional channel can preserve more information, of both the probability sequence itself and the complementary strand.

Out of the three datasets, the best classifier is ROCKET. The ranking of the five classifiers by performance, starting from the best, is as follows: ROCKET, Hydra, MiniROCKET, MultiROCKET-Hydra, and MultiROCKET. It indicates that the features created from the pooling operations that are only in MultiROCKET but not in MiniROCKET, confuse the final classifier. They are mean of positive values (MPV), mean of indices of positive values (MIPV) and longest stretch of

positive values (LSPV) [42]. In contrast, the pooling operator that is only present in ROCKET but not in MiniROCKET, enhances the classification performance. It is maximum (MAX).

For the encoding methods, we have the following observations. Fixed-length grouped channel mappings outperform variable-length counterparts with one exception in the multi-class dataset, likely because fixed-length schemes better preserve the original positional information of nucleotides within the sequence. Global cumulative methods consistently yield better performance than local cumulative methods. It suggests that the upstream information of the cleavage pattern plays a critical role in identifying cleavage sites. Cumulative-based encodings perform better than single-value mappings, with one exception in the 3p dataset, suggesting that the accumulated nucleotide signal is more informative for cleavage site prediction than the local or isolated presence of nucleotides. In the 5p arm dataset, encoding RNA sequence in two channels appears to worsen the result. This suggests that the 5p arm dataset and the 3p arm dataset need different nucleotide grouping methods for the encoding.

One limitation of DiCleave is overfitting during training because of the relatively small size of the dataset [20]. DiCleave is a deep learning-based method. Deep learning models typically require a large amount of training data to generalize effectively. They are data-hungry. In contrast, MTSCCleave leverages ROCKET-based methods for the classification. They rely on random convolutional feature extraction followed by a simple linear classifier. The Ridge classifier used in this study is less data-hungry compared to deep learning methods due to its use of L2 regularization and the simplicity of its linear model nature. It allows ROCKET-based classifiers, and hence MTSCCleave, to maintain strong predictive performance even in settings with a relatively small dataset size.

The subsequence importance reveals some connections between RNA secondary structure and human dicer cleavage site prediction. The perturbation experiment shows that the leading part of 5p arm and the tailing part of 3p arm are important for the classification. These parts are close to the center of the RNA secondary structure of pre-miRNA. It indicates that the center region is more crucial for human dicer cleavage site prediction. It is consistent with the previous study [19].

3.5 Concluding Remarks

We proposed an accurate, fast, and simple multivariate time series classification (MTSC)-based method, termed MTSCCleave, for predicting human dicer cleavage

sites. Base-pair probability sequences of the secondary structures have also been leveraged in the classification. MTSCCLeav consists of three parts: time series encoding, time series transformation, and classification. ROCKET-based methods were used for time series transformation. Ridge Classifier was used for classification. For the computational experiments, we evaluated nine time series encoding methods in conjunction with five time series transformation methods. MTSCCLeav outperformed the SOTA method in all five evaluation metrics for the 5p-arm and multi-class datasets, and four of the metrics for the 3p-arm dataset. In terms of computational efficiency, MTSCCLeav with the optimal setting achieved an average 3.7X to 27.0X speedup over the SOTA method on the three datasets. With the use of a less accurate but faster time series classification method, MTSCCLeav achieved an average speedup of 16.1X to 28.8X, respectively. We analyzed the subsequence importance of the input multivariate time series. The results show that subsequences near the center of the pre-miRNA sequences are more important. This aligns with the findings from previous work. This study demonstrates that time series analysis provides a powerful alternative to conventional modeling in the context of RNA processing. This framework may be extended to other RNA-processing tasks. Notably, the encoding of RNA sequence into time series enables us to utilize any well-established tools from the time series community.

Chapter 4

Scaling with Multiple Scaling Factors in Time Series Searching

Time series data are ubiquitous across many different fields. Many data mining tasks, such as classification, clustering, and motif finding, have been defined for time series data. They utilize similarity search as a core subroutine, making it crucial to design similarity measures that align with our intuitions. To facilitate efficient computation, speedup techniques are essential. Dynamic Time Warping (DTW) is arguably the most prevailing distance measure for time series data. However, studies have shown that for certain data, another distance measure, namely Uniform Scaling (US), is equally crucial as DTW. DTW handles the local distortion, while US handles the global scaling. In addition, studies have demonstrated that combining DTW and US is necessary to obtain meaningful results in some cases. Surprisingly, all existing studies employ only a single scaling factor for the entire time series. A time series could consist of phases. Since each phase of a time series expresses at its own rate, using a single scaling factor is insufficient when comparing two time series that share similar phases but differ in their expression rates. We introduce the first framework that accounts for multiple scaling factors, Piecewise Scaling Distance (PSD). PSD employs other existing distance measures as subroutines. Because the naive implementation of PSD is slow, we propose a constrained version of PSD that enforces constraints based on the allowed segment lengths derived from the given scaling factor bound. It also prevents pathological results. In addition, two other speedup techniques have been proposed, which achieve 10.10X to 191.46X speedup. We also demonstrate the usage of a lower bound when DTW is used as the subroutine of PSD. More-

over, we show that the segmentation results returned by PSD can improve the accuracy of other distance measures.

4.1 Background

To study the mechanism of a process, we take measurements. Measurements are usually taken continuously by the sensors. Measurements of processes always yield continuous values at discrete timestamps. They are time series data. For example, smartphones collect users' GPS data. ECG monitors measure patients' heart rate. The continuous measurements compose a time series. It is not hard to see why time series data are ubiquitous across many different fields. In GPS data, each time series data point consists of the user's latitude and longitude information. They are multivariate time series. In ECG data, each data point represents the amplitude of the patient's cardiac electrical activity. They are univariate time series. In this study, we focus on univariate time series.

Many data mining tasks can be defined on time series data. For example, given a time series database, we can perform clustering based on the pairwise similarity of the time series instances. A classifier can be trained when categorical labels are available. Alternatively, given a long time series, for motif finding, we identify recurring patterns. In contrast, for anomaly detection, we identify abnormal subsequences. Almost all time series data mining tasks can be reduced to arguing the similarity between two time series. A good distance measure, also known as a similarity measure, can determine the success or failure of the algorithms built on it. The choice of an appropriate distance measure is particularly evident in classification. Studies show that simple nearest-neighbor classification (1-NN) is difficult to beat and can compete with more complex methods [38].

A time series is treated as a whole rather than as a collection of individual values. The relationships between values are important. They constitute trends and shapes. Hence, similarity search in time series data is approximate-based rather than exact match-based [46]. Besides, different invariances should be allowed during the comparison.

Dynamic Time Warping (DTW) is one of, if not the most common, similarity measures. DTW provides invariance to time distortion by aligning and measuring the similarity between two series that may be misaligned in time. However, it assumes that the time series are expressed on a similar global expression rate. This assumption limits its performance when comparing two time series expressed at different global expression rates. We often see this behavior in domains such as

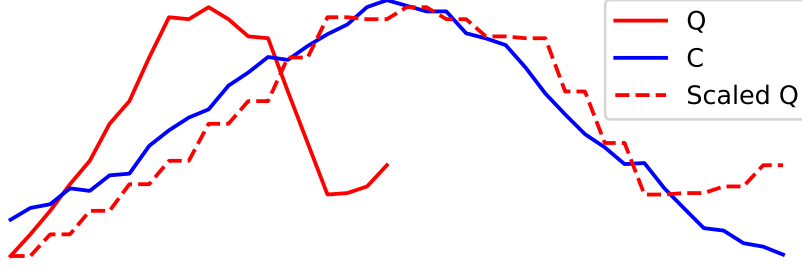


Figure 4.1: Applying nearest neighbor interpolation on Q , which result in Scaled Q , that can better reflect its similarity with C .

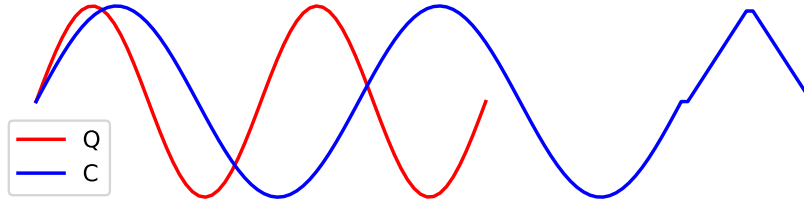


Figure 4.2: Q and C are in different rates. A stretching version of Q is similar to a prefix of C , but not the whole C .

speech recognition, motion analysis, patient biomedical signals, and sensor data in the manufacturing industry.

Uniform Scaling (US) can achieve global scaling invariance by scaling the two time series to the same length via interpolation, such as nearest-neighbor interpolation, before comparison, as shown in Figures 4.1. It is reported that in some domains, such as gestures [47, 48] and music performance [49], the scaling is about 10-15% (i.e., scaling factors: 1.1 to 1.15). The scaling factors are relatively small, since the nature of the music and the gait will change with significant scaling factors. However, in some other domains, we may encounter larger scaling factors. In bioinformatics, gene expression time series data could differ by a factor of 1.41 [50, 51]. In Figure 4.2, Q and a prefix of C are similar, but at different rates. In searching, we typically have a query Q and a longer candidate C . We seek a prefix of C that is close to Q . For better comparison, we need to eliminate the scaling effect. These observations demonstrate the necessity of uniform scaling.

DTW and US are used to achieve different kinds of invariance. DTW handles local distortion, while US handles global scaling. Furthermore, some studies show that the combination of US and DTW, namely USDTW, better reflects similarity [52, 53, 54]. US is first applied to transform the two time series into the same length to eliminate the effect resulting from the different rates. Then, DTW,

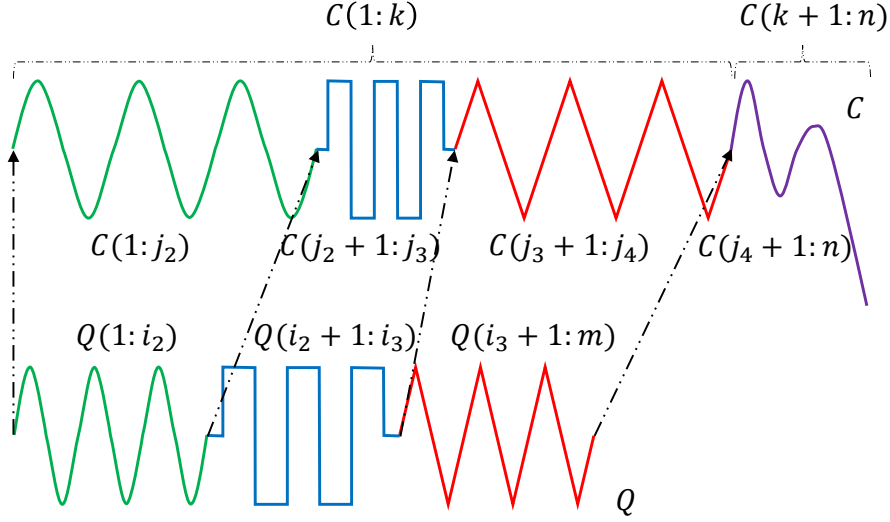


Figure 4.3: Intuition of piecewise scaling (PS).

rather than ED, is applied to address local misalignment. USDTW is computationally more expensive than DTW because it involves the calculation of the DTW between Q and different lengths of each prefix of C . The different lengths of the prefixes correspond to different scaling factors.

It is not uncommon for the data sampling strategy to change over time [55]. There are different phases, each with its own rate. To achieve invariance for this kind of scaling effect resulting from multiple rates, rather than using a single scaling factor, it is beneficial to identify these different phases and use the appropriate scaling factors for these segments, also known as pieces. We refer to this as piecewise scaling (PS). Figure 4.3 shows the intuition of PS. The prefix of C (i.e., $C(1 : k)$) and Q share the same set of segments, but each has a different scaling. Multiple scaling factors must be used. It motivates us to design a new distance measure or framework that considers applying a scaling factor on each of the phases as defined by dashed lines in Figure 4.3, during the comparison of two time series.

Our contributions are as follows:

- We propose the first framework to achieve piecewise scaling (PS) invariance. In particular, we focus on two instantiations of PSD, namely PSED (i.e., ED with PS invariance) and PSDTW (i.e., DTW with PS invariance).
- We design a dynamic programming method to compute PSD.
- We propose a constrained version of PSD (cPSD) based on the allowed segment lengths. Besides, two other speedup techniques have been proposed.

For a particular instantiation of PSD, PDTW, we demonstrate the usage of a lower bound to further speed it up.

- We demonstrate that the segmentation results returned by PSD can improve the accuracy of other distance measures.

The rest of this study is structured as follows. We present related work in Section 5.2 and preliminaries in Section 5.3. Section 5.4 introduces our new distance measure framework, its constrained version, and speedup techniques. It is experimentally demonstrated in Section 4.5 for the problem of querying. In Section 5.6, we conclude this study with some future work.

4.2 Related Work

This study focuses on distance measures of time series. For the overall review of time series, we direct the readers to [46, 56] for a more comprehensive understanding of this field.

For many tasks, having appropriate distance measures that align with our intuition for the domains we work with is essential. One well-known distance measure is Dynamic Time Warping (DTW). It is initially designed for speech analysis [2]. However, DTW is computationally expensive. Lower bounds are used to speed up time series similarity search by admissibly pruning the unpromising candidates. One of the popular exact lower bounds of DTW is LB_{Keogh} . [57] improves the scalability of DTW by introducing a subsequence search suite of their four novel ideas, namely the UCR suite. For an overall review of lower bounds, we refer readers to [58, 59]. There is an approximate algorithm that approximates DTW with high accuracy while drastically cutting down the time and space requirements [60].

While ED is sensitive to distortions in the time axis, uniform scaling (US) has been shown to be a critical invariance in domains such as motion capture. [61] demonstrated that DTW is insufficient for handling global scaling effects, and that identifying DTW is not the solution to achieve this kind of invariance. There is a need for US. [62] extends the importance of uniform scaling to motif discovery. The authors show that meaningful motifs often suffer from a global scaling effect, causing standard motif finding algorithms to miss them completely.

To the best of our knowledge, three studies analyze the combination of US and DTW, namely USDTW. It was first proposed by [52]. It extended LB_{Keogh} to bound the USDTW. However, the extended LB_{Keogh} is still too loose with

invariance to large amounts of uniform scaling. [53] and its follow-up study [54] proposed a new lower bound ¹, namely LB_{Shen} , which has been shown to be tighter than LB_{Keogh} on USDTW.

To our surprise, despite a fruitful discussion of DTW, US, and USDTW, no study has proposed a distance measure capable of handling scaling effects across multiple scaling factors. This is precisely what we will address in this study.

4.3 Preliminaries

We refer to time as the contextual attribute because it provides the context for the measurements to be made. We refer to the measurements as the behavioral attributes. Time series are multivariate when more than one behavioral attribute is present. Otherwise, it is called univariate. We focus on the univariate case.

Definition 1 (Time Series). A time series $T = t_1, t_2, \dots, t_n$ is a sequence of real-valued numbers with length $= n$.

When two time series are involved in the discussion, we denote them as Q (Query) and C (Candidate), with lengths m and n , respectively. Since Q is the query sequence, it is not longer than C (i.e., $m \leq n$). The requirement of “ $m \leq n$ ” is a natural setting. In “Query by Content”, a user is going to search for a candidate in the database from a user-input query in which the query may only contain partial information of the target candidate. For example, a user often wants to find a song or tune that is lingering in their head by humming a part but not whole of the tune [63]. We are also interested in a segment or subsequence of a time series.

Definition 2 (Subsequence). A subsequence $T(i : j)$ of a time series T is a shorter time series that starts from position i and ends at position j with length $= j - i + 1$. Both ends are inclusive. Formally, $T(i : j) = t_i, t_{i+1}, \dots, t_j$, $1 \leq i \leq j \leq n$.

We call $T(1 : j)$ the prefix of T of length j .

Before comparison, we need to standardize or normalize them. A common way is Z-Normalization, which is $T = (T - \text{mean}(T)) / \text{std}(T)$, as shown in Figure 4.4. The most fundamental distance measure is the Euclidean Distance (ED).

¹It is denoted as LB_{New} in the original study. We rename it to prevent any confusion.

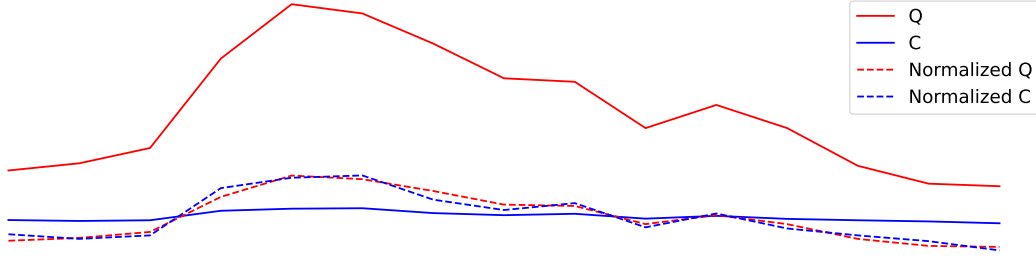


Figure 4.4: Z-normalization. Resulting time series have mean = 0 and std = 1.

4.3.1 Euclidean Distance (ED)

Given two points on a plane, it is intuitive to define the distance between them as the length of the line segment between them. This idea extends to the case of n -dimensions in time series with length n .

Definition 3 (Euclidean Distance (ED)). Given two series Q and C both with length n , the Euclidean Distance between them is defined as:

$$\text{ED}(Q, C) = \sqrt{\sum_{i=1}^n (q_i - c_i)^2} \quad (4.1)$$

A square root is usually involved in the computation of distance measures. It is a monotonic function. Since it does not change the relative ranking of the results, we can omit the square root operation for simplicity and optimization. ED aligns the entries between two series in a one-to-one manner. Two similar series will have a large distance under ED if they are not aligned well in the time dimension. ED cannot handle local distortion along the time axis because warping in the alignment is not allowed. A standard distance measure that provides warping invariance is called Dynamic Time Warping (DTW).

4.3.2 Dynamic Time Warping (DTW)

Dynamic Time Warping (DTW) is an algorithm that measures similarity between two time series while accounting for local distortions. DTW aligns the two series by nonlinearly warping the time axis to minimize the final cumulative distance.

Given two time series, Q and C , we first construct an m by n distance matrix M . The origin $(1, 1)$ is set at the bottom-left element of M . The (i, j) element of M contains the distance $d(q_i, c_j)$ between points q_i and c_j . This local distance

$d(\cdot, \cdot)$ is usually calculated by $(q_i - c_j)^2$. Each (i, j) element refers to an alignment or mapping of the two points. A contiguous set of such elements forms a warping path W . W represents the non-linear alignment of Q and C . $W = w_1, w_2, \dots, w_K$ in which $w_k = (i, j)_k$ represents the mapping between q_i in Q and c_j in C , where $\max(m, n) \leq K \leq m+n-1$. “ $\max(m, n) \leq K$ ” because the alignment of two series must include every point in both Q and C . “ $K \leq m+n-1$ ” because the longest warping path is either the “concatenation of the bottom row and the rightmost column” (with the bottom-right cell being overlapped) or the “concatenation of the leftmost column and the top row” (with the top-left cell being overlapped).

The warping path W is typically subject to the following three constraints.

- Boundary conditions: $w_1 = (1, 1)$ and $w_K = (m, n)$. The first (last) point of Q must map to that of C .
- Continuity: Given $w_k = (i, j)$ and $w_{k-1} = (i', j')$, $i - i' \leq 1$ and $j - j' \leq 1$. An entry in the warping path W is adjacent to its one-step previous entry.
- Monotonicity: Given $w_k = (i, j)$ and $w_{k-1} = (i', j')$, $i - i' \geq 0$ and $j - j' \geq 0$. The warping path W does not go back.

We denote W^* as a set of all allowed possible paths.

Definition 4 (Dynamic Time Warping (DTW) [2]). DTW returns the minimum warping cost:

$$\text{DTW}(Q, C) = \min_{W \in W^*} \sum_{k=1}^{|W|} w_k \quad (4.2)$$

To find the minimum warping cost and its corresponding warping path, we can use dynamic programming (DP) to evaluate the following recurrence.

$$D(i, j) = d(q_i, c_j) + \min \begin{cases} D(i-1, j-1), \\ D(i-1, j), \\ D(i, j-1) \end{cases} \quad (4.3)$$

It can be solved by building the accumulated cost matrix D , where the y-axis refers to Q , and the x-axis refers to C . The base cases, which are the first row and the first column, are defined as $D(1, j) = \sum_{k=1}^j d(q_1, c_k)$, $j \in [1, n]$ and $D(i, 1) = \sum_{k=1}^i d(q_k, c_1)$, $i \in [1, m]$. After we have initialized the base cases, we

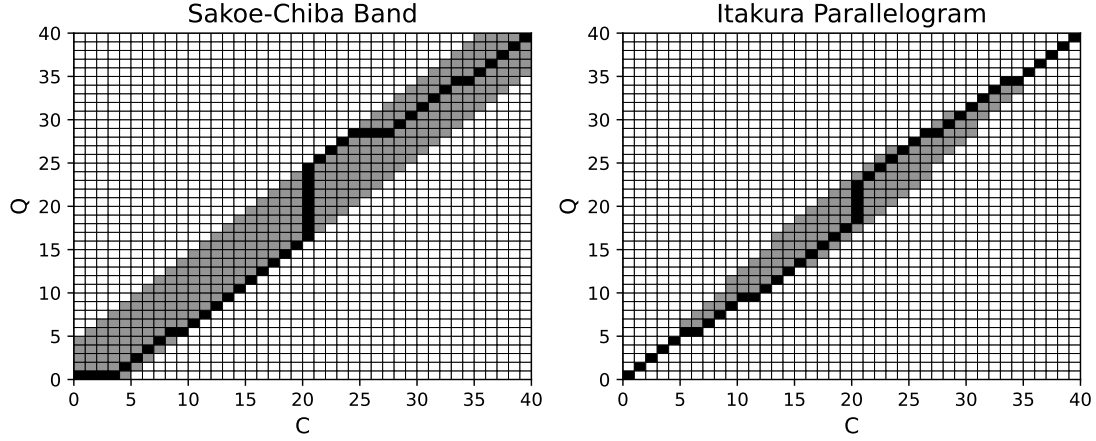


Figure 4.5: Visualization of D with local constraints. The black cells form the warping path.

can fill up D starting from the bottom up. There are $m \times n$ entries in D . It takes $\mathcal{O}(mn)$ to fill it up. Once D is built, we can find the path corresponding to this minimum cost by simple backtracking from the end cell $D(m, n)$ to the origin cell $D(1, 1)$.

Some constraints have been proposed to prevent pathological warping paths with the aim of accuracy and efficiency. Two of the most popular are Sakoe-Chiba Band [2] and Itakura Parallelogram [64]. They only allow the warping paths to pass through the allowed region, as shown in Figure 4.5, by setting the cells outside this region in D to have ∞ accumulated distance cost. [65] suggested that these constraints can be viewed as constraints on the warping path entry $w_k = (i, j)_k$. It represents these constraints as inequalities applying to the indices i and j locally, without depending on the main diagonal in D . In the Sakoe-Chiba Band, the constraints can be represented as $j - r \leq i \leq j + r$, where r is an integer. It is sometimes specified as a fraction (or percentage) of the longer time series length to ensure length invariance. For clarity, we assume r is an integer unless specified otherwise. This means that q_i can only align with c_j if their indices differ by at most r . Since r defines the maximum allowed difference between the mapping indices, $|n - m| \leq r$, to ensure that the end points of Q and C can map. In the Itakura Parallelogram, r is a function of i rather than a constant value. ED can be seen as a special case of DTW where the warping path is fixed to be diagonal. q_i aligns to c_i for every i (i.e., $r = 0$). DTW minimizes over all possible warping paths, and the warping path of ED is one of them. Because of the band, we only need to fill up the cells within the band. The complexity is $\mathcal{O}(\#_of_cells_inside)$.

In the case of the Sakoe-Chiba Band, the band has a constant width $w = 2r + 1$. The complexity becomes $\mathcal{O}(w \times \text{length_of_diagonal}) = \mathcal{O}(rn)$. The constrained DTW is denoted as DTW_r .

4.3.3 Uniform Scaling (US)

In US, we compare the whole sequence of Q to a prefix of C , as shown in Figure 4.2. The two compared sequences are scaled to the same length via interpolation before ED is applied. A common interpolation method is nearest neighbor interpolation.

Definition 5 (Nearest Neighbor Interpolation). Given a time series T of length n and an integer L , Nearest Neighbor Interpolation scales T into T^L as follows:

$$T_j^L = T_{\lceil n(j/L) \rceil} \quad \text{where } 1 \leq j \leq L \quad (4.4)$$

We can scale up or down a given series using Equation 4.4, as shown in Example 1.

Example 1. Given a series $T = 1, 2, \dots, 6$ with length $n = 6$. Let $T(1 : 4)$ be the prefix of T of length $k = 4$ (i.e., $T(1 : 4) = 1, 2, 3, 4$). Given an integer $L = 8$, we compute $T(1 : 4)^8$ as follows.

$$\begin{aligned} T(1 : 4)^8 &= T_{\lceil 4(1/8) \rceil}, T_{\lceil 4(2/8) \rceil}, T_{\lceil 4(3/8) \rceil}, \dots, T_{\lceil 4(8/8) \rceil} \\ &= T_1, T_1, T_2, \dots, T_4 \\ &= 1, 1, 2, \dots, 4. \end{aligned}$$

When $L > k$, T is said to be stretched. When $L < k$, T is said to be shrunk.

Definition 6 (Uniform Scaling (US) [61]). Given two series Q and C , of length m and n respectively, and a scaling factor bound l , where $l \geq 1$. Let $C(1 : k)$ be the prefix of C , where $\lceil m/l \rceil \leq k \leq \min(\lfloor lm \rfloor, n)$, and $C(1 : k)^L$ be a rescaled version of $C(1 : k)$ with length L , where $L = \min(\lfloor lm \rfloor, n)$. L is called the alignment factor. $\min(\lfloor lm \rfloor, n)$ is the largest alignment factor.

$$\text{US}(Q, C, l, L) = \min_{k=\lceil m/l \rceil}^{\min(\lfloor lm \rfloor, n)} \text{ED}(Q^L, C(1 : k)^L) \quad (4.5)$$

L is set as the largest alignment factor [54] to ensure all the points in Q and $C(1 : k)$ are preserved during interpolation because of up-sampling, and the scaled version of all the prefixes is going to have the same length (i.e., L), for fair

comparison, since comparison between two longer time series generally results in a larger distance measure. Through Equation 4.5, we find the minimum value and the corresponding argument (i.e., k) by checking the minimum value of the ED function from $\lceil m/l \rceil$ to $\min(\lfloor lm \rfloor, n)$. The scaling factor is defined by the argument minimum value of k . The smaller k is, the more we need to “stretch” C for Q to compare with $C(1 : k)$, which is m/k times.

Consider a time series database D comprising a set of candidate instances, and a single query series Q . The search task aims to retrieve the most similar instance (or top- k instances) from D to Q . We maintain Q as a fixed reference and extract only the prefixes from each instance in D for comparison. Furthermore, to simplify notation, we apply scaling exclusively to the instances in D , leaving Q unscaled.

4.3.4 Uniform Scaling & Dynamic Time Warping (USD-TW)

Uniform Scaling & Dynamic Time Warping (USD-TW) measures the similarity by applying scaling with an appropriate scaling factor on C and then applying DTW. The tail part of C can be ignored, as shown in Figure 4.2.

Definition 7 (Uniform Scaling & Dynamic Time Warping (USD-TW) [54]). With the same notations defined in Definition 6,

$$\text{USD-TW}_r(Q, C, l, L) = \min_{k=\lceil m/l \rceil}^{\min(\lfloor lm \rfloor, n)} \text{DTW}_r(Q^L, C(1 : k)^L) \quad (4.6)$$

where r is the DTW constraint parameter.

We replace the ED function in Equation 4.5 by DTW function to form Equation 4.6.

As mentioned in Section 5.1, there may exist more than one scaling factor. Hence, both the existing distance measures, US and USD-TW, which are designed to handle only one scaling factor, are insufficient. This motivates us to design a new framework of distance measures.

4.3.5 Lower Bounds for DTW and USD-TW

We first introduce the concept of lower bounds and explain how they can benefit search. DTW is computationally more expensive than ED. They compute an

accumulated cost matrix of size $m \times n$, where m and n denote the lengths of the two time series. It results in quadratic complexity. This complexity poses a challenge for similarity search. The most common approach is to compute a lower bound of the real value, which is computationally cheap and tight. We can use this lower bound to filter out unpromising candidates and perform the expensive distance computation only on the small set of promising candidates. In searching, we would keep track of the best_so_far distance bsf between the query Q and the testing candidates. When testing a new candidate C , we first compute the $\text{LB}(Q, C)$. We only compute the actual DTW when $\text{LB}(Q, C) \leq \text{bsf}$.

We then introduce some common lower bounds.

Kim Lower Bound [66]: LB_{Kim} is a simple and fast lower bound of DTW. The complexity is $O(1)$. It uses the four features in Q and C . We denote t_{-1} as the last point and $t_{\max}$ ($t_{\min}$) as the maximum (minimum) point in time series T .

$$\text{LB}_{\text{Kim}} = \max \begin{cases} d(q_1, c_1) \\ d(q_{-1}, c_{-1}) \\ d(q_{\max}, c_{\max}) \\ d(q_{\min}, c_{\min}) \end{cases} \quad (4.7)$$

$d(\cdot, \cdot)$ refers to the local distance used in the point alignment. The first two lines come from the boundary condition in DTW. The alignment between the first pair of points and the last pair of points must contribute to the accumulated sum of DTW. Each point in Q must align with some point in C , and vice versa. Each alignment contributes a local distance to the final sum. The minimum possible local distance for the alignment of $q_{\max}$ would be the one aligned with $c_{\max}$. The same applies to the last line in the equation.

There is a simplification of LB_{Kim} . Only the first and last pair are used in the lower bound computation. In the normalized time series, the third and fourth rows in Equation 4.7 should have small values [57]. Ignoring them can improve the complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$. The simplified lower bound is $\text{LB}_{\text{KimFL}} = d(q_1, c_1) + d(q_{-1}, c_{-1})$.

Keogh Lower Bound [65]: LB_{Keogh} builds the upper U and lower envelopes L of one of the time series out of the two compared sequences. Usually, the envelopes are constructed for Q instead of C as we will compare one Q against many C 's. Otherwise, we need to build the envelopes for each candidate instead [57].

To the best of our knowledge, they are the first to interpret the constraint as a restriction on the indices of the warping path $w_k = (i, j)_k$ such that $j - r \leq i \leq$

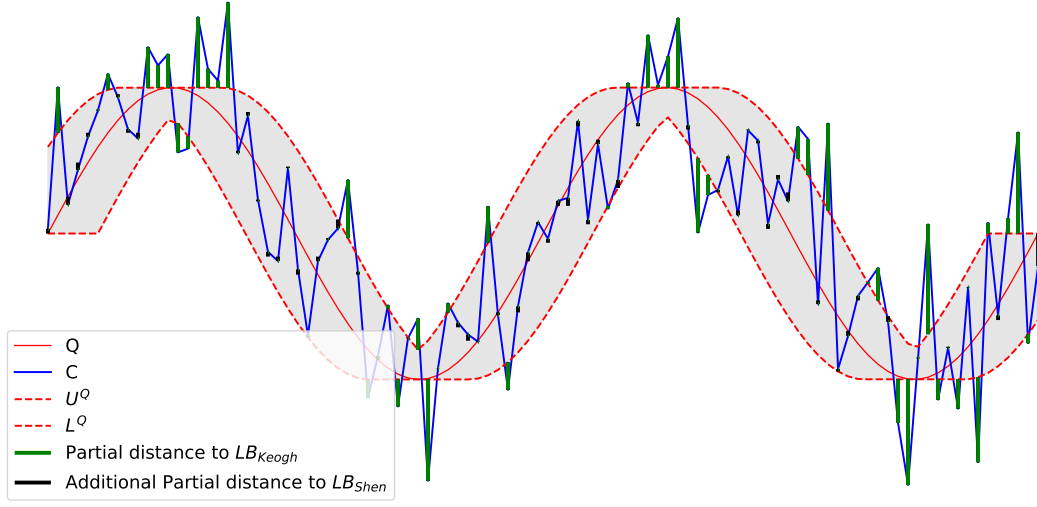


Figure 4.6: Visualization of LB_{Keogh} and LB_{Shen}

$j + r$, where r defines the allowable deviation of alignment between q_i and c_j . For ease of exposition, we focus on the most used constraint in the literature, which is the Sakoe-Chiba Band [67]. Two sequences are constructed for Q , namely the upper U^Q and lower envelopes L^Q of Q as shown in Figure 4.6. For each q_i , we would assign a window of q_i based on its index i as follows.

$$\begin{aligned} U_i^Q &= \max(q_{\max(1, i-r)} : q_{\min(i+r, m)}) \\ L_i^Q &= \min(q_{\max(1, i-r)} : q_{\min(i+r, m)}) \end{aligned} \quad (4.8)$$

$\max(1, \cdot)$ and $\min(\cdot, m)$ are used for handling the boundary cases. U^Q and L^Q together form a bounding envelope that encloses the original Q , it is the grey region in the figure. For each c_j , either (c_j, U_j^Q) or (c_j, L_j^Q) corresponds to the possible alignment that contributes the minimum distances if c_j falls outside the envelope. Herein, the lower bound is the sum of these distances, as shown in the following equation.

$$LB_{Keogh}(Q, C) = \sum_{j=1}^n \begin{cases} d(c_j, U_j^Q) & \text{if } c_j > U_j^Q \\ d(c_j, L_j^Q) & \text{if } c_j < L_j^Q \\ 0 & \text{otherwise} \end{cases} \quad (4.9)$$

Visually, these distances are the distances between c_j outside the envelope and the vertically corresponding points on the envelope. The distances are the green bars in the figure. Equation 4.9 returns the sum of the green bars.

Shen Lower Bound [53, 54]: It leverages the boundary and continuity conditions to create a lower bound of DTW. It can be used to lower-bound the USDTW with slight modification. The intuition is to find the minimum possible alignment of each c_j with points in Q , subject to the local constraint r from DTW and the global constraint l from US.

We will first introduce LB_{Shen} for DTW. Given the candidate sequence C with length n , for each c_j , we create its possible reach $\mathfrak{q}_j$ in Q under DTW as in Equation 4.10. The elements $\mathfrak{q}_j$ form an indexed collection $\mathbb{Q} = (\mathfrak{q}_1, \mathfrak{q}_2, \dots, \mathfrak{q}_{\min(\lfloor lm \rfloor, n)})$.

$$\mathfrak{q}_j = (q_{\max(1, j-r)}, \dots, q_{\min(j+r, m)}) \quad (4.10)$$

We define $\delta(x, Y) = \min_{y \in Y} d(x, y)$. The possible minimum cost contributed by the alignment of c_j and some point in Q to the accumulated sum would be $\delta(c_j, \mathfrak{q}_j)$. The lower bound LB_{Shen} is defined as:

$$\text{LB}_{\text{Shen}}(Q, C) = d(c_1, q_1) + \sum_{j=2}^{n-1} \delta(c_j, \mathfrak{q}_j) + d(c_n, q_m) \quad (4.11)$$

We direct the reader to [53] for the formal proof, while an intuition of the proof is presented here. There are three items on the right-hand side of Equation 4.11. The continuity requirement ensures that each c_i is involved in at least one alignment. The first and the last items are from the boundary condition. The middle item returns the possible minimum distances contributed by each c_j 's, where $2 \leq j \leq n-1$. It is obvious that LB_{Shen} in Equation 4.11 is tighter when we use the first pair of points and the last pair of points instead of doing the middle summation all from $j = 1$ to n . It is because the distance contributed by the first pair (last pair) must be greater than $\delta(c_1, \mathfrak{q}_1)$ ($\delta(c_n, \mathfrak{q}_n)$).

It is proven that it is tighter than LB_{Keogh} [53]. It is shown in Figure 4.6 visually. The black bars refer to the additional lower bound distance sum on top of LB_{Keogh} . We will give an intuitive proof here. Both LB_{Keogh} and LB_{Shen} compute the lower bounds by summing the local distance resulting from the alignment of each c_j with some points in Q , which is guaranteed to be not greater than the partial distance contributed by the real alignment, which we only know until we compute the exact distance measure. In LB_{Keogh} , if this c_j falls outside the envelope, this local distance is the vertical distance between c_j and the envelope. If c_j falls inside, this local distance would be 0. For those points outside the envelope, the local distances for c_j of LB_{Keogh} and LB_{Shen} are the same. But LB_{Shen} aims to return the

minimum possible partial distance for each c_j , even within the envelope. Hence, $\text{LB}_{\text{Keogh}} \leq \text{LB}_{\text{Shen}}$.

In order to compute Equation 4.11 efficiently, we first sort sequences $\mathbf{q}_j$ and the resulting sorted sequences are denoted as $\tilde{\mathbf{q}}_j$. The sorted sequences allow us to do a binary search when we are calculating $\delta(c_j, \tilde{\mathbf{q}}_j)$ in contrast to $\delta(c_j, \mathbf{q}_j)$. The sorting only needs to be done once because we are testing the same Q with different candidates.

It can be extended to the USDTW case [53, 54]. The possible reach now is not only defined by r , but also by the scaling factor bound l .

$$\mathbf{q}_j = (q_{\max(1, \lceil j/l \rceil - r)}, \dots, q_{\min(\lfloor jl \rfloor + r, m)}) \quad (4.12)$$

[53] proves the following theorem to allow us to consider the lower bound between each prefix of C and Q without the scaling up of each prefix of C (i.e., $C(1 : k)^L$) and the scaling up of Q (i.e., Q^L).

Theorem 8. *For any $\lceil m/l \rceil \leq k \leq \min(\lfloor lm \rfloor, n)$, $\text{DTW}_r(Q^{\min(\lfloor lm \rfloor, n)}, C(1 : k)^{\min(\lfloor lm \rfloor, n)})$ is always lower bounded by $\sum_{j=1}^k \delta(c_j, \mathbf{q}_j)$.*

Recall that USDTW calculates DTW distances between each rescaled prefix of C to the rescaled Q , and outputs the minimum DTW distance, as in Equation 4.6. The incremental nature of the lower bound frees us to calculate the lower bound of each DTW distance from scratch. This theorem allows us to first calculate the lower bound of the shortest prefix $C(1 : \lceil m/l \rceil)$ of C and Q , and then incrementally calculate the lower bound of the longer prefix with length from $\lceil m/l \rceil + 1$ to $\min(\lfloor lm \rfloor, n)$ by adding on each $\delta(c_j, \mathbf{q}_j)$. To note, it also means that if $\text{LB}_{\text{Shen}}(Q, C(1 : k))$ is larger than a value, namely bsf, $\text{LB}_{\text{Shen}}(Q, C(1 : k'))$, where $k' > k$, would also be larger than bsf. To note, we can tighten LB_{Shen} by using $d(c_1, q_1)$ instead of $\delta(c_1, \mathbf{q}_1)$.

The above analysis can also apply to the case of US distance. We only need to define the corresponding reaches as follows:

$$\mathbf{q}_j = (q_{\max(1, \lceil j/l \rceil)}, \dots, q_{\min(\lfloor jl \rfloor, m)}) \quad (4.13)$$

4.4 Piecewise Scaling Distance

The motivation stems from the limitations of US and USDTW. They assume that the relationship between Q and C is governed by only a single, global scaling

factor. This assumption fails when applied to multi-rate data, where different phases of the time series express at different rates.

Note that existing studies [61, 57] can find all scaling factors, defined by the chosen prefix of C , ranging from $\lceil m/l \rceil$ to $\min(\lfloor lm \rfloor, n)$. However, they can use only one of them in the scaling. Consider the following illustrative example in ASCII text [57], where character repetition represents the duration of spoken phonemes, and space indicates a pause:

- “time series 20 25” and “time series 20 25”. Here, the Hamming distance (the discrete analogue of ED) fails due to misalignment in the underlined locations, but the string edit distance (the discrete analogue of DTW) can resolve it.
- “time sseerriiss 222000222555”. This sequence exhibits three distinct phases with scaling factors of 1, 2, and 3, respectively. The corresponding invariance cannot be achieved by DTW or US (which is restricted to a single global scalar). US would enforce a single compromised global scale instead.

In Query-by-Content scenarios, such as query-by-humming or gesture retrieval, the query is generated by a human. Humans do not maintain a consistent rate for each phase. For example, humans rush through a familiar sequence but slow down for a new or complex sequence. It is commonly observed in a piece of music performed by a beginner. The piece’s tempo is not uniform.

We introduce a novel distance measure framework, termed Piecewise Scaling Distance (PSD). It addresses the local scaling effect within each phase by employing a scaling factor to each phase, instead of using a single scaling factor for the whole time series. It releases the basic constraint or assumption made in US and USDTW. PSD employs an existing distance metric, such as ED or DTW, to quantify the similarity of aligned segment pairs. While the PSD framework is agnostic to the underlying metric, this study focuses on its two fundamental instantiations:

- PSED, which employs ED to compute the similarity of aligned segment pairs.
- PSDTW, which employs DTW to compute the similarity of aligned segment pairs.

In the formulation that follows, we utilize PSDTW as the running example. PSDTW generalizes PSED. The PSED formulation can be trivially derived from it.

4.4.1 Piecewise Scaling & Dynamic Time Warping (PS-DTW)

Problem formulation: To simplify the discussion, we focus on the comparison between Q and the entire sequence C , rather than a prefix of C . Note that this formulation can be generalized to prefix matching as in Definition 6 and Definition 7.

Given two sequences Q and C , where Q is not longer than C , $|Q| = m \leq |C| = n$, and the number of segments or pieces P allowed, our goal is to segmentalize both Q and C into P contiguous segments automatically in a way that minimize the total sum of DTW distance of aligned segment pairs with interpolation.

Definition 9 (Piecewise Scaling & Dynamic Time Warping (PSDTW)). With the same notations defined in Definition 7,

$$\begin{aligned} \text{PSDTW}_r(Q, C, l, L, P) = \\ \min_{\substack{i_1 < i_2 < \dots < i_{P+1} \\ j_1 < j_2 < \dots < j_{P+1}}} \sum_{p=1}^P \text{DTW}_r(Q(i_p + 1 : i_{p+1})^L, \\ C(j_p + 1 : j_{p+1})^L) \end{aligned} \quad (4.14)$$

, where $i_1 = 0$, $i_{P+1} = m$, $j_1 = 0$, $j_{P+1} = n$ and the setting of L will be discussed later.

Essentially, L needs to be at least the length of all segments to preserve all the points by up-sampling.

We refer to Figure 4.3 to clarify Equation 4.14. Given two sequences Q and C , with the aid of different colors and the dashed lines, we observe that Q consists of three segments while C consists of four segments, with the first three segments of C similar to those of Q . They form three segment pairs. The scaling factor used in each segment pair is determined from the length of the two subsequences involved, i.e., $(i_{p+1} - i_p)/(j_{p+1} - j_p)$. These three parts have different scaling factors. For example, the first part in C is the stretched version of that in Q . The second part in C is the compressed version of that in Q .

Equation 4.14 can be formulated in a recurrence relation as follows. Let $D[i, j, p]$ be the minimum cost to align the first i points in Q (i.e., $Q(1 : i)$) with the first j points in C (i.e., $C(1 : j)$) using exactly p segments:

Algorithm 1 Naive PSDTW

Input: Query series Q , Candidate series C , DTW constraint parameter r (in fraction), Number of pieces P , Scaling parameter L

Output: Distance Matrix D of size $(m+1) \times (n+1) \times (P+1)$

```
1: Initialize  $D$  with  $\infty$ 
2:  $D[0, 0, 0] \leftarrow 0$ 
3: for  $p \leftarrow 1$  to  $P$  do
4:   for  $i \leftarrow 1$  to  $m$  do
5:     for  $j \leftarrow 1$  to  $n$  do
6:        $D[i, j, p] \leftarrow \min_{\substack{i' < i \\ j' < j}} \{ D[i', j', p-1] \\ + \text{DTW}_r(Q(i'+1:i)^L, C(j'+1:j)^L) \}$ 
7: return  $D[m, n, P]$ 
```

Algorithm 2 Line 6 in Algorithm 1

```
1: for  $i' \leftarrow 0$  to  $i-1$  do
2:   for  $j' \leftarrow 0$  to  $j-1$  do
3:      $\text{dist}_{\text{prev}} \leftarrow D[i', j', p-1]$ 
4:      $\text{dist}_{\text{seg}} \leftarrow \text{DTW}_r(Q(i'+1:i)^L, C(j'+1:j)^L)$ 
5:      $D[i, j, p] \leftarrow \min(D[i, j, p], \text{dist}_{\text{prev}} + \text{dist}_{\text{seg}})$ 
```

$$D[i, j, p] = \min_{\substack{i' < i \\ j' < j}} \left\{ D[i', j', p-1] + \text{DTW}_r(Q(i'+1:i)^L, C(j'+1:j)^L) \right\} \quad (4.15)$$

Naive PSDTW:

Our goal is $D[m, n, P]$. Equation 4.15 can be solved exactly by dynamic programming (DP). The base case is $D[0, 0, 0] = 0$. It refers to the zero cost to align the first 0 point (i.e., the empty prefix) of Q with that of C . Other cells in D are first initialized with ∞ . They are calculated using a bottom-up approach via Equation 4.15. A straightforward implementation in DP is shown in Algorithm 1. Line 6 is achieved by looping all the previous indices of the current i and j as in Algorithm 2.

We explain the lines in Algorithm 2. Line 3 retrieves the accumulated distance cost from the beginning up to the endpoints (i', j') , and saves it as $\text{dist}_{\text{prev}}$. Line 4 considers the current aligned segment pair, which consists of $Q(i'+1:i)$ and $C(j'+1:j)$, and they are interpolated to the length L . The DTW distance of this pair is calculated and saved as dist_{seg} .

Algorithm 3 Initialization of PSDTW

- 1: $L_{\text{gmin}}^Q \leftarrow \lceil (m/P)/\sqrt{l} \rceil$, $L_{\text{gmax}}^Q \leftarrow \lfloor (m/P)\sqrt{l} \rfloor$ $\triangleright$ “g” refers to “global”.
 - 2: $L_{\text{gmin}}^C \leftarrow \lceil (n/P)/\sqrt{l} \rceil$, $L_{\text{gmin}}^C \leftarrow \lfloor (n/P)\sqrt{l} \rfloor$
 - 3: $L = \max(L_{\text{gmax}}^Q, L_{\text{gmin}}^C)$
 - 4: Initialize D of size $(m+1) \times (n+1) \times (P+1)$ with ∞
 - 5: $D[0, 0, 0] \leftarrow 0$
-

We now analyze the time complexity of Algorithm 1. There are Pmn entries in D . The $\min$ operator in line 6 takes $\mathcal{O}(mn)$. Hence, the time complexity of Algorithm 1 is $\mathcal{O}(Pm^2n^2) = \mathcal{O}(Pn^4)$, multiplied by the running time of the DTW. It is slow, which prevents us from using it in practice. To note, we use r in a fraction instead of a fixed integer here. This allows the deviation tolerance to scale adaptively with pieces of varying lengths.

4.4.2 Speedup Techniques

Length constraints of the segment: A way to reduce complexity and to prevent pathological segment pairs is to limit the possible segment lengths that are considered by constraining the minimum and maximum lengths of segments. It is similar to constrained DTW, in which we limit the search space of the warping path as in Figure 4.5. The version of PSDTW that considers the segment constraint is termed constrained PSDTW (cPSDTW). It is shown in Algorithm 4. The uncolored part shows the main logic, while the colored part shows the speedup techniques, which will be explained later.

We initialize in Algorithm 3. For a given number of segments P , the expected length of each segment in Q and C would be m/P and n/P , respectively. For Q , we set the minimum possible segment length L_{gmin}^Q to be $\lceil (m/P)/\sqrt{l} \rceil$ and the maximum possible length L_{gmax}^Q to be $\lfloor (m/P)\sqrt{l} \rfloor$ in line 1 such that the scaling ratio of any two segments in Q would be bounded by l . It allows some deviation in the length of the segments from their expected length. Similarly, we compute the segment constraints for C in line 2. We set the maximum segment length be the alignment factor L in line 3. We set the base case in line 5.

We fill the table D in Algorithm 4. In line 3, given p pieces in Q , the ending index of the last piece (i.e., the p^{th} piece) will range from $(p \cdot L_{\text{gmin}}^Q)$, given all the p pieces are in minimum length L_{gmin}^Q , to $\min(p \cdot L_{\text{gmax}}^Q, m)$, given all the p pieces are in maximum length L_{gmax}^Q .

In line 4, we enumerate for all the allowed lengths L^Q . For the segment with length L^Q in Q , the length L^C of the corresponding aligned segment in C will

Algorithm 4 Constrained PSDTW (cPSDTW) with early abandoning and lower bounding

Input: Query series Q , Candidate series C , DTW constraint parameter r (in fraction), Number of pieces P , best_so_far bsf

Output: The final distance $D[m, n, P]$ if $D[m, n, P] \leq \text{bsf}$, otherwise ∞

```

1: Execute Algorithm 3 for initialization
2: for  $p \leftarrow 1$  to  $P$  do
3:   for  $i \leftarrow (p \cdot L_{\text{gmin}}^Q)$  to  $\min(p \cdot L_{\text{gmax}}^Q, m)$  do            $\triangleright$  The iterations can be parallelized.
4:     for  $L^Q \leftarrow L_{\text{gmin}}^Q$  to  $L_{\text{gmax}}^Q$  do
5:        $i' \leftarrow i - L^Q$                                             $\triangleright i'$ : End point of previous segment on  $Q$ .
6:        $Q' \leftarrow \text{rev}(Q(i' + 1 : i))$                               $\triangleright \text{rev}(T)$ : Reverse the input series  $T$ .
7:        $L_{\text{min}}^C \leftarrow \max(L_{\text{gmin}}^C, \lceil L^Q/l \rceil)$ 
8:        $L_{\text{max}}^C \leftarrow \min(\lfloor L^Q/l \rfloor, L_{\text{gmax}}^C)$ 
9:        $r' \leftarrow r \times \max(L^Q, L_{\text{max}}^C)$                               $\triangleright$  Compute the integer value of  $r$ .
10:      for  $k \leftarrow 1$  to  $L_{\text{max}}^C$  do
11:         $\mathbb{Q}_k \leftarrow (q'_{\max(1, \lceil k/l \rceil - r')}, \dots, q'_{\min(\lfloor kl \rfloor + r', L^Q)})$   $\triangleright$  Construct indexed collection  $\mathbb{Q}$ .
12:         $\tilde{\mathbb{Q}}_k \leftarrow \text{sort}(\mathbb{Q}_k)$ 
13:      for  $j \leftarrow (p \cdot L_{\text{gmin}}^C)$  to  $\min(p \cdot L_{\text{gmax}}^C, n)$  do
14:        for  $L^C \leftarrow L_{\text{min}}^C$  to  $L_{\text{max}}^C$  do
15:           $j' \leftarrow j - L^Q$ 
16:           $C' \leftarrow \text{rev}(C(j' + 1 : j))$ 
17:           $\text{dist}_{\text{prev}} \leftarrow D[i', j', p - 1]$ 
18:          if  $\text{dist}_{\text{prev}} = \infty$  then
19:            continue
20:          if  $\text{dist}_{\text{prev}} > \text{bsf}$  then
21:            continue
22:          if  $\text{dist}_{\text{prev}} > D[i, j, p]$  then                              $\triangleright D[i, j, p]$  stores the best so far.
23:            continue
24:          if  $L^C = L_{\text{min}}^C$  then                                        $\triangleright$  Compute the lower bound from sketch.
25:             $lb = (c'_1 - q'_1)^2$   $\triangleright$  Partial distance contributed by the first alignment.
26:            for  $k \leftarrow 2$  to  $L^C$  do
27:               $lb = lb + \delta(c'_k, \tilde{\mathbb{Q}}_k)$ 
28:          else
29:             $lb = lb + \delta(c'_{L^C}, \tilde{\mathbb{Q}}_{L^C})$   $\triangleright$  Compute the lower bound incrementally.
30:             $lb_{\text{check}} = lb - \delta(c'_{L^C}, \tilde{\mathbb{Q}}_{L^C}) + (c'_{-1} - q'_{-1})^2$   $\triangleright$  Tighten the lower bound
31:              by using the last alignment.
32:            if  $\text{dist}_{\text{prev}} + lb_{\text{check}} > D[i, j, p]$  then
33:              continue
34:             $\text{dist}_{\text{seg}} \leftarrow \text{DTW}_r(Q'^L, C'^L)$ 
35:            if  $\text{dist}_{\text{prev}} + \text{dist}_{\text{seg}} < D[i, j, p]$  then
36:               $D[i, j, p] \leftarrow \text{dist}_{\text{prev}} + \text{dist}_{\text{seg}}$   $\triangleright$  Also save pointer  $(i', j')$  for the cut.
36: return  $D[m, n, P]$ 

```

have a range from L_{min}^C to L_{max}^C , as defined in lines 7–8, such that the ratio of L^Q and L^C would be bounded by l . Because L^Q and L^C increase in line 4 and line 14, and the i and j are fixed by the outer loop, in line 3 and line 13, respectively, the i' and j' decrease correspondingly. It is visualized in Figure 4.7. A segment in Q with ending index i and starting index $i' + 1$ would be compared to a set of segments in C , all of which have a fixed end at j and a decreasing starting index,

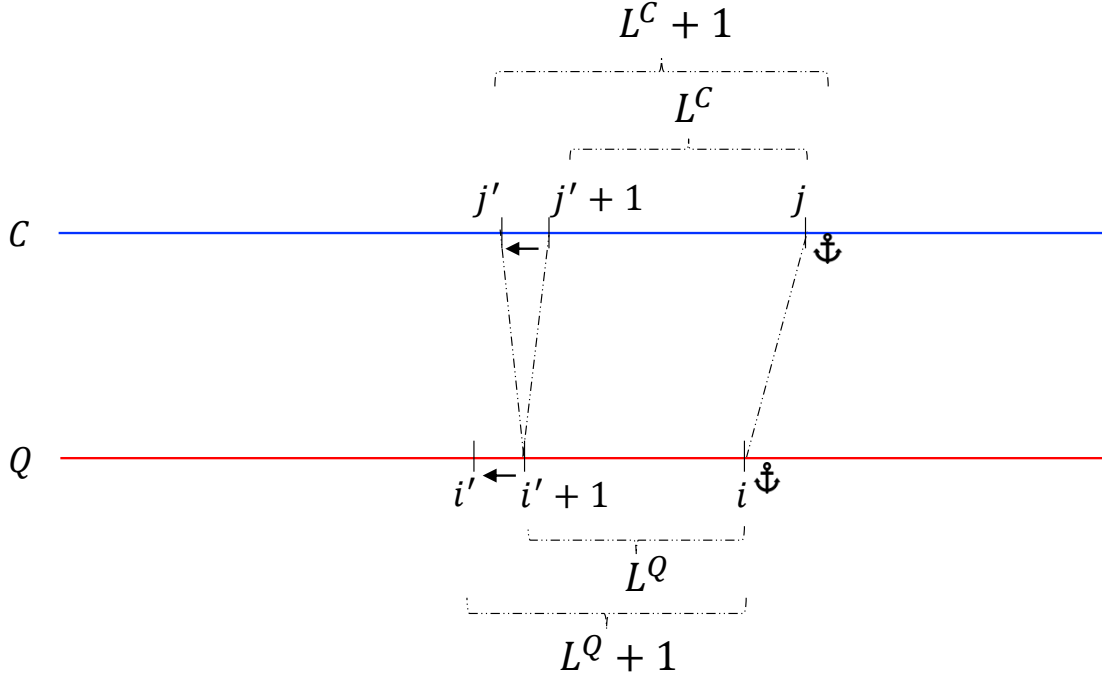


Figure 4.7: Relationship of L_Q and L_C .

starting at $j' + 1$. For example, the starting index of the first segment with being compared is $j' + 1$, which has length L^C , and that of the second segment is j' , which has length $L^C + 1$, as in Figure 4.7.

In line 18, we terminate the current iteration if there are no previously valid segment pairs (i.e., $dist_{\text{prev}} = \infty$).

In line 22, if the accumulated cost $dist_{\text{prev}}$ exceeds the current best so far, we stop the current iteration as the resulting $(dist_{\text{prev}} + dist_{\text{seg}})$ is guaranteed to be greater than the current best so far, which is stored in $D[i, j, p]$.

To be consistent with the result of using the lower bound speedup technique, which will be introduced later, we compute the DTW in the reverse manner in line 33. Due to the nearest neighbor interpolation, $DTW_r(Q'^L, C'^L)$ may not equal to $DTW_r(\text{rev}(Q')^L, \text{rev}(C')^L)$.

In line 35, we also save the pairs (i', j') that serve as cutting points between segments to obtain the segmentation result.

Parallel computing: We observe that the recurrence relation for the state $D[i, j, p]$ at stage p depends exclusively on the states computed at stage $p - 1$, as shown in Equation 4.15. There are no stage dependencies over indices i and j . It allows us to parallelize the loops over indices i and j on lines 3 and 13. In the following experimental section, we distribute the i-loop iterations (i.e., line 3 in

Algorithm 4, which is colored in blue) across available threads only because there are already sufficient iterations from the i -loop to fill the available threads. There are $\left(\min(p \cdot L_{\text{gmax}}^Q, m) - (p \cdot L_{\text{gmin}}^Q) + 1\right)$ iterations from the i -loop. The parallel execution of the i -loop is implemented by `prange` in Numba in Python.

Early Abandoning in nearest neighbor search: To accelerate the nearest neighbor search (or top- k search) for a query Q on a candidate set, we employ an early abandoning strategy. It prunes the search branch within the PSD computation of a specific candidate C as soon as the result corresponding to this search branch is determined to be suboptimal. We maintain a variable, `bsf` (best-so-far), which represents the minimum final distance among the candidates processed with Q so far. `bsf` serves as an upper-bound threshold. During the evaluation of a new candidate C , we monitor the accumulated partial distance, $\text{dist}_{\text{prev}}$. If $\text{dist}_{\text{prev}}$ exceeds `bsf`, the corresponding final distance is guaranteed to exceed `bsf`. In such cases, the current search branch is immediately terminated. The implementation of this pruning mechanism is detailed in lines 20–21 in Algorithm 4, which are colored in orange. In the case of top- k search, we must maintain the top- k final distances and use the k -th distance as the threshold.

Lower bound:

When we use DTW as a routine in PSD, the computation of the DTW of the interpolated segment can be sped up by computing LB_{Shen} for the lower bound. From Figure 4.7, we observe that a segment of Q , with length L^Q , is compared to a set of growing segments of C , with a fixed end at j . The length of these segments is from L_{min}^C to L_{max}^C , as indicated in line 14 in Algorithm 4. They share the same suffix with length L_{min}^C . It encourages us to view both Q and C reversely. The reversed segments are denoted as Q' and C' , as in lines 6 and 16 in Algorithm 4. In this reversed view, they share the same prefix with length L_{min}^C . Hence, we construct an indexed collection $\tilde{Q}$ for Q' with the maximum length of the segments being compared in C , which is L_{max}^C , as in lines 10–12.

In lines 24–27, we first compute the lower bound between Q' and the shortest segment of C . We add the minimum possible contribution of each c' to the distance contributed by the first alignment, which is $lb = (c'_1 - q'_1)$. For the lower bound of the subsequent segments, we compute them incrementally in line 29. Because the last point of segment C' must map to the last point of Q' , we further tighten the lower bound by using the last alignment in line 30 instead.

Furthermore, we can reduce the computational overhead of constructing the sorted reaches $\tilde{q}_k$ (lines 9–12). As illustrated in Figure 4.7, the segment of Q involved in the comparison grows incrementally. Since the reversed versions of these

Algorithm 5 Replace lines 3 to 9 in Algorithm 5 to reuse the computed sorted reaches $\tilde{q}_k$.

```

1:  $r'_{\text{cache}} \leftarrow -1$ 
2: for  $i \leftarrow (p \cdot L_{\text{gmin}}^Q)$  to  $\min(p \cdot L_{\text{gmax}}^Q, m)$  do
3:   for  $L^Q \leftarrow L_{\text{gmin}}^Q$  to  $L_{\text{gmax}}^Q$  do
4:      $i' \leftarrow i - L^Q$ 
5:      $Q' \leftarrow \text{rev}(Q(i' + 1 : i))$ 
6:      $L_{\text{min}}^C \leftarrow \max(L_{\text{gmin}}^C, \lceil L^Q/l \rceil)$ 
7:      $L_{\text{max}}^C \leftarrow \min(\lfloor L^Q/l \rfloor, L_{\text{gmax}}^C)$ 
8:      $r' \leftarrow r \times \max(L^Q, L_{\text{max}}^C)$ 
9:     if  $r'_{\text{cache}} = r'$  then
10:      for  $k \leftarrow 1$  to  $L_{\text{max}}^C$  do
11:        if  $k \leq |Q|$  then
12:           $e_{\text{prev}} \leftarrow \min(\lfloor kl \rfloor + r', L^Q - 1)$ 
13:           $e_{\text{new}} \leftarrow \min(\lfloor kl \rfloor + r', L^Q)$ 
14:          if  $e_{\text{new}} > e_{\text{prev}}$  then
15:             $\tilde{q}_k \leftarrow \tilde{q}_k \cup q'_{e_{\text{new}}}$ 
16:          else
17:             $q_k \leftarrow (q'_{\max(1, \lceil k/l \rceil - r')}, \dots,$ 
18:               $q'_{\min(\lfloor kl \rfloor + r', L^Q)})$   $\triangleright L^Q$  equals to  $|Q'|$ .
19:             $\tilde{q}_k \leftarrow \text{sort}(q_k)$ 
20:          else
21:            for  $k \leftarrow 1$  to  $L_{\text{max}}^C$  do
22:               $q_k \leftarrow (q'_{\max(1, \lceil k/l \rceil - r')}, \dots,$ 
23:                 $q'_{\min(\lfloor kl \rfloor + r', L^Q)})$ 
24:               $\tilde{q}_k \leftarrow \text{sort}(q_k)$ 
25:             $r_{\text{cache}} \leftarrow r'$ 

```

segments share a common prefix, the sorted reaches $\tilde{q}_k$ computed for a segment Q' of length L^Q can be reused to compute those for the subsequent segments. This optimization is detailed in Algorithm 5, with the new components highlighted in blue. It is important to note that because r' is a function of L^Q , and the construction of $\tilde{q}_k$ depends on r , reuse is limited to cases where r' remains constant. We construct reaches $\tilde{q}_k$ from the sketch in lines 20–22 and keep track of the r' used for construction in line 23. If r' remains constant, we can use previously computed reaches $\tilde{q}_k$ to compute the new set of reaches $\tilde{q}_k$. Since the ending index of reaches depends on $\min(\lfloor kl \rfloor + r', L^Q)$, we need to check whether we have a new ending index when L^Q increases. If the ending index has been changed, we need to add the new data points $q'_{e_{\text{new}}}$ to the sorted sequence $\tilde{q}_k$, as in line 15.

L_{max}^C depends of L^Q . If L_{max}^C increase because L^Q increase, we construct the new reach $\tilde{q}_k$ and sort it in lines 16–18.

4.4.3 Guided Distance

For faster computation, one would want to use a distance measure with linear complexity, such as ED, as the base measure for PSD. While PSED is effective for identifying phase-scaling changes, certain applications require capturing complex properties within those segments that ED cannot handle. There are two ways to address it. One approach is to use an alternative distance measure that captures these complex properties as the base distance for computing the PSD, such as DTW. But PSDTW is slower than PSED. The other approach is to use the segmentation result returned by PSED. To address this, we propose a two-stage framework in which PSED-derived segmentation guides the application of advanced distance metrics M . Let $\mathcal{P}^* = \{(i_1, j_1), (i_2, j_2), \dots, (i_{P+1}, j_{P+1})\}$ be the set of optimal cut points on Q and C obtained by minimizing the PSED. We utilize $\mathcal{P}^*$ to partition both series into P aligned pairs of segments. The final distance, denoted as M^{PSED} , is calculated by summing the distances of these pairs using a target metric M :

$$M^{\text{PSED}}(Q, C) = \sum_{p=1}^P M(Q_p^L, C_p^L) \quad (4.16)$$

, where $Q_p = Q(i_p + 1 : i_{p+1})$ and $C_p = C(j_p + 1 : j_{p+1})$.

4.5 Experiments

We evaluate the performance of our proposed distance measure framework, PSD, via its two instantiations: PSED and PSDTW. Specifically, we focus on a query retrieval task in which the query exhibits piecewise-scaled distortion, that is, distinct phases of the query exhibit different expression rates relative to the target in the candidate dataset. Our objective is to verify whether PSD achieves invariance to these multiple scaling distortions, thereby allowing it to correctly retrieve the most similar candidate. We detail the experimental setup in Section 4.5.1 and present the results in Section 4.5.2. The code and data are available at <https://github.com/colemanyu/k-scaling-factor-dtw>.

4.5.1 Experimental Setup

We utilize the GunPoint dataset, originally released in 2003 [68], as the running example. Widely regarded as the “iris” dataset of the time series community [69],

it has appeared in over one thousand publications. Beyond its ubiquity, the dataset addresses a critical distinction: misinterpreting the act of aiming a gun as merely pointing a finger could have life-threatening consequences.

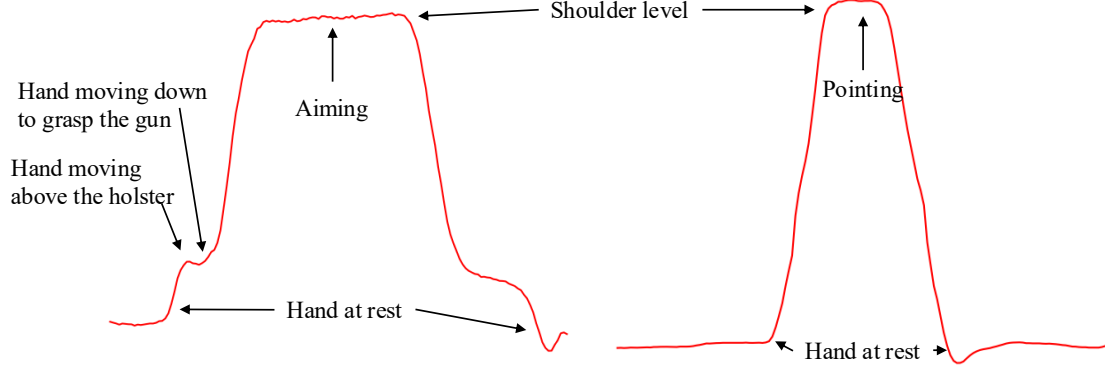


Figure 4.8: Visualization of the GunPoint dataset. Left (Right): A time series of the “Gun” (“Point”) scenario. Critical periods, such as “Aiming”, are annotated.

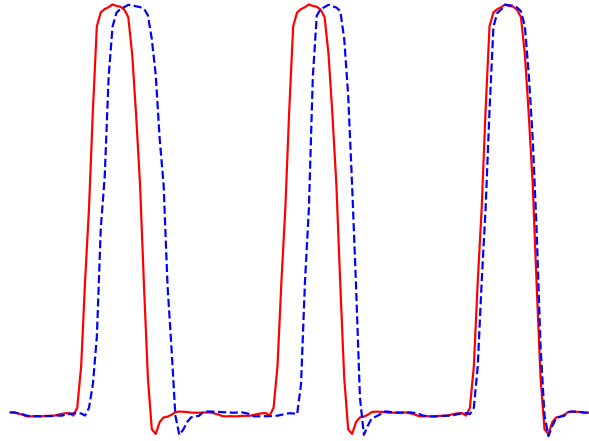


Figure 4.9: The red solid time series shows an instant in the target set. The blue dashed time series shows an instant in the query set.

The dataset contains two classes: “Gun” and Point”. Actors aim at an eye-level target using either a replica gun or their index finger, as illustrated in Figure 4.8. The resulting time series represent the X-axis centroid of the actor’s right hand. Each action lasts for 5 seconds, with the pointing/aiming phase occurring for approximately one second. Recorded at 30 fps, each sample consists of 150 data points. The dataset comprises 50 training and 150 test series, all of length 150.

A key limitation of the original dataset is the assumption that every action lasts exactly 5 seconds. In reality, actors perform actions at varying speeds. If an

actor is asked to perform the action three times continuously in a row, we are likely to observe a time series containing three phases, each with a unique rate. The first phase should take longer than subsequent phases because it is the first time the action is performed. This phenomenon is depicted in Figure 4.9, where the red curve represents an ideal case that consists of three identical phases (i.e., identical rate), while the blue curve represents a realistic scenario with varying rates. The first action is slower than the second action. Consequently, retrieving such patterns requires assigning different scaling factors to each phase to accommodate the phase-specific rates.

We now describe the procedure for generating the target and query sets for the retrieval task. To construct the target set, we concatenate P repetitions of each time series instance from the source dataset. To ensure a fair comparison, we constrain the resulting time series to match the original length n . This is achieved by rescaling each phase to a length of n/P prior to concatenation. Note that we must handle remainders to ensure that the final time series length is exactly n . An example of such a target (where $P = 3$) is depicted by the red solid line in Figure 4.9.

To construct the query set, we first determine the specific lengths for the P segments. Starting with an expected mean length of n/P , we define the minimum segment length as $L_{\min} = (n/P)\sqrt{l}$ and the maximum as $L_{\max} = (n/P)\sqrt{l}$. This formulation ensures that the ratio of the lengths of any two segments is bounded by l . We then generate P random integers within $[L_{\min}, L_{\max}]$ subject to the constraint that their sum is exactly n . Finally, we construct the query by rescaling P copies of the source time series to these generated random lengths and concatenating them. The resulting time series maintains the original length n . An example of such a query (where $P = 3$) is depicted by the blue dashed line in Figure 4.9.

Both the query and target sets are derived from the same source dataset. Consequently, the ground truth target for a given query is defined as the instance in the target set that is generated from the same underlying source time series.

Table 4.1 details the additional datasets used in this study. The column labeled “Train/Test?” specifies which split was employed as the source dataset.

In our experiments, we set the warping window parameter $r = 0.1$, as suggested in the literature. The algorithms were implemented in Python. We utilized the `aeon` [70] library to obtain the baseline distance measures. All experiments were conducted on a workstation equipped with an Intel Xeon Gold 6326 CPU and 256GB of RAM.

Table 4.1: Details of the ten datasets for the experiments

Name	Type	Train/Test?	Size	Class	Length
SonyAIBORobotSurface1	Sensor	Test	601	2	70
ECG200	ECG	Train	100	2	96
MedicalImages	Image	Train	381	10	99
CBF	Simulated	Train	30	3	128
SwedishLeaf	HAR	Train	500	15	128
Plane	Sensor	Train	105	7	144
PowerCons	Device	Train	180	2	144
GunPoint	HAR	Train	50	2	150
Adiac	Image	Train	390	37	176
Epilepsy	HAR	Train	137	4	207

4.5.2 Experimental Results

We employ Top- k Accuracy (often referred to as Precision@ k [71]) as the primary evaluation metric. For a single query Q , this metric indicates whether the correct match is successfully retrieved within the top k candidates:

$$P@k(Q) = \begin{cases} 1 & \text{if the ground truth is in the top-}k \text{ results} \\ 0 & \text{otherwise} \end{cases} \quad (4.17)$$

To determine the top- k results, we compute the distance between Q and every time series in the target set, generating a distance profile of length equal to the dataset size. The top- k results correspond to the k instances with the smallest distances in this profile. Finally, we evaluate the overall performance by computing the mean Top- k Accuracy across the entire query set $\mathcal{D}$:

$$\overline{P@k} = \frac{\sum_{Q \in \mathcal{D}} P@k(Q)}{|\mathcal{D}|} \quad (4.18)$$

We choose $k \in 1, 3$. $k = 1$ refers to the exact retrieval. $k = 3$ give some tolerance for the retrieval.

Results of GunPoint dataset: We utilize the GunPoint dataset to evaluate how the parameters P and l affect the ability of PSD to achieve invariance under piecewise scaling. The results are presented in Table 4.2, where the best performance of $\overline{P@1}$ is highlighted in bold, and the second-best is underlined. We benchmark our method against several state-of-the-art distance measures, including ADTW [9], DDTW [3], shapeDTW [5], WDDTW [4], and WDTW [4]., PSED

Table 4.2: The accuracy comparison for eight distance measures of the GunPoint dataset

P	l	ED		DTW [2]		ADTW [9]		DDTW [3]		shapeDTW [5]		WDDTW [4]		WDTW [4]		PSED		PSDTW	
		$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$
2	1.25	0.30	0.56	0.82	1.00	0.54	0.84	0.84	0.92	<u>0.96</u>	1.00	0.88	0.96	0.82	0.98	0.98	1.00	0.86	1.00
	1.50	0.14	0.36	0.88	1.00	0.38	0.64	0.78	0.94	1.00	1.00	0.80	0.92	0.88	0.96	1.00	1.00	<u>0.96</u>	1.00
	1.75	0.16	0.34	0.82	1.00	0.38	0.66	0.64	0.80	0.90	1.00	0.66	0.80	0.82	0.96	1.00	1.00	<u>0.94</u>	1.00
	2.00	0.12	0.24	0.84	0.98	0.34	0.66	0.72	0.88	<u>0.96</u>	1.00	0.68	0.86	0.82	0.98	1.00	1.00	<u>0.96</u>	1.00
3	1.25	0.30	0.50	0.84	0.92	0.70	0.88	0.80	0.92	0.96	0.98	0.84	0.94	0.86	0.98	<u>0.94</u>	1.00	0.88	0.96
	1.50	0.10	0.28	0.84	0.96	0.60	0.78	0.66	0.86	<u>0.90</u>	1.00	0.72	0.86	0.84	0.98	0.96	1.00	0.86	0.96
	1.75	0.10	0.28	<u>0.88</u>	1.00	0.42	0.78	0.72	0.88	0.76	0.94	0.74	0.88	<u>0.88</u>	1.00	1.00	1.00	<u>0.88</u>	0.98
	2.00	0.02	0.12	<u>0.86</u>	0.94	0.38	0.52	0.60	0.78	0.70	0.94	0.60	0.80	<u>0.82</u>	0.92	0.98	1.00	<u>0.86</u>	0.98
4	1.25	0.34	0.50	0.70	0.86	0.68	0.88	0.60	0.78	0.70	0.90	0.64	0.80	0.70	0.86	0.86	0.94	<u>0.72</u>	0.88
	1.50	0.22	0.38	0.64	0.80	0.60	0.92	0.52	0.64	0.68	0.90	0.52	0.66	0.64	0.82	0.86	0.98	<u>0.72</u>	0.82
	1.75	0.16	0.30	0.72	0.92	0.56	0.80	0.60	0.78	0.56	0.80	0.56	0.80	0.70	0.88	0.92	1.00	<u>0.80</u>	0.94
	2.00	0.06	0.12	0.58	0.78	0.50	0.72	0.46	0.64	0.50	0.82	0.50	0.64	0.56	0.82	0.88	0.98	<u>0.64</u>	0.88

achieves the highest accuracy, followed closely by PSDTW. We attribute PSED’s superior performance over PSDTW to the specific nature of the distortions in the query set, that is, the “pure” piecewise scaling distortions. Since the query set exhibits only piecewise scaling distortions, the corresponding segments of the query and target time series differ solely in length (scale). Consequently, the additional flexibility provided by DTW in PSDTW is unnecessary and may inadvertently increase the similarity of incorrect matches, thereby reducing discriminative power relative to the stricter PSED measure. However, PSDTW remains theoretically essential for handling local distortions within the phase. PSDTW applies DTW within the scaled segment, enabling robust alignment across local nonlinearities.

Table 4.3: The accuracy comparison for six PSED-guided distance measures of the GunPoint dataset

P	l	DTW ^{PSED}		ADTW ^{PSED}		DDTW ^{PSED}		shapeDTW ^{PSED}		WDDTW ^{PSED}		WDTW ^{PSED}	
		$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$
2	1.25	0.86	1.00	0.98	1.00	0.72	0.92	0.98	1.00	0.74	0.96	0.86	1.00
	1.50	0.88	1.00	1.00	1.00	0.60	0.90	0.98	1.00	0.64	0.90	0.92	1.00
	1.75	0.94	1.00	1.00	1.00	0.84	0.94	1.00	1.00	0.86	0.96	0.94	1.00
	2.00	0.98	1.00	1.00	1.00	0.60	0.78	0.98	1.00	0.66	0.80	0.98	1.00
3	1.25	0.82	0.96	0.94	1.00	0.74	0.90	0.92	0.98	0.76	0.90	0.82	0.96
	1.50	0.86	1.00	0.96	1.00	0.72	0.90	0.94	1.00	0.80	0.90	0.88	1.00
	1.75	0.88	1.00	1.00	1.00	0.68	0.90	1.00	1.00	0.70	0.94	0.90	1.00
	2.00	0.86	0.96	0.98	1.00	0.54	0.86	0.96	1.00	0.62	0.86	0.88	0.96
4	1.25	0.74	0.88	0.86	0.94	0.78	0.80	0.80	0.92	0.78	0.82	0.74	0.88
	1.50	0.72	0.84	0.86	0.98	0.68	0.84	0.84	0.96	0.66	0.84	0.72	0.86
	1.75	0.74	0.96	0.92	1.00	0.54	0.90	0.90	1.00	0.56	0.88	0.74	0.96
	2.00	0.66	0.90	0.88	0.98	0.58	0.82	0.88	0.98	0.62	0.80	0.66	0.90

We further investigate whether the segmentation results returned by PSED can serve as a guide to enhance other distance measures. As shown in Table 4.3, this approach generally improves accuracy. The exceptions are highlighted in bold,

indicating cases where the segmentation led to worse performance. Notably, in most cases, only DDTW and WDDTW failed to benefit from PSED-guided segmentation. We argue that the performance degradation of DDTW and WDDTW stems from their derivative-based nature. They rely on matching slope or shape features. Consequently, they are highly sensitive to segmentation boundaries. A non-perfect cut that falls within a shape feature segmentize it, and these features are then destroyed. When the algorithm subsequently attempts to map these features, it fails to find correct correspondences, resulting in high distances.

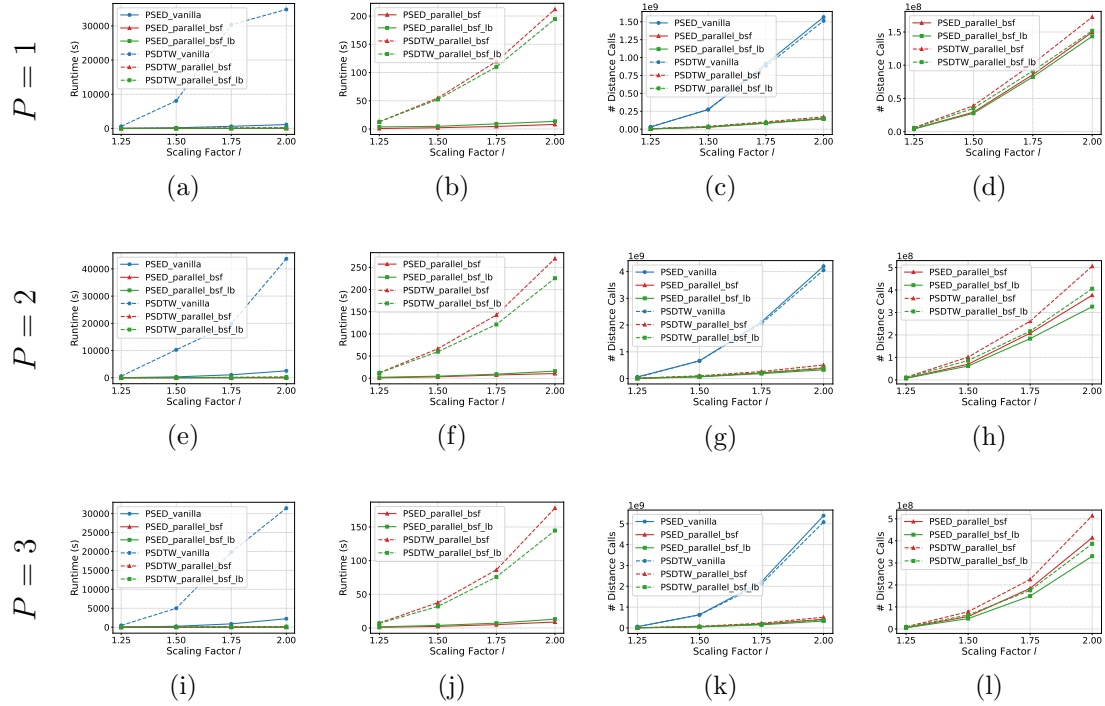


Figure 4.10: Runtime and number of distance calls comparison of GunPoint dataset. (a)-(d) $P = 1$, (e)-(h) $P = 2$, (i)-(l) $P = 3$.

Figure 4.10 illustrates the runtime performance across varying parameters P and l . We evaluate two variants of PSD, PSED and PSDTW, each implemented with three levels of

1. **vanilla** (i.e., Basic implementation)
2. **parallel_bsf** (i.e., With parallelization with Best-So-Far early abandoning)
3. **parallel_bsf_lb** (i.e., Incorporating lower bound pruning)

This yields a total of six methods. The figure is organized into four columns plotted against the scaling factor l . The rows refer to the number of pieces P .

The first and third columns display the running time and the number of distance calculations, respectively, for all six methods. To better visualize the performance differences among the efficient implementations, the second and fourth columns omit the `vanilla` baselines and focus exclusively on the four optimized variants.

The results indicate that `vanilla_PSDTW` is orders of magnitude slower than the other approaches, whereas the optimized methods operate within a similar performance range. As anticipated, the computation time for all methods increases with the scaling factor l . The fourth column confirms that the lower bounding strategy effectively reduces the number of distance calculations (pruning power). However, the second column reveals a critical trade-off in actual runtime. While the lower bound successfully accelerates PSDTW, it actually slows down PSED. This suggests that for the computationally lighter ED, the overhead of calculating the lower bound outweighs the time saved by pruning. Overall, the results demonstrate that PSED is significantly faster than PSDTW.

Results of the ten datasets:

Table 4.4: The accuracy comparison for eight distance measures of the ten datasets

Dataset	ED		DTW		ADTW		DDTW		shapeDTW		WDDTW		WDTW		PSED	
	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$
SonyAIBORobotSurface1	0.04	0.08	0.51	0.63	<u>0.60</u>	0.74	0.45	0.57	0.20	0.29	0.45	0.58	0.51	0.64	0.67	0.77
ECG200	0.12	0.20	0.70	0.81	<u>0.78</u>	0.83	0.60	0.74	0.59	0.76	0.59	0.73	0.70	0.80	0.81	0.88
MedicalImages	0.08	0.19	0.63	0.78	0.69	0.85	0.52	0.68	0.57	0.75	0.51	0.68	0.62	0.78	0.76	0.88
CBF	0.53	0.67	0.83	1.00	<u>0.90</u>	1.00	0.73	0.87	0.73	0.90	0.77	0.90	<u>0.90</u>	1.00	1.00	1.00
SwedishLeaf	0.07	0.12	0.82	0.92	<u>0.84</u>	0.93	0.70	0.83	0.78	0.91	0.69	0.83	0.83	0.92	0.97	0.99
Plane	0.09	0.14	0.50	0.74	<u>0.59</u>	0.78	0.38	0.64	0.53	0.79	0.37	0.61	0.49	0.75	0.75	0.93
PowerCons	0.26	0.43	0.77	0.98	0.80	1.00	0.77	0.98	0.77	0.99	0.77	0.99	0.77	0.99	0.80	1.00
GunPoint	0.10	0.28	<u>0.84</u>	0.96	0.60	0.78	0.66	0.86	0.90	1.00	0.72	0.86	<u>0.84</u>	0.98	0.96	1.00
Adiac	0.01	0.02	<u>0.37</u>	0.50	0.12	0.21	0.32	0.42	0.27	0.41	0.31	0.41	0.36	0.50	0.57	0.71
Epilepsy	0.18	0.26	0.74	0.88	<u>0.80</u>	0.91	0.65	0.82	0.38	0.47	0.58	0.79	0.70	0.83	0.83	0.91

Table 4.5: The accuracy comparison for six PSED-guided distance measures of the ten datasets

Dataset	DTW ^{PSED}		ADTW ^{PSED}		DDTW ^{PSED}		shapeDTW ^{PSED}		WDDTW ^{PSED}		WDTW ^{PSED}	
	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$	$P@1$	$P@3$
SonyAIBORobotSurface1	0.53	0.64	0.64	0.76	0.37	0.53	0.65	0.76	0.38	0.53	0.53	0.65
ECG200	0.73	0.84	0.80	0.88	0.54	0.68	0.81	0.88	0.53	0.69	0.73	0.84
MedicalImages	0.66	0.78	0.76	0.88	0.51	0.69	0.74	0.88	0.51	0.70	0.66	0.79
CBF	0.90	1.00	0.93	1.00	0.73	0.93	0.93	0.97	0.70	0.93	0.90	1.00
SwedishLeaf	0.82	0.92	0.97	0.99	0.72	0.87	0.97	0.99	0.73	0.88	0.82	0.92
Plane	0.60	0.80	0.75	0.94	0.50	0.67	0.74	0.90	0.50	0.69	0.59	0.80
GunPoint	0.86	1.00	0.96	1.00	0.72	0.90	0.94	1.00	0.80	0.90	0.88	1.00
PowerCons	0.78	0.99	0.79	1.00	0.79	1.00	0.79	1.00	0.79	1.00	0.78	1.00
Adiac	0.33	0.45	0.57	0.71	0.21	0.34	0.53	0.70	0.20	0.33	0.34	0.45
Epilepsy	0.77	0.88	0.78	0.88	0.66	0.79	0.74	0.89	0.66	0.82	0.80	0.89

For the remaining nine datasets, we fix the parameters at $P = 3$ and $l = 1.5$. We have the following findings from the previous experiment:

Table 4.6: The number of distance calls and runtime on PSED_vanilla and PSED_parallel_bsf of the ten datasets

Name	Size	Length	PSED_vanilla	PSED_parallel_bsf	% distance calls pruned	Speed Up
			Time (s)	Time (s)		
SonyAIBORobotSurface1	601	70	697	69	90.40%	10.10X
ECG200	100	96	91	3	91.08%	30.33X
MedicalImages	381	99	2082	40	96.03%	52.05X
CBF	30	128	43	2	75.31%	21.50X
SwedishLeaf	500	128	16601	107	96.24%	155.15X
Plane	105	144	621	6	96.05%	103.50X
PowerCons	180	144	3629	49	82.32%	74.06X
GunPoint	50	150	159	3	89.42%	53.00X
Adiac	390	175	23550	123	96.71%	191.46X
Epilepsy	137	206	13131	336	39.88%	39.08X

1. From Table 4.3, PSED outperformed PSDTW in handling piecewise scaling distortions.
2. From Figure 4.10, the lower bound offered no efficiency gain for PSED.

Hence, we select `PSED_parallel_bsf` as the representative method for this evaluation. The accuracy results are presented in Table 4.4, while the results for the PSED-guided distance measures are detailed in Table 4.5. Finally, the runtime efficiency for all datasets is summarized in Table 4.6. It shows a significant speedup, ranging from 10.10X to 191.46X.

4.6 Concluding Remarks

In this study, we proposed a novel distance measure framework, Piecewise Scaling Distance (PSD), which relaxes the strict assumption of a single uniform scaling factor across the entire time series. We presented an exact dynamic programming (DP) algorithm to solve this problem. To enhance efficiency and prevent pathological segment alignments, we introduced a constraint version, which limits the search space of segment lengths based on given scaling factors. To enhance computational efficiency, we integrated two optimization techniques for the general PSD framework. In addition, we propose incorporating a lower-bounding strategy to accelerate PSDTW. Our experimental results demonstrate the necessity and effectiveness of PSD when identifying matches between a query Q and a candidate C under piecewise scaling distortions.

We outline several directions for future research. First, we aim to develop a lower bound specifically optimized for PSED that can improve actual runtime. Second, while the current PSDTW algorithm requires the number of segments P

to be specified as a hyperparameter, it is preferable for the algorithm to determine this value adaptively. A simple heuristic is to test a range of P values and select the configuration that yields the minimum distance. Finally, we plan to investigate efficiency improvements for PSDTW. Currently, PSDTW computes dynamic time warping on two growing subsequences after scaling. While techniques for Incremental DTW [72] allow for the reuse of the accumulating cost matrix D to avoid redundant calculations, applying this to PSDTW is non-trivial. The interpolation performed before DTW fundamentally alters the subsequence structure, preventing the straightforward extension of D (e.g., by appending rows or columns) to reuse the computed D that is possible in standard DTW.

Chapter 5

Leveraging Nearest Neighbors for Time Series Forecasting with Matrix Profile

Time series forecasting is a crucial task used in many applications, including the financial and medical sectors. It is unsurprising that it has been a focus of intensive research for years and that many methods have been proposed. Deep learning methods have proved promising for time series forecasting, especially for long sequence time series forecasting (LSTF). However, they tend to be complex and computationally intensive. A recent study suggests that a well-known baseline in machine learning, namely Gradient Boosting Regression Tree (GBRT), can compete with deep learning methods by appropriately data-engineering the data. A time series forecasting task is transformed into a window-based regression problem. A sliding window is used to transform the time series into predictor-response pairs. At time point t , the previous w target values up to t concatenated with the covariates at t are flattened to form a predictor. The subsequent h target values form a response. A multi-output GBRT model is trained on the predictor-response pairs. Given a new predictor, the corresponding response can be predicted. The model's performance depends on the quality of the covariates. We propose using the past nearest neighbor's information, such as its immediate subsequence and similarity of the nearest neighbor, for each predictor as its covariates to improve the model's performance. The nearest neighbors are found using a data primitive called Matrix Profile. In brief, it allows us to identify the nearest neighbor for every m -subsequence of a time series, where m is user-given. Our approach has also been extended to the case of k -nearest neighbors. Extensive experiments

show that covariates derived from the past nearest neighbors of the predictors can improve the model’s performance.

5.1 Background

Humans have made predictions since ancient times. In ancient societies, accurate predictions were important to the success of subsistence activities such as hunting, planting, and harvesting. There was a need to predict weather dynamics, such as rainfall and temperature. Our ancestors used divination tools such as bones to make predictions. Without doubt, the accuracy was not guaranteed. Prediction is also essential nowadays. Predicting traffic-jam patterns only a few hours ahead can save time by enabling the selection of an alternative route [73]. A wealth could be created by forecasting stock market trends. Predicting the future, also known as time series forecasting, is crucial.

With advances in hardware technologies, we collect enormous amounts of data from diverse sources continuously in the form of time series data. A time series is an ordered sequence of measures, represented in real-valued numbers, at discrete equal-interval timestamps [74]. The vast data collections have created the era of “Big Data”, which provides a wealth of datasets for developing and deploying reliable, robust, data-driven forecasting techniques [75]. Applications can be found in the financial sector, such as predicting business cycles and stock market movements [76, 73, 77, 78] and the medical sector, such as the status of critical patients according to their vital signs [79, 80] and the propagation of diseases such as influenza [78, 81] and COVID-19 [79, 82].

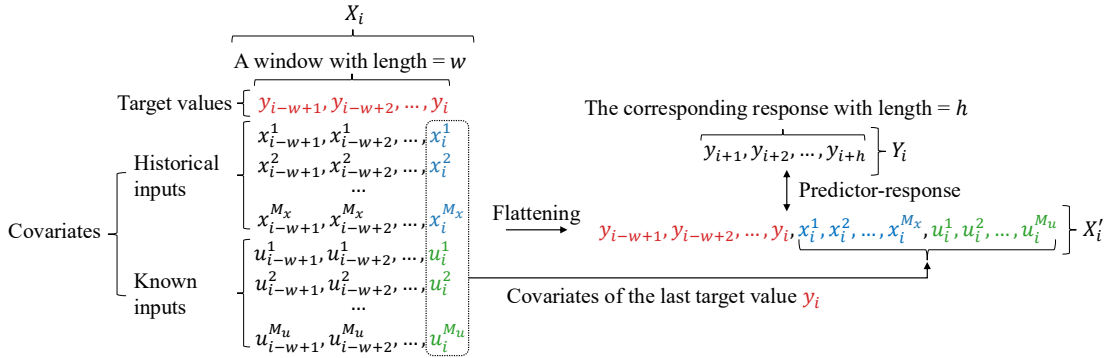


Figure 5.1: Flattening a 2D window X_i of size $(1 + M_x + M_u) \times w$ to a 1D vector X'_i of length $w + M_x + M_u$. The entries in the window are scalars. X'_i serves as the *predictor* for the regressor. Its expected *response* Y_i is a vector of length h . The regressor learns from the predictor-response pairs.

Many methods have been proposed, including traditional statistical methods (e.g., autoregressive integrated moving averages (ARIMA)) and deep learning methods (e.g., recurrent neural networks). A shortcoming of statistical methods is that they cannot handle complex, non-linear dynamics well because of their underlying assumptions of data stationarity and linearity. A shortcoming of deep learning methods is that they are complex and computationally intensive. A recent study [83] demonstrates that a well-known machine learning baseline, Gradient Boosting Regression Tree (GBRT), such as XGBoost, equipped with an appropriate data engineering of the data, can achieve competitive or even superior performance than the deep learning method. In detail, they transform the time series forecasting task into a window-based regression problem, as shown in Figure 5.1. For each training window of length w with the target value y_i of the last time point i , and its lagged values $y_{i-1}, y_{i-2}, \dots, y_{i-w+1}$ are concatenated with the covariates of y_i to form the *predictor* for a multi-output GBRT. This transformation is called flattening.

Note that the standard GBRT returns only a single output. A variant, multi-output GBRT, is used. It is a group of GBRT models where each predicts a single output in a designated position τ in the output, where $\tau \in \{1, 2, \dots, h\}$, while having the same input. The corresponding *response* (i.e., the horizon) is the following h target values of the last value y_i in the predictor. It provides a simple, more efficient yet accurate method for time series forecasting.

This study proposes a method to improve the performance of the GBRT approach by leveraging information from the past nearest neighbor of each subsequence in the target variable y . For each time point y_i , a window Q_i of length w is constructed with y_i as the last point, then we retrieve the window’s left nearest neighbor and use its information to create new covariates for the window, as shown in Figure 5.2. Only the left (past) nearest neighbor is considered to prevent data leakage. For the information of the left nearest neighbor NN^L , we are interested in its immediate subsequence of length l , where l is user-given, the similarity between NN^L and Q_i , NN^L ’s target values of length w , and covariates of the last point of NN^L . We also extend this idea to the case of k -nearest neighbors.

We make the following contributions:

- We are the first to propose leveraging the matrix profile to create covariates for each predictor to improve forecaster performance. It addresses one of the limitations that predictions are made only on observations within the look-back window by incorporating information about the window’s past nearest

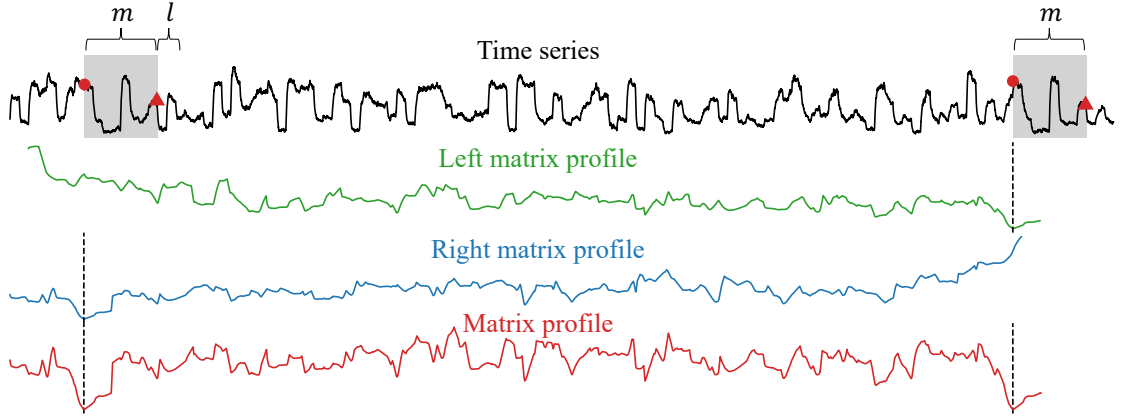


Figure 5.2: The **left** (**right**) matrix profile and the **matrix profile** of a time series. The **matrix profile** shows the distances between each m -subsequence and its nearest neighbor, where m is a user-given value. The **left** (**right**) matrix profile shows the same information but is limited to its left (right) nearest neighbor. For a particular m -subsequence indicated by the right gray box, its nearest neighbor is the left gray box, as indicated by the dashed line in the **left matrix profile** and the **matrix profile**. Similarly, the nearest neighbor of the left gray box is the right gray box, as indicated by the dashed line in the **right matrix profile** and the **matrix profile**. The first (last) point in each box is denoted by a **red** circle (triangle). l denotes the length of the immediate subsequence of the nearest neighbor. Intuitively, this subsequence should be similar to the immediate subsequence of the right gray box. To note, the **left** (**right**) matrix profile starts (ends) at a later (earlier) index.

neighbor. The nearest neighbor provides a "look-ahead" of the future trajectory by providing a similar historical trajectory.

- We have proposed two simple methods, namely, **GBRT-NN**, which uses the immediate subsequence of the nearest neighbor with length l as the covariates, and **GBRT-NN-S**, which also incorporates the similarity as a covariate. Furthermore, we study the effect of incorporating additional information from the nearest neighbor, such as its target values and covariates of the last point, to further extend the two simple methods.
- We also extend the idea to the case of k -nearest neighbors.
- Extensive experimental results show that our approach can improve the GBRT forecaster in almost all cases, tested on the four datasets, only with a small cost to compute the matrix profile, which is $\mathcal{O}(n^2)$, where n is the length of the time series.

The rest of this study is organized as follows. Section 5.2 presents the related work. In Section 5.3, we cover the necessary background knowledge. In Section 5.4,

we introduce our method. Section 5.5 contains an empirical evaluation. Finally, we conclude this study and provide future work in Section 5.6.

5.2 Related Work

Many methods have been developed for time series forecasting. We introduce them by categories.

5.2.1 Traditional methods

Traditional methods include rolling averages (RA), vector auto-regression (VAR) [73, 84], and auto-regressive integrated moving averages (ARIMA) [83, 85, 84]. Because of their rigorous statistical properties, they have long been the standard. The shortcomings of ARIMA and its variants include their high computational cost [73]. In contrast, VAR is arguably the most widely used method in this category, particularly in multivariate time series analysis, owing to its simplicity. However, most of these approaches have certain limitations. They perform well when the data meet specific statistical assumptions, such as stationarity [86], which means that the mean and variance of the time series remain constant over time. It motivates the community to develop machine learning methods, particularly deep learning methods for time series forecasting.

5.2.2 Deep learning methods

Many deep learning models have been proposed, including RNN-based models, CNN-based models, GNN-based models, Transformer-based models, and compound models that incorporate different base models mentioned before [78]. The compound models are promising. For example, RNNs are well suited to capturing long-term dependencies, whereas CNNs are well suited to capturing short-term dependencies. A good way to improve performance is to compound them. For example, LSTnet [73] integrates CNN, RNN, and autoregressive [87] techniques to extract both short-term and long-term patterns. Using the occupancy rate of a freeway as an example [73] to explain these two patterns, the “short-term” patterns refer to the morning peaks against evening peaks, while the “long-term” patterns refer to the workday patterns against weekend patterns. Clearly, a good forecaster needs to capture and distinguish both kinds of patterns. Despite the superior performance deep learning methods have achieved, they tend to be overly

complex, opaque, and incur high computational costs. The model proposed in this study can also be regarded as a compound model.

5.2.3 Nearest neighbor-based methods

k -NN has also shown to be a promising method for time series forecasting [88, 89, 90]. The k -NN uses the $w - 1$ lagged values of the last time point i to form a query Q with length w . It identifies the k previous similar subsequences to Q and uses their immediate subsequences to predict the immediate subsequence of Q with length h . The immediate subsequence of Q is also called the forecasting window. The intuition of this method is that history repeats itself. They are similar, and so are their immediate subsequences. The previous (historical) subsequences that are similar to Q can provide a hint about the future of Q . Figure 5.2 depicts this idea. Observe that the immediate subsequence of the right gray box is similar to that of the left gray box, which is the past (left) nearest neighbor of the right gray box.

The fundamental difference between this study and previous approaches [88, 89, 90] is that they directly use the subsequent points for prediction, whereas we use the nearest neighbor’s information for each predictor as covariates, which are used as inputs for other forecasters. We explain this subtle difference by Figure 5.2. The previous approaches simply use the information of the nearest neighbors of the last lookback window, which consisted of the last time point and its lagged values (i.e., the right gray box), for prediction. In contrast, we use the information of the nearest neighbors of *all* of the windows in the time series.

5.2.4 Matrix Profile-based methods

To the best of our knowledge, there are only four studies using the matrix profile in time series forecasting. [90] finds the most similar historical subsequence and uses its succeeding values as the prediction, similar to the nearest neighbor-based methods mentioned above. [91] used the matrix profile to visualize the financial time series data by identifying motifs. [92] used matrix profile to detect structural anomalies and incorporated the anomaly information to guide an attention-based long short-term memory (LSTM) model in forecasting COVID-19 indicators. [93] proposed an encoding-decoding methodology to learn repeating granular trends in periodic data. It first identify a special kind of motif called “time series chain”. The raw time series is then transformed into a level-encoded sequence based on the time series chain. The model predicts future labels instead of raw values. The

predicted labels are decoded back into the original domain (i.e., raw values) using a weighted percentage change based on the historical centrality of the chain.

None of these studies uses the matrix profile to find nearest neighbors and uses their information as covariates, which can be fed into any forecaster, such as GBRT [83] and DeepGlo [94], that supports covariates.

5.3 Preliminaries

To begin, we define the data type of interest: *time series*.

Definition 10 (Time series). A *time series* $T = t_1, t_2, \dots, t_n$ is a sequence of real-valued numbers with length $= n$.

Since each entry t_i is one-dimensional, T is called a *univariate* time series. Definition 10 can be extended to define *multivariate* time series by using a vector with a number of dimensions greater than one for t_i . A multivariate time series T can also be viewed as a vector of univariate time series, where each univariate time series is called a *channel* of T .

The local properties of T can be analyzed through its *subsequences*.

Definition 11 (Subsequence). A *subsequence* $T_{i,m}$ of a T is a sequence $t_{i:i+m-1} = t_i, t_{i+1}, \dots, t_{i+m-1}$ that consists of a continuous subset of the entries from T of length m starting from position i .

The distance measure used is the z-normalized Euclidean distance, which computes the Euclidean distance between the two time series after they have been z-normalized to have a mean of zero and a standard deviation of one. For a time series T , the resulting z-normalized time series $\hat{T}$ would have entries $\hat{t}_i = (t_i - \mu(T))/\sigma(T)$, where $\mu(T)$ and $\sigma(T)$ are the mean and the standard deviation, of T respectively. For two time series X and Y , the z-normalized Euclidean distance $\text{dist}(X, Y)$ is defined as follows:

$$\text{dist}(X, Y) = \sqrt{\sum_{i=1}^m (\hat{x}_i - \hat{y}_i)^2} \quad (5.1)$$

$\text{dist}(X, Y)$ is bounded between zero and $2m$ [95].

5.3.1 Matrix Profile

We will first introduce the definitions of Distance Profile and Matrix Profile. Then, we will introduce the algorithms to compute them fast. Finally, we will focus on a specific variation of Matrix Profile, which is Left Matrix Profile. It is a main primitive to identify the left nearest neighbor for each predictors.

In brief, a matrix profile P is a time series that annotates an input time series T , storing the z-normalized Euclidean distance between each m -subsequence and its nearest neighbor in T .

Given a length m , we can select an m -subsequence from T as a query and compute its distances to all m -subsequences in T . The resulting ordered distance vector is a *distance profile*.

Definition 12 (Distance profile). A *distance profile* $D_i = d_{i,1}, d_{i,2}, \dots, d_{i,n-m+1}$ of a T is a vector of the distances between a given subsequence $T_{i,m}$ and each subsequences $T_{j,m}$ in T , where $d_{i,j}$ is the distance between $T_{i,m}$ and $T_{j,m}$, $1 \leq i, j \leq n - m + 1$.

Definition 13 (Matrix profile). A *matrix profile* $P = \min(D_1), \min(D_2), \dots, \min(D_{n-m+1})$ of T is a vector of Euclidean distances between every subsequence $T_{i,m}$ of T and its nearest neighbor in T .

Another closed related time series of P , namely the Matrix Profile Index I , stores the location of the corresponding nearest neighbor.

Definition 14 (Matrix profile index). A *matrix profile index* $I = I_1, I_2, \dots, I_{n-m+1}$ of T is a vector of integers, where $I_i = j$ if $d_{i,j} = \min D_i$.

To retrieve a non-trivial result of the Matrix profile, we need to define the notion of an exclusion zone. For a given m -subsequence Q , the distance profile is zero at the location of Q (because Q must be a nearest neighbor of Q) and close to zero just before and after it. But the corresponding m -subsequences are trivial matches of Q . We need to define a zone, namely an exclusion zone, for this region. It is defined as $m/2$ before and after the location of Q [96]. The community implementation uses $m/4$ instead [97]. We use $m/4$ as the exclusion zone in this study.

It may seem computationally expensive to perform Matrix Profile annotation at first glance. Algorithm 6 shows the brute force approach to compute the matrix profile. The two for-loops and the computation of z-normalized Euclidean distance, which takes $\mathcal{O}(m)$, indicate that the computational complexity is $\mathcal{O}(n^2m)$. The

Algorithm 6 The brute force approach to compute Matrix Profile

```
1: for  $i \leftarrow 1$  to  $n - m + 1$  do  
2:   for  $j \leftarrow 1$  to  $n - m + 1$  do  
3:     Compute the z-normalized Euclidean distance between  $t_{i,m}$  and  $t_{j,m}$ .
```

space complexity is $\mathcal{O}(n^2)$ because of the pairwise distance of each subsequence with the other subsequence. However, the matrix profile can be computed in $\mathcal{O}(n^2)$ using an exact method, namely STOMP [96] or its community-open-sourced version, STUMP [97]. The main idea is using MASS [98] to compute the for-loop in line 2 in Algorithm 6 and reuse the computed results of D_{i-1} to compute D_i across the for-loop in line 1 because $t_{i-1,m}$ and $t_{i,m}$ differ only in the first point of $t_{i-1,m}$ and the last point of $t_{i,m}$ [99].

MASS

The computation of D_i may seem like costing $\mathcal{O}(nm)$, as depicted in lines 2-3 in Algorithm 6, where line 3 takes $\mathcal{O}(m)$. In fact, line 3 takes $2m$ arithmetic operations because the m -window is scanned twice, once for z-normalization and once for distance calculation. However, it can be computed in just $\mathcal{O}(n \log(n))$ by an algorithm called MASS [98]. We explain the main ideas of MASS here and direct readers to [98] for a more detailed description. We will start by explaining the most basic component of MASS: just-in-time normalization. Equation 5.1 can be rewritten as follows.

$$\text{dist}(X, Y) = \sqrt{2m(1 - \frac{\sum_{i=1}^m x_i y_i - m\mu(x)\mu(y)}{m\sigma(X)\sigma(Y)}} \quad (5.2)$$

In line 3, the same query is comparing to other subsequences throughout the loop. Hence, we can perform z-normalization on the query once before the for-loop in line 2. Let's assume Y is the z-normalized query, hence $\mu(Y) = 0$ and $\sigma(Y) = 1$, and X is the non-z-normalized window. Equation 5.2 can then be reduced as follows.

$$\text{dist}(X, Y) = \sqrt{2m(1 - \frac{\sum_{i=1}^m x_i y_i}{m\sigma(X)}} \quad (5.3)$$

The z-normalization scan of X can be skipped in each iteration by following. The main idea is that $\sigma(X)$ of moving windows with size m can be calculated in one linear scan before entering the for-loop in line 2. It has been shown that the two cumulative sum vectors are sufficient to calculate μ and σ of any subsequence of any length at any iteration [96]. First, the cumulative sums of $C = \sum_{i=1}^n x$

and $C^2 = \sum_{i=1}^n x^2$ are calculated. The sums over any window $S_i = C_{i+m} - C_i$ and $S_i^2 = C_{i+m}^2 - C_i^2$ can be obtained by subtracting the two cumulative sums. Finally, σ of any window can be calculated in linear time using the following equation.

$$\sigma_i = \sqrt{\frac{S_i^2}{m} - \left(\frac{S_i}{m}\right)^2} \quad (5.4)$$

The overall complexity of lines 2-3 in Algorithm 6 is still $\mathcal{O}(nm)$. However, since the z-normalization scan of X has been skipped, each iteration now takes m arithmetic operations only for calculating the dot product in the numerator of Equation 5.3.

The complexity of lines 2-3 can be further improved to $\mathcal{O}(n \log n)$ by using the fast fourier transform (FFT) to compute the dot products between the query and *all* subsequences $t_{j,m}$ [98]. The dot products are called sliding dot products because the windows X (i.e., $t_{j,m}$) are extracted through a sliding window across T . It is faster than $\mathcal{O}(nm)$ since $m > \log n$ holds in most cases. One of the strengths is that the complexity $\mathcal{O}(n \log n)$ does not depend on the length m of the subsequence. The above optimization is called MASS V1 and there are three other levels of optimization namely MASS V2, MASS V3, and MASS V4 to further reduce the running time [98].

Running time of Matrix Profile

The direct usage of MASS will make Algorithm 6 become $\mathcal{O}(n^2 \log n)$ [96]. Observe that the windows $t_{i,m}$ share most of the parts with $t_{i-1,m}$ except the first point of $t_{i,m}$ and the last point of $t_{i-1,m}$. Once we have the distance profile D_{i-1} for the subsequence $t_{i-1,m}$, the distance profile D_i for the next subsequence $t_{i,m}$ can be computed in just $\mathcal{O}(1)$. Hence, it results $\mathcal{O}(n^2)$ for computing the matrix profile [99].

Besides, the running time can be further sped up by parallelization for a single machine with multiple computation units, such as CPUs or GPUs [99].

5.3.2 Left Matirx Profile

In this study, instead of the general matrix profile (index), we are interested in the left version of it. The left version of the matrix profile informs us of the information about the left nearest neighbor of any m -subsequences in T [100]. It is shown in Figure 5.2. The left matrix profile has high values at the beginning because it can only find the left neighbor, and there are few left subsequences initially. In

contrast, the right matrix profile has low values at the beginning because there are many right subsequences there. To note, each entry i in Matrix Profile P_i is simply the minimum of P_i^L and P_i^R (i.e., $P_i = \min(P_i^L, P_i^R)$).

Definition 15 (Left distance profile). A *left distance profile* $D_i^L = d_{i,1}, d_{i,2}, \dots, d_{i,i-\lceil m/4 \rceil - 1}$ of T is a vector of Euclidean distances between a given subsequence $T_{i,m}$ and each subsequence that appears before $T_{i,m}$. To note, $i - \lceil m/4 \rceil - 1$ is the index location of the last eligible subsequence before $T_{i,m}$ because of the exclusion zone.

Definition 16 (Left matrix profile). A *left matrix profile* $P^L = \min(D_1^L), \min(D_2^L), \dots, \min(D_{n-m+1}^L)$ of T is a vector of Euclidean distances between every subsequence $T_{i,m}$ of T and its nearest neighbor in T before it.

Definition 17 (Left matrix profile index). A *left matrix profile index* I^L is a vector of integers $I_1^L, I_2^L, \dots, I_{n-m+1}^L$ of T , where $I_i^L = j$ if $d_{i,j} = \min D_i^L$.

The computational time of the left matrix is the same as the general matrix profile, since it is just separating the computation into two halves, one for the left and one for the right, which is not considered in this study.

5.3.3 Gradient boosting

Gradient boosting [101] is a boosting algorithm that ensembles a group of weak learners (usually decision trees) to make predictions. It sequentially adds learners to an ensemble, with each learner connecting its predecessor. It constructs weak learners in a way that each learner strategically corrects the predecessor’s mistakes by fitting the new learner to the residual errors made by the predecessor [102, 103]. It can be used in classification and regression. In this study, we focus on its use in regression. For the usage in regression, the model is called “Gradient Boosting Regression Tree (GBRT)”. Some popular optimized implementations of gradient boosting are XGBoost [104], CatBoost [105], and LightGBM [106]. In this study, we use XGBoost.

5.4 Materials and Methods

In this section, we first introduce the four datasets used in this study. We then formulate the time series forecasting problem and present the evaluation method, including the evaluation metrics. We then explain how to use a Gradient Boosting

Table 5.1: Dataset Statistics. N : the number of time series in the dataset. n : the length of each time series T in the dataset. rate: measuring rate. start date: (date, time). h : the size of the forecasting window. T_{train} (T_{test}): the training (test) subsequence of T . To note, $|T_{\text{train}}| + |T_{\text{test}}| = n$.

Dataset	Data				Forecasting Task		
	N	n	rate	start date	h	$ T_{\text{train}} $	$ T_{\text{test}} $
Electricity [94]	70	26,136	hourly	2012-01-01, 00:00	24	25,968	168
Traffic [94]	90	10,560	hourly	2012-01-01, 00:00	24	10,392	168
PeMSD7 [94]	228	12,672	/5 mins	2012-05-01, 00:00	9	11,232	1,440
Exchange-Rate [73]	8	7,536	daily	1990-01-01, 00:00	24	6,048	1,488

Regression Tree (GBRT) for forecasting by casting the time series into predictor-response pairs, as shown in Figure 5.1. Finally, we discuss how to leverage the nearest neighbor of a m -subsequence where the last data point is y_i to derive covariates for y_i . The derived covariates can improve the performance of the GBRT forecaster.

5.4.1 Datasets

Four datasets are used, as listed in Table 5.1. The time series are univariate and consist only of target values y_i , where $1 \leq i \leq n$. Each time series T is split into two contiguous time series, namely T_{train} and T_{test} . The N time series are considered as multiple, independent univariate time series instead of a multivariate time series with N channels.

Originally, each time point is not associated with any covariates (i.e., $M_x = M_u = 0$). Given a start date for the measurement of a time series T and its measuring rate, we can assign the (date, time) information for each y_i in T . For example, y_2 of a time series in **Traffic** is measured at 2012-01-01, 01:00. We can derive some covariates for y_i based on its (date, time) information. For example, six covariates can be derived for y_2 as follows. They are month, day, hour, day_of_week, day_of_year, and week_of_year. The corresponding values are 1, 1 (Day 1 of the month), 1 (where the first hour is 0), 6 (Sunday, where Monday is 0), 1, and 52 (Week 1 of 2012 begins on 2012-01-01 (Monday), so it is in Week 52 of 2011). These derived covariates are normalized to be from -0.5 to 0.5 [83]. They are called known inputs $\mathbf{u}_i$ because they are known even in the future (i.e., T_{test}). Table 5.2 shows the derived known inputs for the datasets.

Table 5.2: Derived known input time-covariates with size M_u for each dataset.

Dataset	Derived known input time-covariates	M_u
Electricity	month, day, hour, day_of_week, day_of_year, week_of_year	6
Traffic	month, day, hour, day_of_week, day_of_year, week_of_year	6
PeMSD7	month, day, hour, minute, day_of_week, day_of_year, week_of_year	7
Exchange-Rate	month, day, day_of_week, day_of_year, week_of_year	5

5.4.2 Problem Formulation

Time series forecasting models predict h future values $y_{t+1}, y_{t+2}, \dots, y_{t+h}$ of a target y at a given time point t . h is the number of steps we want to predict in the future w.r.t. t . The simplest case is one-step-ahead forecasting, where $h = 1$. It is preferable to predict multiple future points. In this case, it is called multi-horizon (i.e., multi-step) forecasting, where $h > 1$. For the actual value y_i , its predicted value is denoted as $\hat{y}_i$. A forecasting task is represented in Equation 5.5 [86].

$$\hat{y}_{t+\tau} = f(y_{t-w+1:t}, \mathbf{x}_{t-w+1:t}, \mathbf{u}_{t-w+1:t+\tau}, \tau) \quad (5.5)$$

where

- $\hat{y}_{t+\tau}$ is a prediction of the target value at $t + \tau$, where $\tau \in \{1, 2, \dots, h\}$.
- $f(\cdot)$ is the prediction function learnt by the model.
- w is the length of a lookback window. The observations in the window are considered during forecasting.
- $y_{t-w+1:t}$ are the actual target values consisting of the current value y_t and the $w - 1$ lag values before y_t . y_{t-i} is called the lag of i or i -lag.
- $\mathbf{x}_{t-w+1:t}$ are historical inputs. $\mathbf{x}_t$ is not known until time t . $\mathbf{x}_i$ is a vector of length M_x , where $t - w + 1 \leq i \leq t$.
- $\mathbf{u}_{t-w+1:t+\tau}$ are known inputs. It is known for any $t \geq 1$. For example, date information, such as month and day. $\mathbf{u}_i$ is a vector of length M_u , where $t - w + 1 \leq i \leq t + \tau$.

$\mathbf{x}_i$ and $\mathbf{u}_i$ are called covariates of y_i .

We explain the task of forecasting using Equation 5.5. The model estimates the value of $y_{t+\tau}$, denoted by $\hat{y}_{t+\tau}$, with the aim to minimize the error function, typically represented as a function of $y_{t+\tau} - \hat{y}_{t+\tau}$ for each τ .

It is clear that the derived time-covariates from the date information are useful for forecasting when the target variable depends on the time of measurement. For example, if the target is the electricity consumption rate, there is a clear monthly pattern: consumption is higher in winter and summer, when heaters and air conditioners are used.

5.4.3 Evaluation Method

Given a dataset D of N time series T_i , where $1 \leq i \leq N$, the error made by the forecaster on D is simply the summation of errors made by the forecaster on each T_i . For each time series T , the trained model is tested on T_{test} using rolling forecasting. The prediction is always based on the ground truth, not on the model's prediction results. The model is tested for $|T_{\text{test}}|/h$ times for each T . In detail, the model first predicts the target values $\hat{y}_{t+1:t+h}$ in one batch, where $t = |T_{\text{train}}|$. The error is computed between the predicted target values $\hat{y}_{t+1:t+h}$ and the actual values $y_{t+1:t+h}$. For the second prediction, the input is the ground truth (i.e., $y_{t+1:t+h}$), rather than the predicted values (i.e., $\hat{y}_{t+1:t+h}$), and the output is $\hat{y}_{t+h+1:t+2h}$. For example, in **Traffic**, $|T_{\text{test}}| = 168$ and $h = 24$, the model is tested for $168/24 = 7$ times for each time series.

Rolling forecasting is used to prevent the accumulation of errors. This concept is also called “Teacher forcing” in RNN [107] because the ground truth (given by the “teacher”) is fed into the forecaster when we roll the forecaster on T_{test} [108]. The intuition is that, suppose each question (except the first one) in an exam depends on the answers to the previous questions, rather than simply grading every answer in the end, a teacher would grade (evaluate) the answer once it is given by the student (i.e., the forecaster), and provide the correct answer (i.e., ground truth) to the student so he can answer the next question based on the correct answer.

To evaluate the forecaster, we used the following three evaluation metrics defined as:

- Root Mean Square Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (5.6)$$

- Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (5.7)$$

- Weighted Absolute Percentage Error (WAPE)

$$\text{WAPE} = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n |y_i|} \quad (5.8)$$

where n is the length of the time series, y_i and $\hat{y}_i$ are the true value and predicted value, respectively.

By rewriting Equation 5.8 to Equation 5.9, it is more obvious that it is a weighted absolute percentage error.

$$\text{WAPE} = \sum_{i=1}^n w_i \frac{|y_i - \hat{y}_i|}{|y_i|} \quad (5.9)$$

where the weights are given by

$$w_i = \frac{|y_i|}{\sum_{i=1}^n |y_i|} \quad (5.10)$$

RMSE and MAE are widely used metrics. WAPE was introduced by [109]. RMSE is sensitive to large prediction errors because it squares the error terms $(y_i - \hat{y}_i)$ before averaging them. They contribute significantly to the final RMSE. Hence, it is more sensitive to outliers. In contrast, MAE treats all errors linearly because of the absolute sign operator. Hence, it is less sensitive to outliers. MAE can better reflect the actual error situation than RMSE [76]. MAE and WAPE are correlated. WAPE can be obtained by multiply MAE by n , followed by a division of the sum of the actual values $\sum_{i=1}^n |y_i|$. MAE is dependent on the scale of the data (i.e., different units used for the targets). WAPE provides a normalized percentage, as shown in Equation 5.9. It makes WAPE independent of the scale of the data. For all of them, a lower value is better.

5.4.4 Forecasting via GBRT

In order to apply GBRT into time series forecasting problem, we need to cast the input into an appropriate format to input into GBRT. The casting approach is

similar to the successful time series forecasting models, which reconfigure the time series into the predictor-response pairs [83].

Figure 5.1 presents the casting. The window X_i with size $w \times (1 + M_x + M_u)$ is flattened into a 1D vector X'_i with length $w + M_x + M_u$. To note, only the covariates of the last target value y_i are kept [83]. By casting, we obtain the predictor-response pairs. In detail, the predictor X'_i is $y_{i-w+1}, y_{i-w+2}, \dots, y_i, x_i^1, x_i^2, \dots, x_i^{M_x}, u_i^1, u_i^2, \dots, u_i^{M_u}$. With this predictor-response formulation, the forecasting problem becomes a multi-output regression problem. The corresponding response is $y_{i+1}, y_{i+2}, \dots, y_{i+h}$, with length h . As suggested in the original papers of the datasets, w is chosen to equal h .

Conventional XGBoost returns a single number instead of a sequence of numbers. We can treat a multi-output regression problem as a group of single-output regression problems. The forecasting task of predicting h steps ahead is treated as h individual regression problems. So, the final output is produced by h individual regressors. One may argue that the h regressors operate individually and hence the temporal relationship in the output sequence is lost. However, as the individual regressors are trained on the same flattened input X'_i , the prediction Y_i would still preserve the temporal relationship [83].

We explain the above process through **Traffic** dataset, where $n = 10560$ (in which $|T_{\text{train}}| = 10392$), $h = 24$, $w = h = 24$, $M_x = 0$, and $M_u = 6$. There are $10392 - 48 + 1 = 10345$ predictor-response pairs, which are extracted by a sliding window of size 48 on T_{train} . The size of the sliding window is the sum of the length w of the window X_i and the length h of the output Y_i . $|X'_i| = w + M_x + M_u$. If the known inputs u are incorporated into the forecasting, $|X'_i| = 30$. Otherwise, $|X'_i| = 24$.

5.4.5 Feature Engineering via Matrix Profile

Recall that for each m -subsequence $T_{i,m}$ in a time series T , where m is user-given, we can identify its left nearest neighbor NN_i^L via the left matrix profile index P^L and retrieve the similarity between $T_{i,m}$ and NN_i^L via the left matrix profile I^L .

For each target value y_i , we construct a w -subsequence Q_i consisting of $y_{i-w+1:i}$. By using I^L with $m = w$, we can efficiently identify NN_i^L for each of these w -subsequences. NN_i^L can reveal the historical reference of Q_i . History repeats itself. For example, since Q_i is similar to NN_i^L , the immediate subsequence of Q_i should be similar to that of NN_i^L , which tells the future trajectory after y_i . The immediate subsequence can be regarded as a guess or template for the future

Algorithm 7 Feature Engineering via Matrix Profile

Input: Time series y , Time-covariate sequence u , Target value index i , Subsequence length m , Immediate subsequence length l

Output: Extracted covariates for y_i

- 1: Compute P^L and I^L for y with subsequence length m
 - 2: $k \leftarrow i - m + 1$ $\triangleright$ Map index i to standard query start index
 - 3: $idx_{NN} \leftarrow I^L[k]$ $\triangleright$ Start index of NN_i^L
 - 4: $j \leftarrow idx_{NN} + m - 1$ $\triangleright$ Index of the last point of NN_i^L
 - 5: $s \leftarrow P^L[k]$ $\triangleright$ Similarity
 - 6: $NN_{imm} \leftarrow y[j + 1 : j + l]$ $\triangleright$ Immediate subsequence following NN_i^L
 - 7: $NN_{last} \leftarrow y[j]$ $\triangleright$ Last target value of NN_i^L
 - 8: $NN_{tar} \leftarrow y[idx_{NN} : j]$ $\triangleright$ All the target values of NN_i^L
 - 9: $NN_{cov} \leftarrow u[j]$ $\triangleright$ Covariates at the last point of NN_i^L
 - 10: **return** $s, NN_{imm}, NN_{last}, NN_{tar}, NN_{cov}$
-

trajectory. The similarity obtained via P^L can be regarded as the confidence of guessing such a future trajectory based on the immediate subsequence of Q_i . A time series with different raw values can have the same z-normalized representation. Hence, a set of historical subsequences with the exact same z-normalized representation could correspond to different raw values. By extracting the raw values of the nearest neighbor, we restore this lost magnitude information and allow the GBRT to learn from it.

Once the location of NN_i^L is identified via the I^L , we extract the following information about NN_i^L as in Algorithm 7. They are used as u_i for y_i .

1. s : It is normalized to be from -0.5 to 0.5 as in time-derived covariates.
2. NN_{imm} : The subsequence of length l immediately following NN_i^L , where l is user-given.
3. NN_{last} : The last target values of NN_i^L .
4. NN_{tar} : All the m target values that make up NN_i^L itself.
5. NN_{cov} : The known input covariates of the last point of NN_i^L .

Note that the indexing systems differ between the matrix profile and the look-back window. In the matrix profile, an m -subsequence is indexed by its starting position, as shown in Figure 5.2. The gray boxes are indexed by their first points (the circles in the figures). The lookback windows are indexed by their last points (the triangles in the figures). Hence, for Q_i , which is a m -subsequences with y_i as its last point, we map i to the conventional starting index $k = i - m + 1$ to

Table 5.3: Summary of the proposed models and their incorporated covariates. NN_{imm} : the immediate subsequence of NN. NN_{last} : the last target value of NN. NN_{tar} : all the target values of NN. NN_{cov} : time-covariates of the last point of the NN. Details of the incorporated covariates are in Algorithm 7.

Method	Base Features	NN_{imm}	NN_{last}	NN_{tar}	NN_{cov}
GBRT	✓				
GBRT-NN	✓	✓			
GBRT-NN ⁺	✓	✓	✓		
GBRT-NN ⁺⁺	✓	✓		✓	
GBRT-NNC	✓	✓			✓
GBRT-NNC ⁺	✓	✓	✓		✓
GBRT-NNC ⁺⁺	✓	✓		✓	✓

Note: Appending -S to any method name additionally incorporates the similarity s into the models.

retrieve the correct nearest neighbor location and similarity score as in line 2 in Algorithm 7.

GBRT utilizes the baseline features, including the target values and the known input time-covariates [83]. To note, we can also only consider the target values but not the time-covariates in the forecasting. By integrating the extracted information, we propose several variants of GBRT as summarized in Table 5.3

It is important to note how these new features are integrated into the GBRT model. The covariates as shown in Figure 5.1 are scalars. But the extracted features NN_{imm} , NN_{tar} , and NN_{cov} are vectors of length l , m , and M_u , respectively. To satisfy the 1D input requirement of the GBRT model, these vectors are flattened and appended to the predictor vector during the data casting step. In detail, each data point within the vector is treated as an independent scalar feature and concatenated to the predictor vector X'_i directly, instead of being encapsulated in the corresponding vector $\mathbf{x}_i$. For example, for GBRT-NN is $w + M_x + l$, given $NN_{\text{imm}} = t_{j+1}, t_{j+2}, \dots, t_{j+l}$, it results the predictor $y_{i-w+1}, y_{i-w+2}, \dots, y_i, u_i^1, u_i^2, \dots, u_i^{M_u}, t_{j+1}, t_{j+2}, \dots, t_{j+l}$ with length $w + M_x + l$. Each individual points in the immediate subsequence are appended directly to the predictor vector.

5.4.6 Extends to k-Nearest Neighbors

We further extend our methodology to incorporate features from the k -nearest neighbors (k -NN) rather than just the nearest neighbor (1-NN) of each predictor.

During the extraction process over the Left Distance Profile (D_i^L), instead of retaining only the lowest distance value, We iteratively identify the k lowest distance values as follows. We would like to retrieve the k -nearest neighbors without overlapping as defined by the exclusion time. First, we locate the index of the minimum value in D_i^L to identify the first nearest neighbor. Before the next iteration, we mask the exclusion zone surrounding this selected index. We repeat this process to identify the remaining $k - 1$ minima. Once the locations of the k non-overlapping nearest neighbors are determined, we extract their respective information ($s, NN_{\text{imm}}, NN_{\text{last}}, NN_{\text{tar}}, NN_{\text{cov}}$) and flatten them into scalar features similar to what we did for the nearest neighbor case.

To denote the variants utilizing k -nearest neighbors, we update the name by prefixing the “NN” with the integer k in the method name. For example, when $k = 2$, the GBRT-NN is denoted as GBRT-2NN.

5.5 Experiments

This section presents experiments demonstrating the effectiveness of using nearest neighbor information to improve GBRT performance on the time series forecasting task.

The experiments are conducted on a personal laptop equipped with an Apple M1 Pro chip and 16 GB of memory, using Python for implementation. We employ `stumpy` for the matrix profile implementation and `xgboost` for the XGBoost implementation. The code and data are available at <https://github.com/colmanyu/matrix-profile-motif-forecasting>.

The evaluation is conducted on four datasets, as listed in Table 5.1. Three evaluation metrics are used, namely RMSE, MAE, and WAPE. There are two scenarios for the datasets’ use case.

- Without known input time-covariates $\mathbf{u}_i$ (i.e. $M_u = 0$): The models rely solely on the target values.
- With known input time-covariates $\mathbf{u}_i$ (i.e. $M_u > 0$): The models incorporate additional time-derived known input covariates u_i , as listed in Table 5.2

GBRT is used as our baseline for both scenarios. The results are in Tables 5.4, 5.5. In the table of experimental results, **bold** represents the best result, and underlined represents the second best. We first evaluate two primary variants, GBRT-NN and GBRT-NN-S, for initial testing.

Table 5.4: Experimental Results without known input time-covariates.

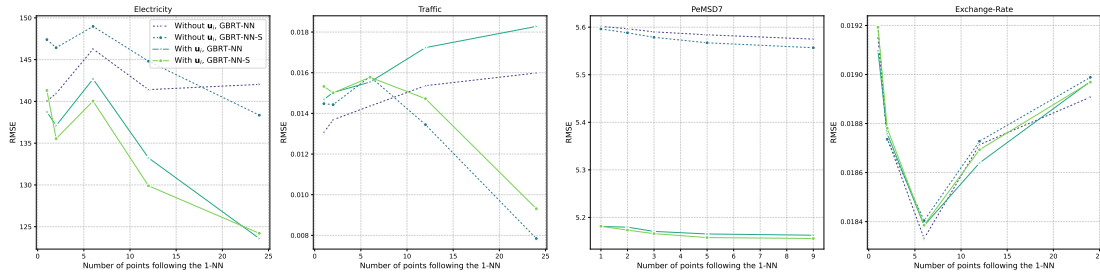
Dataset	Metric	GBRT	GBRT-NN	GBRT-NN-S	GBRT-NN ⁺	GBRT-NN ⁺ -S	GBRT-NN ⁺⁺	GBRT-NN ⁺⁺ -S
Electricity	RMSE	136.254	142.035	<u>138.354</u>	146.399	141.019	143.645	142.546
	WAPE	0.103	<u>0.101</u>	0.100	0.102	0.100	<u>0.101</u>	0.100
	MAE	57.929	56.464	55.972	56.898	56.019	56.401	<u>55.990</u>
Traffic	RMSE	<u>0.012</u>	0.016	0.008	0.016	0.008	0.015	0.008
	WAPE	0.108	0.079	<u>0.037</u>	0.075	<u>0.037</u>	0.064	0.035
	MAE	0.006	0.004	0.002	0.004	0.002	<u>0.003</u>	0.002
PeMSD7	RMSE	5.610	5.575	5.557	5.571	<u>5.554</u>	5.564	5.547
	WAPE	0.051	0.051	0.051	0.051	0.051	0.051	0.051
	MAE	2.984	2.965	2.957	2.963	<u>2.955</u>	2.959	2.951
Exchange-Rate	RMSE	<u>0.020</u>	0.019	0.019	0.019	0.019	0.021	0.021
	WAPE	<u>0.016</u>	0.015	0.015	0.015	0.015	<u>0.016</u>	<u>0.016</u>
	MAE	<u>0.012</u>	0.011	<u>0.012</u>	<u>0.012</u>	<u>0.012</u>	<u>0.012</u>	<u>0.012</u>

Table 5.5: Experimental Results with known input time-covariates.

Dataset	Metric	GBRT	GBRT-NN	GBRT-NN-S	GBRT-NN ⁺	GBRT-NN ⁺ -S	GBRT-NN ⁺⁺	GBRT-NN ⁺⁺ -S
Electricity	RMSE	132.669	123.591	<u>124.215</u>	130.869	130.428	125.822	129.078
	WAPE	0.093	0.089	0.089	<u>0.090</u>	<u>0.090</u>	0.089	<u>0.090</u>
	MAE	52.023	49.606	<u>49.739</u>	50.385	50.671	50.112	50.342
Traffic	RMSE	<u>0.014</u>	0.018	0.009	0.019	0.009	0.019	0.009
	WAPE	0.109	0.106	0.062	0.102	<u>0.061</u>	0.096	0.058
	MAE	0.006	0.006	0.003	0.006	0.003	<u>0.005</u>	0.003
PeMSD7	RMSE	5.191	5.163	5.156	5.161	<u>5.148</u>	5.152	5.145
	WAPE	0.048	0.048	0.048	0.048	0.048	0.048	0.048
	MAE	2.796	2.779	2.778	2.780	<u>2.773</u>	2.774	2.771
Exchange-Rate	RMSE	<u>0.020</u>	0.019	0.019	0.019	0.019	0.021	0.021
	WAPE	<u>0.016</u>	0.015	0.015	0.015	0.015	<u>0.016</u>	<u>0.016</u>
	MAE	0.012	0.012	0.012	0.012	0.012	0.012	0.012

5.5.1 Effect of the length l of the immediate subsequence

The effect of the length l of the immediate subsequence is investigated. We tested for five different lengths $l \in 1, 2, \lceil h/4 \rceil, \lceil h/2 \rceil, h$. For Exchange-Rate with $h = 9$, $l \in \{1, 2, 6, 12, 24\}$. For other datasets with $h = 24$, $l \in \{1, 2, 3, 5, 9\}$. The

Figure 5.3: Effect of different lengths l of the immediate subsequence (i.e., Number of points following the 1-NN) on RMSE.

results are plotted in Figures 5.3, 5.4. Since the WAPE plot is similar to the MAE plot, only the RMSE and WAPE plots are shown here.

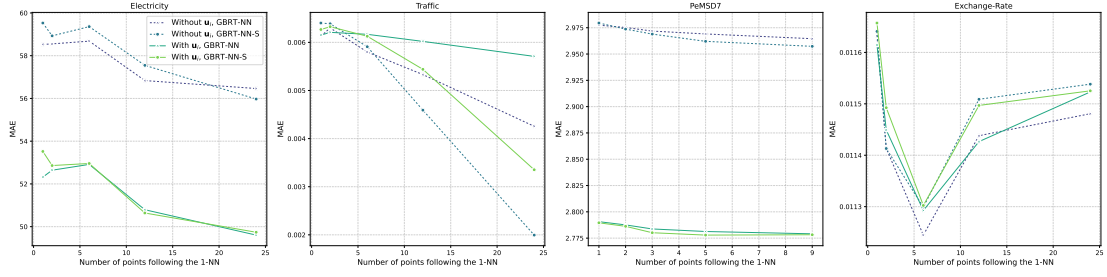


Figure 5.4: Effect of different lengths l of the immediate subsequence (i.e., Number of points following the 1-NN) on MAE.

For **Electricity** and **PeMSD7**, the performance improves for all the metrics as l increases. For **Traffic**, the performance of **GBRT-NN-S** on RMSE improves as l increases. In contrast, the performance of **GBRT-NN** on RMSE decays as l increases. But for MAE, both models improve as l increases. For **Exchange-Rate**, both models achieve the best result when $l = 6$ for all the metrics.

5.5.2 Comparison between the two scenarios

For **Electricity** and **PeMSD7**, the models perform better when time-covariates are incorporated. For **Traffic**, incorporating the covariates worsens the results. For **Exchange-Rate**, the performances of the models are similar between the two scenarios. This indicates that this dataset is less dependent on time.

5.5.3 Effect of using z-normalized instead of raw values

In this part, we study the effect of using z-normalized target values of the immediate subsequences. By default, we use the target values directly as the features. We have tested using the z-normalized representation of the target values instead of the raw values in all the previous experiments. In detail, for each dataset, there are two scenarios, and the two models have been tested five times for five different l . It results in 20 combinations in the experimental setting. The percentage change of using the z-normalized representation instead is shown in Figure 5.5 It shows that using the raw values of the immediate subsequence is preferable.

5.5.4 Effect of using pairwise difference representation instead of raw values

In this part, we study the effect of using pairwise differences, which captures the trend, as the features. We have tested using the pairwise difference representation

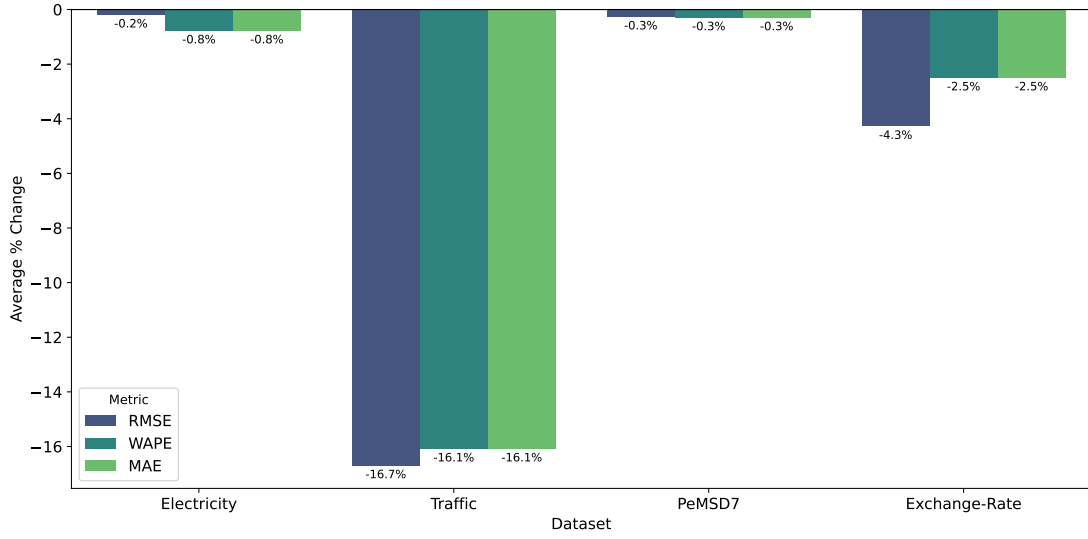


Figure 5.5: Using the z-normalized values instead of the raw values.

of the target values instead of the raw values in all the previous experiments. For a given time series $T = t_1, t_2, \dots, t_n$, its pairwise differences representation T' refers to $T' = t_2 - t_1, t_3 - t_2, \dots, t_n - t_{n-1}$. To note, the first entry is computed by subtracting the first entry of the immediate subsequence from the last entry of the nearest neighbor. Hence, the length of the pairwise difference representation is the same as the original one. In detail, for each dataset, there are two scenarios, and the two models have been tested five times for five different l . It results in 20 combinations in the experimental setting. Same as the previous part, it has been tested for 20 combinations for using the pairwise difference representation instead. The result is plotted on Figure 5.6. It shows that using the raw values of the immediate subsequence is preferable.

5.5.5 Summary of the experiments on the two primary models

Based on the above experiments, we present the following summary for the downstream analysis.

1. **Variable Impact of Covariates:** The inclusion of time-covariates does not always help. It depends on the dataset.
2. **Lag Selection:** We set $l = 6$ for `Exchange-Rate` and $l = h$ for other datasets.

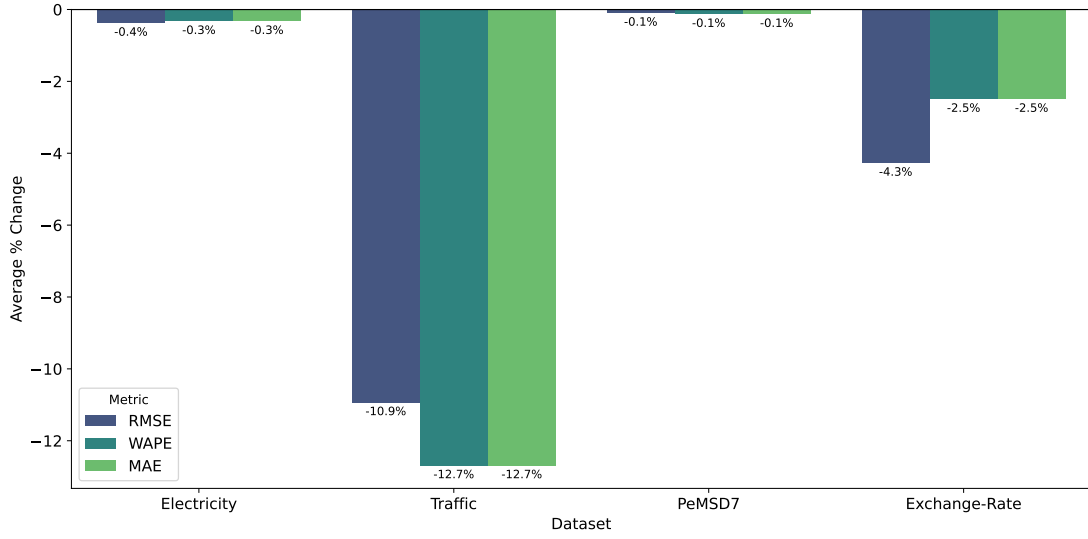


Figure 5.6: Using the pairwise differences instead of the raw values.

3. **Data Representation:** Using raw values for the immediate subsequences. We argue that it is because raw values preserve original information.
4. Across 12 evaluation combinations (four datasets with three metrics), **GBRT-NN-S** demonstrated improvement in 9 cases, remained consistent in 2, and showed a performance decline in only 1 when incorporating time-covariates, while showing improvement in 10 cases, and remained consistent in 2 for not incorporating known input covariates.
5. **GBRT-NN-S** performs much better than **GBRT-NN** in **Traffic** and performs similarly on the other datasets in both scenarios.
6. **Exchange-Rate** only has a mild improvement on our models. It indicates that this dataset is not strongly dependent on historically similar occurrences.

5.5.6 Incorporating the target values of the nearest neighbor

We have further incorporated the target values (either only the last point or all of them) of the nearest neighbor. The results are in Tables 5.4, 5.5. By incorporating these features, the models improve on **Traffic** and **PeMSD07**. **GBRT-NNC⁺⁺-S** performs the best for both the scenarios.

For **Electricity**, **GBRT-NNC⁺⁺-S** achieves the best and the second best on WAPE in the two scenarios, respectively. It worsens RMSE for the case without known input-time covariates compared to the baseline while still improve RMSE for the case with known input-time covariates. It indicates that in the absence of time covariates, **GBRT-NNC⁺⁺-S** makes wrong predictions for some outliers.

In general, these models improve average prediction performance but cannot handle certain outlier cases.

As mentioned above, **Exchange-Rate** is insensitive to long-range history. It can also be noted by the small best l in Figure 5.3, 5.4.

5.5.7 Incorporating the covariates of the last point of the nearest neighbor

Table 5.6: Experimental Results with known input time-covariates for models incorporating the known input time-covariates of the last point of the nearest neighbors.

Dataset	Metric	GBRT-NNC	GBRT-NNC-S	GBRT-NNC ⁺	GBRT-NNC ⁺ -S	GBRT-NNC ⁺⁺	GBRT-NNC ⁺⁺ -S
Electricity	RMSE	131.486	125.806	132.076	129.145	130.269	<u>127.217</u>
	WAPE	<u>0.090</u>	0.089	0.091	<u>0.090</u>	<u>0.090</u>	<u>0.090</u>
	MAE	50.576	49.928	50.808	50.494	50.490	<u>50.240</u>
Traffic	RMSE	0.014	<u>0.010</u>	0.014	0.009	0.014	<u>0.010</u>
	WAPE	0.098	0.068	0.093	<u>0.067</u>	0.085	0.066
	MAE	<u>0.005</u>	0.004	<u>0.005</u>	0.004	<u>0.005</u>	0.004
PeMSD7	RMSE	5.145	5.135	5.141	<u>5.132</u>	5.134	5.126
	WAPE	<u>0.048</u>	0.047	0.047	0.047	0.047	0.047
	MAE	2.771	2.764	2.767	<u>2.763</u>	2.766	2.759
Exchange-Rate	RMSE	0.019	0.019	0.019	0.019	<u>0.021</u>	<u>0.021</u>
	WAPE	0.015	0.015	0.015	0.015	<u>0.016</u>	<u>0.016</u>
	MAE	0.012	0.012	0.012	0.012	0.012	0.012

We also incorporate the covariates of the last point in the nearest-neighbor. The results is shown in Table 5.6. **PeMSD7** achieves better performance in these models and reaches the best at this moment for using **GBRT-NNC⁺⁺-S**. It does not improve the performance for the other datasets.

5.5.8 Extend to the case of k-nearest neighbor

Our idea has been extended to the case for k -nearest neighbors. We have tested for $k \in 2, 3$ instead of $k = 1$, which has already demonstrated in the previous parts. The results are shown in Tables 5.7, 5.8, 5.9.

Table 5.7: Experimental Results without known input time-covariates in the k -nearest neighbors case, where $k \in \{2, 3\}$.

Dataset	Metric	GBRT-2NN-S	GBRT-3NN-S	GBRT-2NN ⁺ -S	GBRT-3NN ⁺ -S	GBRT-2NN ⁺⁺ -S	GBRT-3NN ⁺⁺ -S
Electricity	RMSE	147.895	<u>147.706</u>	148.846	149.191	148.035	145.004
	WAPE	0.102	0.101	0.101	0.101	<u>0.100</u>	0.099
	MAE	56.313	56.032	56.126	56.155	<u>55.565</u>	54.895
Traffic	RMSE	0.008	<u>0.009</u>	0.008	<u>0.009</u>	0.008	0.008
	WAPE	0.039	<u>0.038</u>	0.039	0.040	0.037	<u>0.038</u>
	MAE	0.002	0.002	0.002	0.002	0.002	0.002
PeMSD7	RMSE	5.536	5.529	5.533	5.527	<u>5.525</u>	5.514
	WAPE	<u>0.051</u>	<u>0.051</u>	<u>0.051</u>	0.050	0.050	0.050
	MAE	2.946	2.943	2.943	2.940	<u>2.939</u>	2.932
Exchange-Rate	RMSE	0.019	0.019	0.019	0.019	0.019	0.019
	WAPE	0.016	0.016	0.016	0.016	0.016	0.016
	MAE	0.012	0.012	0.012	0.012	0.012	0.012

Table 5.8: Experimental Results with known input time-covariates in the k -nearest neighbors case, where $k \in \{2, 3\}$.

Dataset	Metric	GBRT-2NN-S	GBRT-3NN-S	GBRT-2NN ⁺ -S	GBRT-3NN ⁺ -S	GBRT-2NN ⁺⁺ -S	GBRT-3NN ⁺⁺ -S
Electricity	RMSE	133.017	132.646	132.450	<u>131.059</u>	129.694	132.146
	WAPE	<u>0.089</u>	<u>0.089</u>	<u>0.089</u>	0.088	0.088	0.088
	MAE	49.349	49.250	49.261	<u>48.917</u>	48.822	49.011
Traffic	RMSE	<u>0.010</u>	0.009	0.011	<u>0.010</u>	<u>0.010</u>	0.009
	WAPE	0.063	<u>0.060</u>	0.066	0.062	0.063	0.058
	MAE	0.003	0.003	<u>0.004</u>	0.003	0.003	0.003
PeMSD7	RMSE	5.137	<u>5.128</u>	5.133	5.127	5.127	5.116
	WAPE	0.047	0.047	0.047	0.047	0.047	0.047
	MAE	2.767	<u>2.761</u>	2.764	<u>2.761</u>	2.762	2.755
Exchange-Rate	RMSE	0.019	0.019	0.019	0.019	0.019	0.019
	WAPE	0.016	0.016	0.016	0.016	0.016	0.016
	MAE	0.012	0.012	0.012	0.012	0.012	0.012

Table 5.9: Experimental Results with known input time-covariates for models incorporating the known input time-covariates of the last point of the nearest neighbors in the k -nearest neighbors case, where $k \in \{2, 3\}$.

Dataset	Metric	GBRT-2NNC-S	GBRT-3NNC-S	GBRT-2NNC ⁺ -S	GBRT-3NNC ⁺ -S	GBRT-2NNC ⁺⁺ -S	GBRT-3NNC ⁺⁺ -S
Electricity	RMSE	133.426	132.127	132.043	131.639	<u>130.922</u>	130.684
	WAPE	<u>0.089</u>	<u>0.089</u>	<u>0.089</u>	0.088	0.088	0.088
	MAE	49.252	49.215	49.160	49.000	48.840	<u>48.932</u>
Traffic	RMSE	<u>0.012</u>	<u>0.012</u>	0.014	0.013	0.011	0.011
	WAPE	0.085	0.077	0.094	0.087	<u>0.074</u>	0.073
	MAE	<u>0.005</u>	0.004	<u>0.005</u>	<u>0.005</u>	0.004	0.004
PeMSD7	RMSE	5.109	5.100	5.110	<u>5.099</u>	5.106	5.090
	WAPE	0.047	0.047	0.047	0.047	0.047	0.047
	MAE	2.749	<u>2.745</u>	2.750	<u>2.745</u>	2.750	2.743
Exchange-Rate	RMSE	0.019	0.019	0.019	0.019	0.019	0.019
	WAPE	0.016	0.016	0.016	0.016	0.016	0.016
	MAE	0.012	0.012	0.012	0.012	0.012	0.012

Table 5.10: Best models for each dataset with (RMSE, WAPE, MAE) respectively.

Dataset	Without time-covariates	With time-covariates
Electricity	GBRT (136.254, -, -), GBRT-3NN ⁺⁺ -S (-, 0.099, 54.895)	GBRT-NN (123.591, -, -), GBRT-2NN ⁺⁺ -S (-, 0.088, 48.822)
Traffic	GBRT-NN ⁺⁺ -S (0.008, 0.035, 0.002)	GBRT-NN ⁺⁺ -S (0.009, 0.058, 0.003)
PeMSD7	GBRT-3-NN ⁺⁺ -S (5.514, 0.050, 2.932)	GBRT-3NNC ⁺⁺ -S (5.090, 0.047, 2.743)
Exchange-Rate	GBRT-NN (0.019, 0.015, 0.011)	GBRT-NN (0.019, 0.015, 0.012)

5.5.9 Summary of the experiment

We outline the best model for each dataset in Table 5.10. Note that in some datasets, no single model achieves the best performance across all three metrics. In this case, we report all of them. When there are ties, we report the simpler models (i.e., ones incorporating fewer covariates).

5.6 Concluding Remarks

In this study, we present a method to augment the original time series by finding nearest neighbors and using their information as covariates for each subsequence. The information includes the immediate subsequences, similarity, constituent points, and the covariates of the nearest neighbor. Experiment shows that the augmented data allow the GBRT model to have a better prediction result in the task of time series forecasting in terms of three evaluation metrics, which are RMSE, WAPE, and MAE.

5.6.1 Future Work

Leveraging nearest neighbors’ location information: It would be beneficial to retrieve the nearest neighbors for each subsequence in a specific range with respect to it. Real-world applications often require the separation of information of short-term and long-term repeating patterns for making accurate predictions [73]. Notably, the matrix profile also provides the locations of the nearest neighbors from the matrix profile index. Using this location (index) information, we can retrieve the nearest neighbors of each subsequence in a specified range with respect to it to find those “short-term” and “long-term” patterns.

Extend to multidimensional case: This study focuses on univariate time series forecasting, where there is a single target and no exogenous inputs. It only requires us to find one-dimensional nearest neighbors. When there are multiple targets or a single target with multiple exogenous inputs, we need to identify multidimensional nearest neighbors [110] and leverage their information for forecasting.

Identify outliers of immediate subsequences: When we have a set of immediate subsequences, as in the case of k -nearest neighbors, we can determine whether an immediate subsequence is an outlier or not by comparing it with others. Or we can compute the consensus immediate subsequences of them. In addition, we can also consider the similarity of each of the nearest neighbors as the weight of their corresponding immediate subsequences when computing the consensus immediate subsequences.

Chapter 6

Conclusion and Future Directions

In this thesis, we contribute to time series analysis by addressing three aspects, with applications in bioinformatics. The first is to frame the prediction of biological sequence problems as time series classification tasks. The second addresses a fundamental limitation in existing time series distance measures. The third is to augment the original data with the nearest neighbors of each subsequence. Models that are trained on the augmented data have a better result than models that are trained on the original data.

In the first study, we investigate the prediction of human Dicer cleavage sites. This task is important for the biogenesis of microRNAs (miRNAs). Accurate prediction of Dicer cleavage sites is crucial for elucidating mechanisms of post-transcriptional gene regulation. Computationally, this task is a classification problem. First, we curate the dataset based on existing studies. The resulting datasets are 14-strings. Then, they are transformed into time series. We employ ROCKET-based classifiers for the classification. The main contributions are summarized as follows. We are the first to frame this problem as a multivariate time series classification problem. We introduced nine encoding methods for the transformation. In the transformation, to our surprise, we are the first to use the base-pair probabilities derived from the predicted secondary structure. We employ state-of-the-art time-series classifiers, namely ROCKET-based classifiers. They use random convolutional kernels to generate the summary statistics and then use a simple ridge classifier to generate the final results. Because of the simplicity of the transformation method and the classifiers we adopted, our framework, namely MTSCCleave, is fast. It achieved predictive performance comparable to or even better than deep learning-based state-of-the-art methods. Furthermore, MTSCCleave demonstrated substantial computational efficiency, with speedups ranging from 3.7X to 28.8X relative to existing methods. We carried out perturbation-based experi-

ments to identify the subsequence that are important for the classification. We found that regions near the center of the pre-miRNA secondary structure are most critical for Dicer cleavage site determination. It aligns with the existing study. Future work for this study is as follows. We make use of the predicted secondary structure information to construct the complementary strand and the base pair probability sequence for the input strand. However, there is more than one predicted secondary structure for the given RNA sequence. One future work is to make use of all potential secondary structures, each with its own pair probability sequence, and encode this data into a multivariate time series with more channels. Another area for future work is to use interpretable time series classifiers, such as those based on time series shapelets. By doing this, we can study which subsequence is critical for the definition of the classes, namely “5p cleav”, “5p non-cleav”, “3p cleav”, and “3p non-cleav” because shapelets serve as the subsequence that has the most discriminating power between classes.

The second study develops a new distance measure framework, namely PSD. It aims to release a fundamental assumption that the prior studies have overlooked. There is only one scaling rate throughout the entire time series. However, there are much data that exhibits multiple rates. For example, human motion or music performance. They consist of phases. Each phase has its own expression rate. Existing distance measures cannot account for such variations. For example, DTW is designed for handling local distortions. US assumes that there is only one scaling factor in the whole series. To address this, we introduced the Piecewise Scaling Distance (PSD) framework, the first of its kind to account for multiple scaling factors. Recall that PSD is agnostic to the base measure we used. We can use any existing distance measure as the base measure. We studied the two instantiations of it. They are PSED (using Euclidean Distance as the base measure) and PSDTW (using DTW as the base measure). We provided an exact dynamic programming solution to compute PSD and three general ways to speed it up. In particular, we proposed a constrained version of it that limits the search space based on allowed segment lengths derived from scaling factor bounds. Besides, we use parallel computing and early abandoning to further accelerate it. For PSDTW, due to its quadratic complexity, we can further speed it up using lower-bounding techniques. Experiments show that PSD, and in particular PSED, perform best when the query contains multi-rate distortions, compared with ED, DTW and the other five DTW-based methods.

Future work on it is as follows. Currently, the number of segments P is given by users. It is preferable to develop a heuristic or algorithmic approach to automat-

ically determine P . A simple heuristic is to test a range of possible P . Besides, while we have successfully applied a lower bound on PSDTW and accelerated it, the computational overhead of calculating a lower bound on PSED outweighs the pruning benefit and makes the computation slower eventually. Developing a specialized lower bound for PSED could further improve its running time. In addition, some existing works on speeding up DTW such as “incremental DTW” can reuse the accumulated cost matrix D . However, it is challenging to apply a similar method to USDTW and PSDTW due to time series interpolation.

The third study develops a new method to create covariates for time series data using the immediate subsequences of the left nearest neighbor of each forecasting windows. We have studied the effect of the length of the immediate subsequences on the forecaster. Future work of this study includes finding nearest neighbors in a specific range to capture the short pattern and long pattern, extending the framework to a multivariate time series forecasting task, using motif family instead of simply top- k nearest neighbors, and handling the outliers among the immediate subsequences.

Bibliography

- [1] M. Middlehurst, P. Schäfer, and A. Bagnall, “Bake off redux: A review and experimental evaluation of recent time series classification algorithms,” *Data Mining and Knowledge Discovery*, vol. 38, no. 4, pp. 1958–2031, Jul. 2024.
- [2] H. Sakoe and S. Chiba, “Dynamic programming algorithm optimization for spoken word recognition,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, Feb. 1978.
- [3] E. J. Keogh and M. J. Pazzani, “Derivative dynamic time warping,” in *Proceedings of the 2001 SIAM International Conference on Data Mining (SDM)*, ser. Proceedings. Society for Industrial and Applied Mathematics, Apr. 2001, pp. 1–11.
- [4] Y.-S. Jeong, M. K. Jeong, and O. A. Omitaomu, “Weighted dynamic time warping for time series classification,” *Pattern Recognition*, vol. 44, no. 9, pp. 2231–2240, Sep. 2011.
- [5] J. Zhao and L. Itti, “shapedtw: Shape dynamic time warping,” *Pattern Recognition*, vol. 74, pp. 171–184, Feb. 2018.
- [6] E. Keogh, K. Chakrabarti, M. Pazzani, and S. Mehrotra, “Dimensionality reduction for fast similarity search in large time series databases,” *Knowledge and Information Systems*, vol. 3, no. 3, pp. 263–286, Aug. 2001.
- [7] B.-K. Yi and C. Faloutsos, “Fast time sequence indexing for arbitrary lp norms,” in *Proceedings of the 26th International Conference on Very Large Data Bases*, ser. VLDB ’00. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., Sep. 2000, pp. 385–394.
- [8] J. Zhao and L. Itti, “Decomposing time series with application to temporal segmentation,” in *2016 IEEE Winter Conference on Applications of Computer Vision (WACV)*, Mar. 2016, pp. 1–9.

- [9] M. Herrmann and G. I. Webb, “Amercing: An intuitive and effective constraint for dynamic time warping,” *Pattern Recognition*, vol. 137, p. 109333, May 2023.
- [10] L. A. Urry, M. L. Cain, S. A. Wasserman, P. V. Minorsky, R. B. Orr, and N. A. Campbell, *Campbell Biology*, twelfth ed. New York, NY: Pearson, 2020.
- [11] B. Alberts, *Molecular Biology of the Cell*, 7th ed. New York: W. W. Norton & Company, 2022.
- [12] W. W. Cohen, *A Computer Scientist’s Guide to Cell Biology: A Travelogue from a Stranger in a Strange Land*. New York, NY: Springer-Verl, 2007.
- [13] Y. Lee, K. Jeon, J.-T. Lee, S. Kim, and V. N. Kim, “MicroRNA maturation: Stepwise processing and subcellular localization,” *The EMBO Journal*, vol. 21, no. 17, pp. 4663–4670, Sep. 2002.
- [14] S. Gu, L. Jin, Y. Zhang, Y. Huang, F. Zhang, P. N. Valdmann, and M. A. Kay, “The loop position of shrnas and pre-mirnas is critical for the accuracy of dicer processing in vivo,” *Cell*, vol. 151, no. 4, pp. 900–911, Nov. 2012.
- [15] Y. Feng, X. Zhang, P. Graves, and Y. Zeng, “A comprehensive analysis of precursor microRNA cleavage by human dicer,” *RNA*, vol. 18, no. 11, pp. 2083–2092, Jan. 2012.
- [16] I. J. MacRae, K. Zhou, and J. A. Doudna, “Structural determinants of rna recognition and cleavage by dicer,” *Nature Structural & Molecular Biology*, vol. 14, no. 10, pp. 934–940, Oct. 2007.
- [17] F. Ahmed, R. Kaundal, and G. P. Raghava, “Phdcleav: A svm based method for predicting human dicer cleavage sites using sequence and secondary structure of mirna precursors,” *BMC Bioinformatics*, vol. 14, no. 14, p. S9, Oct. 2013.
- [18] Y. Bao, M. Hayashida, and T. Akutsu, “Lbsizecleav: Improved support vector machine (svm)-based prediction of dicer cleavage sites using loop/bulge length,” *BMC Bioinformatics*, vol. 17, no. 1, p. 487, Nov. 2016.
- [19] P. Liu, J. Song, C.-Y. Lin, and T. Akutsu, “Recgbm: A gradient boosting-based method for predicting human dicer cleavage sites,” *BMC Bioinformatics*, vol. 22, no. 1, p. 63, Feb. 2021.

- [20] L. Mu, J. Song, T. Akutsu, and T. Mori, “Dicleave: A deep learning model for predicting human dicer cleavage sites,” *BMC Bioinformatics*, vol. 25, no. 1, p. 13, Jan. 2024.
- [21] L. Mu and T. Akutsu, “Dicleaveplus: A transformer-based model to detect human dicer cleavage sites within cleavage patterns,” *Genes to Cells*, vol. 31, no. 1, p. e70074, 2026.
- [22] S. Griffiths-Jones, H. K. Saini, and S. van Dongen, “mirbase: Tools for microrna genomics,” *Nucleic Acids Research*, vol. 36, no. suppl_1, pp. D154–D158, Jan. 2008.
- [23] T. Xu, N. Su, L. Liu, J. Zhang, H. Wang, W. Zhang, J. Gui, K. Yu, J. Li, and T. D. Le, “mirbaseconverter: An r/bioconductor package for converting and retrieving mirna name, accession, sequence and family information in different versions of mirbase,” *BMC Bioinformatics*, vol. 19, no. 19, p. 514, Dec. 2018.
- [24] M. J. Zvelebil, J. O. Baum, and M. Zvelebil, *Understanding Bioinformatics*. New York: Garland Science, 2008.
- [25] R. Lorenz, S. H. Bernhart, C. Höner zu Siederdissen, H. Tafer, C. Flamm, P. F. Stadler, and I. L. Hofacker, “Viennarna package 2.0,” *Algorithms for Molecular Biology*, vol. 6, no. 1, p. 26, Nov. 2011.
- [26] C. C. Aggarwal, *Data Mining: The Textbook*. Cham: Springer International Publishing, 2015.
- [27] G. Mendizabal-Ruiz, I. Román-Godínez, S. Torres-Ramos, R. A. Salido-Ruiz, and J. A. Morales, “On dna numerical representations for genomic similarity computation,” *PLOS ONE*, vol. 12, no. 3, p. e0173288, Mar. 2017.
- [28] D. Anastassiou, “Genomic signal processing,” *IEEE Signal Processing Magazine*, vol. 18, no. 4, pp. 8–20, Jul. 2001.
- [29] T. Rakthanmanon, B. Campana, A. Mueen, G. Batista, B. Westover, Q. Zhu, J. Zakaria, and E. Keogh, “Searching and mining trillions of time series subsequences under dynamic time warping,” in *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’12. New York, NY, USA: Association for Computing Machinery, Aug. 2012, pp. 262–270.

- [30] P. D. Cristea, “Conversion of nucleotides sequences into genomic signals,” *Journal of Cellular and Molecular Medicine*, vol. 6, no. 2, pp. 279–303, 2002.
- [31] N. Chakravarthy, A. Spanias, L. D. Iasemidis, and K. Tsakalis, “Autoregressive modeling and feature analysis of dna sequences,” *EURASIP Journal on Advances in Signal Processing*, vol. 2004, no. 1, pp. 1–16, Dec. 2004.
- [32] J. Zhao, X. W. Yang, J. P. Li, and Y. Y. Tang, “Dna sequences classification based on wavelet packet analysis,” in *Wavelet Analysis and Its Applications*. Springer, Berlin, Heidelberg, 2001, pp. 424–429.
- [33] T. Holden, R. Subramaniam, R. Sullivan, E. Cheung, C. Schneider, G. T. Jr, A. Flamholz, D. H. Lieberman, and T. D. Cheung, “Atcg nucleotide fluctuation of deinococcus radiodurans radiation genes,” in *Instruments, Methods, and Missions for Astrobiology X*, vol. 6694. SPIE, Oct. 2007, pp. 402–411.
- [34] A. S. Nair and S. P. Sreenadhan, “A coding measure scheme employing electron-ion interaction pseudopotential (eiip),” *Bioinformation*, vol. 1, no. 6, pp. 197–202, Oct. 2006.
- [35] M. Akhtar, J. Epps, and E. Ambikairajah, “On dna numerical representations for period-3 based exon prediction,” in *2007 IEEE International Workshop on Genomic Signal Processing and Statistics*, Jun. 2007, pp. 1–4.
- [36] R. Zhang and C.-T. Zhang, “Z curves, an intuitive tool for visualizing and analyzing the dna sequences,” *Journal of Biomolecular Structure and Dynamics*, vol. 11, no. 4, pp. 767–782, Feb. 1994.
- [37] J. A. Berger, S. K. Mitra, M. Carli, and A. Neri, “Visualization and analysis of dna sequences using dna walks,” *Journal of the Franklin Institute*, vol. 341, no. 1, pp. 37–53, Jan. 2004.
- [38] A. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh, “The great time series classification bake off: A review and experimental evaluation of recent algorithmic advances,” *Data Mining and Knowledge Discovery*, vol. 31, no. 3, pp. 606–660, May 2017.
- [39] A. P. Ruiz, M. Flynn, J. Large, M. Middlehurst, and A. Bagnall, “The great multivariate time series classification bake off: A review and experimental evaluation of recent algorithmic advances,” *Data Mining and Knowledge Discovery*, vol. 35, no. 2, pp. 401–449, Mar. 2021.

- [40] A. Dempster, F. Petitjean, and G. I. Webb, “Rocket: Exceptionally fast and accurate time series classification using random convolutional kernels,” *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, Sep. 2020.
- [41] A. Dempster, D. F. Schmidt, and G. I. Webb, “Minirocket: A very fast (almost) deterministic transform for time series classification,” in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, ser. KDD ’21. New York, NY, USA: Association for Computing Machinery, Aug. 2021, pp. 248–257.
- [42] C. W. Tan, A. Dempster, C. Bergmeir, and G. I. Webb, “Multirocket: Multiple pooling operators and transformations for fast and effective time series classification,” *Data Mining and Knowledge Discovery*, vol. 36, no. 5, pp. 1623–1646, Sep. 2022.
- [43] A. Dempster, D. F. Schmidt, and G. I. Webb, “Hydra: Competing convolutional kernels for fast and accurate time series classification,” *Data Mining and Knowledge Discovery*, vol. 37, no. 5, pp. 1779–1805, Sep. 2023.
- [44] B. W. Matthews, “Comparison of the predicted and observed secondary structure of t4 phage lysozyme,” *Biochimica et Biophysica Acta (BBA) - Protein Structure*, vol. 405, no. 2, pp. 442–451, Oct. 1975.
- [45] J. Gorodkin, “Comparing two k-category assignments by a k-category correlation coefficient,” *Computational Biology and Chemistry*, vol. 28, no. 5, pp. 367–374, Dec. 2004.
- [46] T.-c. Fu, “A review on time series data mining,” *Engineering Applications of Artificial Intelligence*, vol. 24, no. 1, pp. 164–181, Feb. 2011.
- [47] C. Rose, B. Guenter, B. Bodenheimer, and M. F. Cohen, “Efficient generation of motion transitions using spacetime constraints,” in *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH ’96. New York, NY, USA: Association for Computing Machinery, Aug. 1996, pp. 147–154.
- [48] Y. Li, T. Wang, and H.-Y. Shum, “Motion texture: A two-level statistical model for character motion synthesis,” *ACM Trans. Graph.*, vol. 21, no. 3, pp. 465–472, Jul. 2002.

- [49] R. B. Dannenberg, W. P. Birmingham, G. P. Tzanetakis, C. P. Meek, N. P. Hu, and B. P. Pardo, “The musart testbed for query-by-humming evaluation,” *Comput. Music J.*, vol. 28, no. 2, pp. 34–48, Jun. 2004.
- [50] K.-C. Li, M. Yan, and S. Yuan, “A simple statistical model for depicting the cdc15-synchronized yeast cell-cycle regulated gene expression data,” *Statistica Sinica*, vol. 12, no. 1, pp. 141–158, 2002.
- [51] G. E. A. P. A. Batista, E. J. Keogh, O. M. Tataw, and V. M. A. de Souza, “Cid: An efficient complexity-invariant distance for time series,” *Data Mining and Knowledge Discovery*, vol. 28, no. 3, pp. 634–669, May 2014.
- [52] A. W.-C. Fu, E. Keogh, L. Y. H. Lau, C. A. Ratanamahatana, and R. C.-W. Wong, “Scaling and time warping in time series querying,” *The VLDB Journal*, vol. 17, no. 4, pp. 899–921, Jul. 2008.
- [53] Y. Shen, Y. Chen, E. Keogh, and H. Jin, “Searching time series with invariance to large amounts of uniform scaling,” in *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*, Apr. 2017, pp. 111–114.
- [54] ———, “Accelerating time series searching with large uniform scaling,” in *Proceedings of the 2018 SIAM International Conference on Data Mining (SDM)*, ser. Proceedings. Society for Industrial and Applied Mathematics, May 2018, pp. 234–242.
- [55] C. C. Aggarwal and C. K. Reddy, Eds., *Data Clustering: Algorithms and Applications*, 1st ed. Boca Raton: Chapman and Hall/CRC, Aug. 2013.
- [56] P. Esling and C. Agon, “Time-series data mining,” *ACM Computing Surveys*, vol. 45, no. 1, pp. 1–34, Nov. 2012.
- [57] T. Rakthanmanon, B. Campana, A. Mueen, G. Batista, B. Westover, Q. Zhu, J. Zakaria, and E. Keogh, “Addressing big data time series: Mining trillions of time series subsequences under dynamic time warping,” *ACM Trans. Knowl. Discov. Data*, vol. 7, no. 3, pp. 10:1–10:31, Sep. 2013.
- [58] C. W. Tan, F. Petitjean, and G. I. Webb, “Elastic bands across the path: A new framework and method to lower bound dtw,” in *Proceedings of the 2019 SIAM International Conference on Data Mining (SDM)*, ser. Proceedings. Society for Industrial and Applied Mathematics, May 2019, pp. 522–530.

- [59] A. Shifaz, C. Pelletier, F. Petitjean, and G. I. Webb, “Elastic similarity and distance measures for multivariate time series,” *Knowledge and Information Systems*, vol. 65, no. 6, pp. 2665–2698, Jun. 2023.
- [60] S. Salvador and P. Chan, “Toward accurate dynamic time warping in linear time and space,” *Intelligent Data Analysis*, vol. 11, no. 5, pp. 561–580, Oct. 2007.
- [61] E. Keogh, T. Palpanas, V. B. Zordan, D. Gunopulos, and M. Cardle, “Indexing large human-motion databases,” in *Proceedings of the Thirtieth International Conference on Very Large Data Bases - Volume 30*, ser. VLDB ’04. Toronto, Canada: VLDB Endowment, Aug. 2004, pp. 780–791.
- [62] D. Yankov, E. Keogh, J. Medina, B. Chiu, and V. Zordan, “Detecting time series motifs under uniform scaling,” in *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’07. New York, NY, USA: Association for Computing Machinery, Aug. 2007, pp. 844–853.
- [63] Y. Zhu and D. Shasha, “Warping indexes with envelope transforms for query by humming,” in *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’03. New York, NY, USA: Association for Computing Machinery, Jun. 2003, pp. 181–192.
- [64] F. Itakura, “Minimum prediction residual principle applied to speech recognition,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 23, no. 1, pp. 67–72, Feb. 1975.
- [65] E. Keogh and C. A. Ratanamahatana, “Exact indexing of dynamic time warping,” *Knowledge and Information Systems*, vol. 7, no. 3, pp. 358–386, Mar. 2005.
- [66] S.-W. Kim, S. Park, and W. Chu, “An index-based approach for similarity search supporting time warping in large sequence databases,” in *Proceedings 17th International Conference on Data Engineering*, Apr. 2001, pp. 607–614.
- [67] C. W. Tan, M. Herrmann, G. Forestier, G. I. Webb, and F. Petitjean, “Efficient search of the best warping window for dynamic time warping,” in *Proceedings of the 2018 SIAM International Conference on Data Mining (SDM)*, ser. Proceedings. Society for Industrial and Applied Mathematics, May 2018, pp. 225–233.

- [68] C. A. Ratanamahatana and E. Keogh, “Everything you know about dynamic time warping is wrong,” in *3 Rd International Workshop on Mining Temporal and Sequential Data (TDM-04)*. Citeseer, 2004.
- [69] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, “The ucr time series archive,” *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, Nov. 2019.
- [70] M. Middlehurst, A. Ismail-Fawaz, A. Guillaume, C. Holder, D. Guijo-Rubio, G. Bulatova, L. Tsaprounis, L. Mentel, M. Walter, P. Schäfer, and A. Bagnall, “Aeon: A python toolkit for learning from time series,” *Journal of Machine Learning Research*, vol. 25, no. 289, pp. 1–10, 2024.
- [71] N. Tatbul, T. J. Lee, S. Zdonik, M. Alam, and J. Gottschlich, “Precision and recall for time series,” in *Advances in Neural Information Processing Systems*, vol. 31. Curran Associates, Inc., 2018.
- [72] M. Leodolter, C. Plant, and N. Brändle, “Incdtw: An r package for incremental calculation of dynamic time warping,” *Journal of Statistical Software*, vol. 99, pp. 1–23, Sep. 2021.
- [73] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, “Modeling long- and short-term temporal patterns with deep neural networks,” in *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, ser. SIGIR ’18. New York, NY, USA: Association for Computing Machinery, Jun. 2018, pp. 95–104.
- [74] C. Chatfield and H. Xing, *The Analysis of Time Series: An Introduction with R*. Boca Raton: Chapman and Hall/CRC, 2019.
- [75] O. Y. Al-Jarrah, P. D. Yoo, S. Muhaidat, G. K. Karagiannidis, and K. Taha, “Efficient machine learning for big data: A review,” *Big Data Research*, vol. 2, no. 3, pp. 87–93, Sep. 2015.
- [76] W. Li and K. L. E. Law, “Deep learning models for time series forecasting: A review,” *IEEE Access*, vol. 12, pp. 92 306–92 327, 2024.
- [77] N. I. Sapankevych and R. Sankar, “Time series prediction using support vector machines: A survey,” *IEEE Computational Intelligence Magazine*, vol. 4, no. 2, pp. 24–38, May 2009.

- [78] Z. Chen, M. Ma, T. Li, H. Wang, and C. Li, “Long sequence time-series forecasting with deep learning: A survey,” *Information Fusion*, vol. 97, p. 101819, Sep. 2023.
- [79] J. F. Torres, D. Hadjout, A. Sebaa, F. Martínez-Álvarez, and A. Troncoso, “Deep learning for time series forecasting: A survey,” *Big Data*, vol. 9, no. 1, pp. 3–21, Feb. 2021.
- [80] D. B. da Silva, D. Schmidt, C. A. da Costa, R. da Rosa Righi, and B. Eskofier, “Deepsigns: A predictive model based on deep learning for the early detection of patient health deterioration,” *Expert Systems with Applications*, vol. 165, p. 113905, Mar. 2021.
- [81] N. Wu, B. Green, X. Ben, and S. O’Banion, “Deep transformer models for time series forecasting: The influenza prevalence case,” Jan. 2020.
- [82] F. Martínez-Álvarez, G. Asencio-Cortés, J. F. Torres, D. Gutiérrez-Avilés, L. Melgar-García, R. Pérez-Chacón, C. Rubio-Escudero, J. C. Riquelme, and A. Troncoso, “Coronavirus optimization algorithm: A bioinspired meta-heuristic based on the covid-19 propagation model,” *Big Data*, vol. 8, no. 4, pp. 308–322, Aug. 2020.
- [83] S. Elsayed, D. Thyssens, A. Rashed, H. S. Jomaa, and L. Schmidt-Thieme, “Do we really need deep learning models for time series forecasting?” Oct. 2021.
- [84] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. Hoboken, New Jersey: Wiley, 2015.
- [85] G. E. P. Box and D. A. Pierce, “Distribution of residual autocorrelations in autoregressive-integrated moving average time series models,” *Journal of the American Statistical Association*, vol. 65, no. 332, pp. 1509–1526, 1970.
- [86] B. Lim and S. Zohren, “Time-series forecasting with deep learning: A survey,” *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 379, no. 2194, p. 20200209, Feb. 2021.
- [87] G. U. Yule, “On a method of investigating periodicities in disturbed series, with special reference to wolfer’s sunspot numbers,” *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, vol. 226, pp. 267–298, 1927.

- [88] S. Tajmouati, B. E. L. Wahbi, A. Bedoui, A. Abarda, and M. Dakkon, “Applying k-nearest neighbors to time series forecasting: Two new approaches,” *Journal of Forecasting*, vol. 43, no. 5, pp. 1559–1574, 2024.
- [89] F. Martínez, M. P. Frías, M. D. Pérez, and A. J. Rivera, “A methodology for applying k-nearest neighbor to time series forecasting,” *Artificial Intelligence Review*, vol. 52, no. 3, pp. 2019–2037, Oct. 2019.
- [90] D. T. Thi, N. T. Phong, and N. D. Bui, “Forecast timeseries based on matrix profile,” *Journal of Informatics and Innovative Technologies (JIIT)*, 2021.
- [91] E. Cartwright, M. Crane, and H. J. Ruskin, “Financial time series: Market analysis techniques based on matrix profiles,” *Engineering Proceedings*, vol. 5, no. 1, p. 45, 2021.
- [92] Q. Liu, D. L. X. Fung, L. Lac, and P. Hu, “A novel matrix profile-guided attention lstm model for forecasting covid-19 cases in usa,” *Frontiers in Public Health*, vol. 9, Oct. 2021.
- [93] P. Saha, P. Nath, A. I. Middya, and S. Roy, “Improving temporal predictions through time-series labeling using matrix profile and motifs,” *Neural Computing and Applications*, vol. 34, no. 16, pp. 13 169–13 185, Aug. 2022.
- [94] R. Sen, H.-F. Yu, and I. S. Dhillon, “Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting,” in *Advances in Neural Information Processing Systems*, vol. 32. Curran Associates, Inc., 2019.
- [95] A. Mueen and E. Keogh, “Online discovery and maintenance of time series motifs,” in *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’10. New York, NY, USA: Association for Computing Machinery, Jul. 2010, pp. 1089–1098.
- [96] C.-C. M. Yeh, Y. Zhu, L. Ulanova, N. Begum, Y. Ding, H. A. Dau, D. F. Silva, A. Mueen, and E. Keogh, “Matrix profile i: All pairs similarity joins for time series: A unifying view that includes motifs, discords and shapelets,” in *2016 IEEE 16th International Conference on Data Mining (ICDM)*. Barcelona, Spain: IEEE, Dec. 2016, pp. 1317–1322.
- [97] S. M. Law, “Stumpy: A powerful and scalable python library for time series data mining,” *Journal of Open Source Software*, vol. 4, no. 39, p. 1504, Jul. 2019.

- [98] S. Zhong and A. Mueen, “Mass: Distance profile of a query over a time series,” *Data Mining and Knowledge Discovery*, vol. 38, no. 3, pp. 1466–1492, May 2024.
- [99] Y. Zhu, Z. Zimmerman, N. S. Senobari, C.-C. M. Yeh, G. Funning, A. Mueen, P. Brisk, and E. Keogh, “Matrix profile ii: Exploiting a novel algorithm and gpus to break the one hundred million barrier for time series motifs and joins,” in *2016 IEEE 16th International Conference on Data Mining (ICDM)*. Barcelona, Spain: IEEE, Dec. 2016, pp. 739–748.
- [100] Y. Zhu, M. Imamura, D. Nikovski, and E. Keogh, “Matrix profile vii: Time series chains: A new primitive for time series data mining,” in *2017 IEEE International Conference on Data Mining (ICDM)*, Nov. 2017, pp. 695–704.
- [101] J. H. Friedman, “Greedy function approximation: A gradient boosting machine.” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001.
- [102] L. G. Serrano and S. Thrun, *Grokking Machine Learning*. Shelter Island: Manning, 2021.
- [103] A. Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems*. US: O’Reilly Media, 2025.
- [104] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16. New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794.
- [105] A. V. Dorogush, V. Ershov, and A. Gulin, “Catboost: Gradient boosting with categorical features support,” in *Workshop on ML Systems at NIPS 2017*, 2017.
- [106] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.
- [107] R. J. Williams and D. Zipser, “A learning algorithm for continually running fully recurrent neural networks,” *Neural Computation*, vol. 1, no. 2, pp. 270–280, Jun. 1989.

- [108] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, ser. Adaptive Computation and Machine Learning. Cambridge, Massachusetts London, England: The MIT Press, 2022.
- [109] S. Kolassa and W. Schütz, “Advantages of the mad/mean ratio over the mape,” *Foresight: The International Journal of Applied Forecasting*, vol. 6, pp. 40–43, 2007.
- [110] C.-C. M. Yeh, N. Kavantzaz, and E. Keogh, “Matrix profile iv: Using weakly labeled time series to predict outcomes,” *Proc. VLDB Endow.*, vol. 10, no. 12, pp. 1802–1812, Aug. 2017.